

Power Planning and Routing

- Power Planning:
 - Use the `addRing` command to generate rings with the CORERING and BLOCKRING shapes.
 - Use the `addStripe` command to generate stripes with the STRIPE shape. Use the `editPowerVia` command after the `addStripe` command to drop vias within macro power pins during stripe creation.
- Power Routing: Use the `sroute` command to connect to the following patterns:
 - `padPin` , `blockPin` , and `floatingStripe` : Connected nets are generated with the IOWIRE, BLOCKWIRE, and STRIPE shapes.
 - Standard cell pins and secondary power pins: Connected nets are generated with the FOLLOWPIN (in row area) and COREWIRE shapes (outside the row for further connection with other shapes).
 - `padPin` on left and right side of pad cells to generate `padding` : Connected nets are generated with the PADRING shape.
- VIAGEN Engine: Use it for special via insertion. The VIAGEN Engine:
 - Is called with the `addRing` , `addStripe` , and `sroute` commands to generate special vias.
 - Controls the behavior of special via editing command, `editPowerVia` .

Use Model of Power Planning and Routing Commands

The following table presents the use model of the power planning and routing commands:

Action Command	Mode Setup Command(s)
<code>addRing</code>	<code>setAddRingMode</code> <code>setViaGenMode</code>
<code>addStripe</code>	<code>setAddStripeMode</code> <code>setViaGenMode</code>
<code>sroute</code>	<code>setSrouteMode</code> <code>setViaGenMode</code>
<code>editPowerVia</code>	<code>setViaGenMode</code>

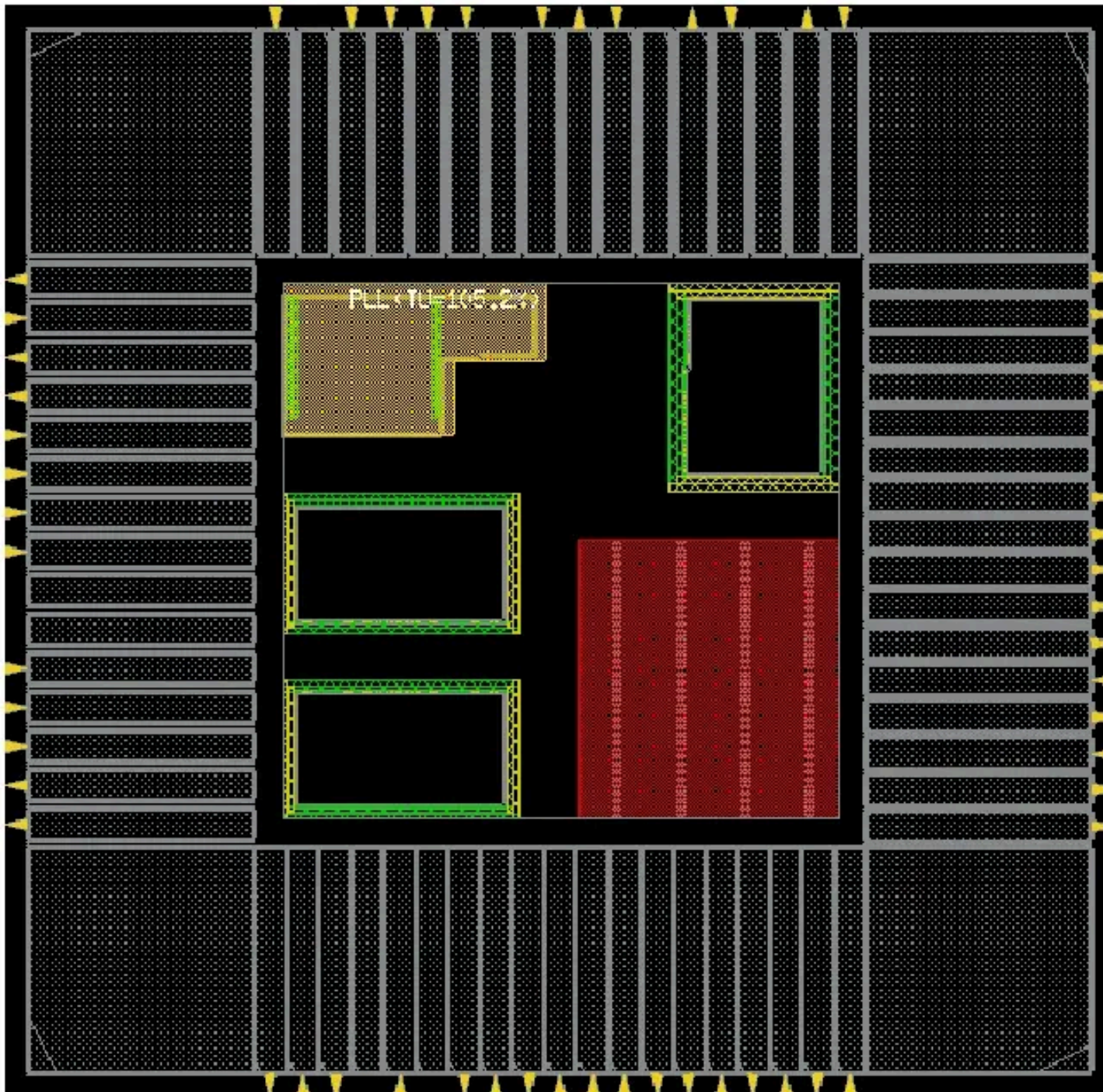
Each action command has one or two mode setup command(s) to control their characteristics. For non-default characteristics, you need to update/change the value of the mode setup command(s) before applying the action command. For example:

```
setViaGenMode -viarule_preference {VIAGEN67 VIAGEN56} -optimize_cross_via 1  
setAddStripeMode -stacked_via_bottom_layer M5 -stacked_via_top_layer M7  
addStripe -nets VDD -width 0.144 -spacing 0.441 -set_to_set_distance 2.34 -layer M6 -  
direction horizontal
```

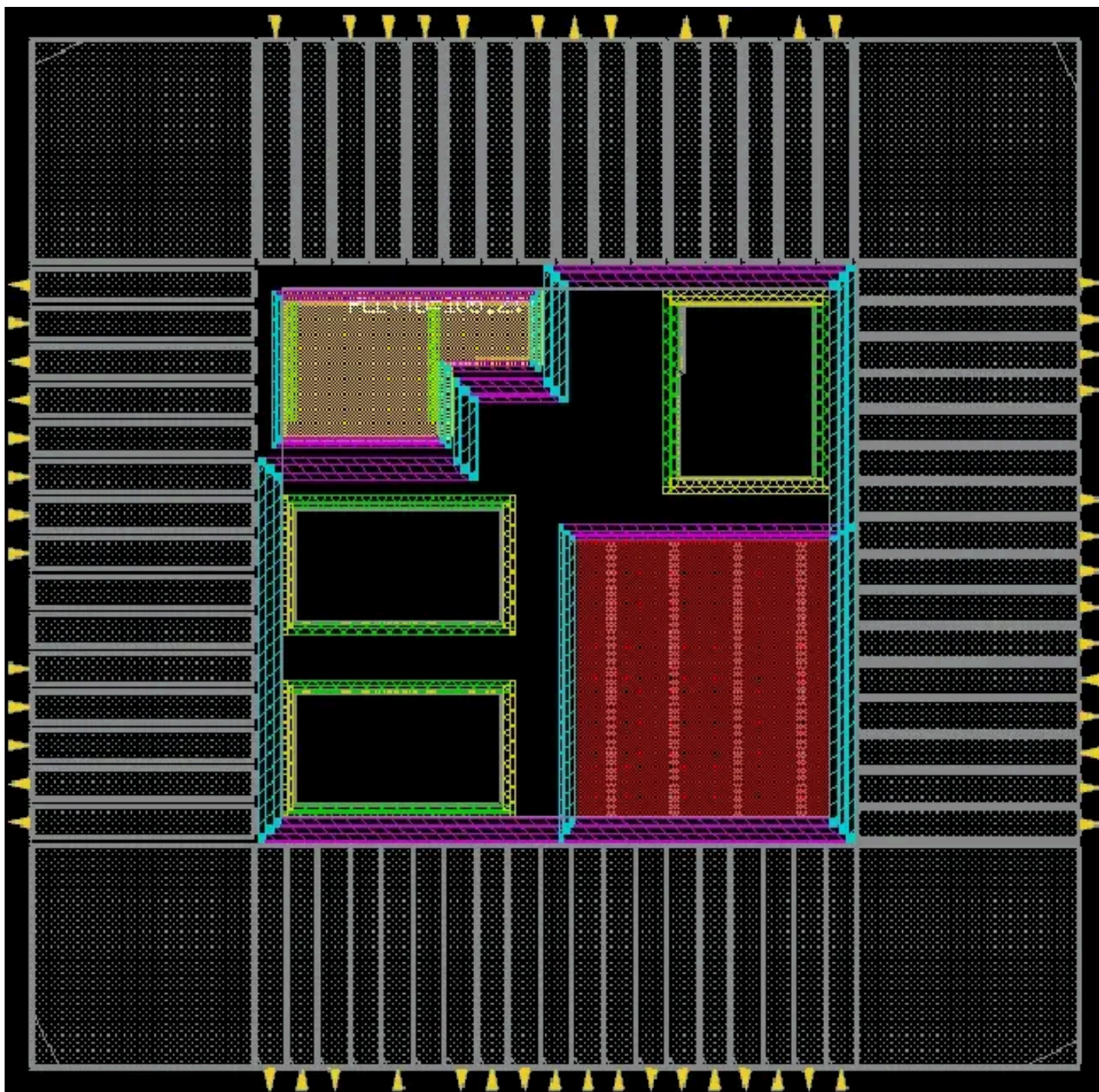
Power Planning and Routing Flow

This section presents a real design as an example.

- A floorplan design is shown below, where all PG pins or tie-hi/tie-low pins have been connected with a global power/ground net:



- An example of adding rings is presented below. In the figure below:
 - Core rings are added around the core area (excluding the PLL area) for the `vdd_lp_s`, `vss`, and `vdd` nets.
 - Block rings are added around the block PLL for the `Avss` and `Avdd` nets.
 - Domain rings are added around non-default domain TDSP for the `vdd_lp_s`, `vss`, and `vdd` nets.



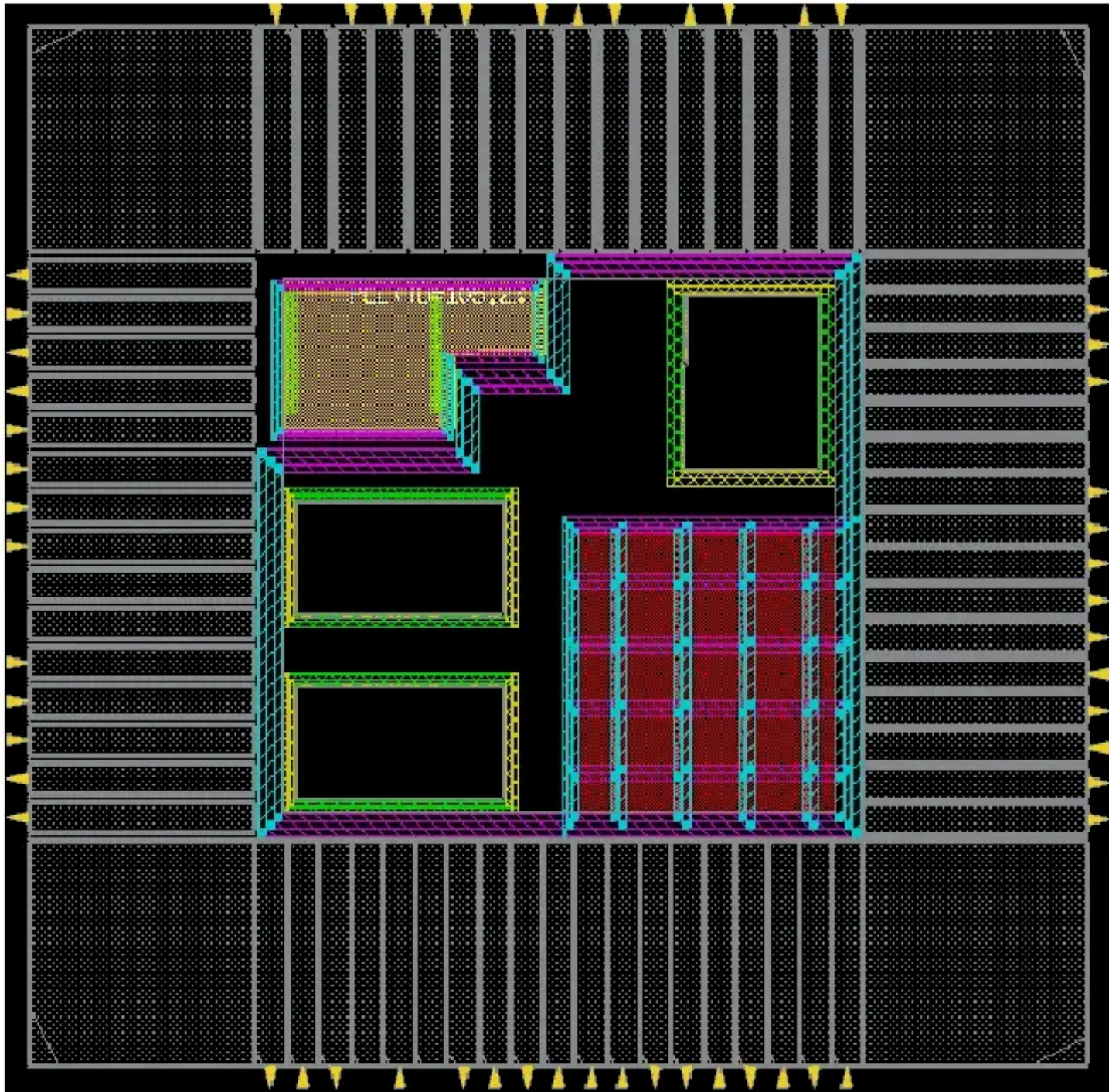
You need to run the following commands for this example:

```
selectObject Group PLL
```



```
addRing -type core_rings -nets {vdd_lp_s vss vdd} -layer {top METAL7 bottom  
METAL7 left METAL8 right METAL8} -offset 1 -width 8 -spacing 1.0 -  
exclude_selected 1  
  
deselectAll  
  
selectInst DTMF_INST/PLLCLK_INST  
  
addRing -type block_rings -nets {Avss Avdd} -around selected -layer {top  
METAL7 bottom METAL7 left METAL8 right METAL8} -width 5 -spacing 1 -offset 1  
  
deselectAll  
  
selectObject Group TDSPCore  
  
addRing -type block_rings -nets {vdd_lp_s vss vdd} -around power_domain -  
layer {top METAL7 bottom METAL7 left METAL8 right METAL8} -width 5 -spacing  
1 -offset 1  
  
deselectAll
```

- Examples of adding stripes are the following:
 - In the figure below, domain stripes are added around the stripes over switch cell pins for each different layer:



You need to run the following commands for this example:

```
selectObject Group TDSPCore

setAddStripeMode -skip_via_on_pin {}

addStripe -over_pins 1 -nets vdd_lp_s -over_power_domain 1 -layer METAL4 -
width 8 -master HEAD16DM -pin_layer METAL2

addStripe -over_power_domain 1 -nets vss -direction vertical -layer METAL4 -
width 8 -set_to_set_distance 70 -start_from left -start_offset 43.46 -
spacing 1

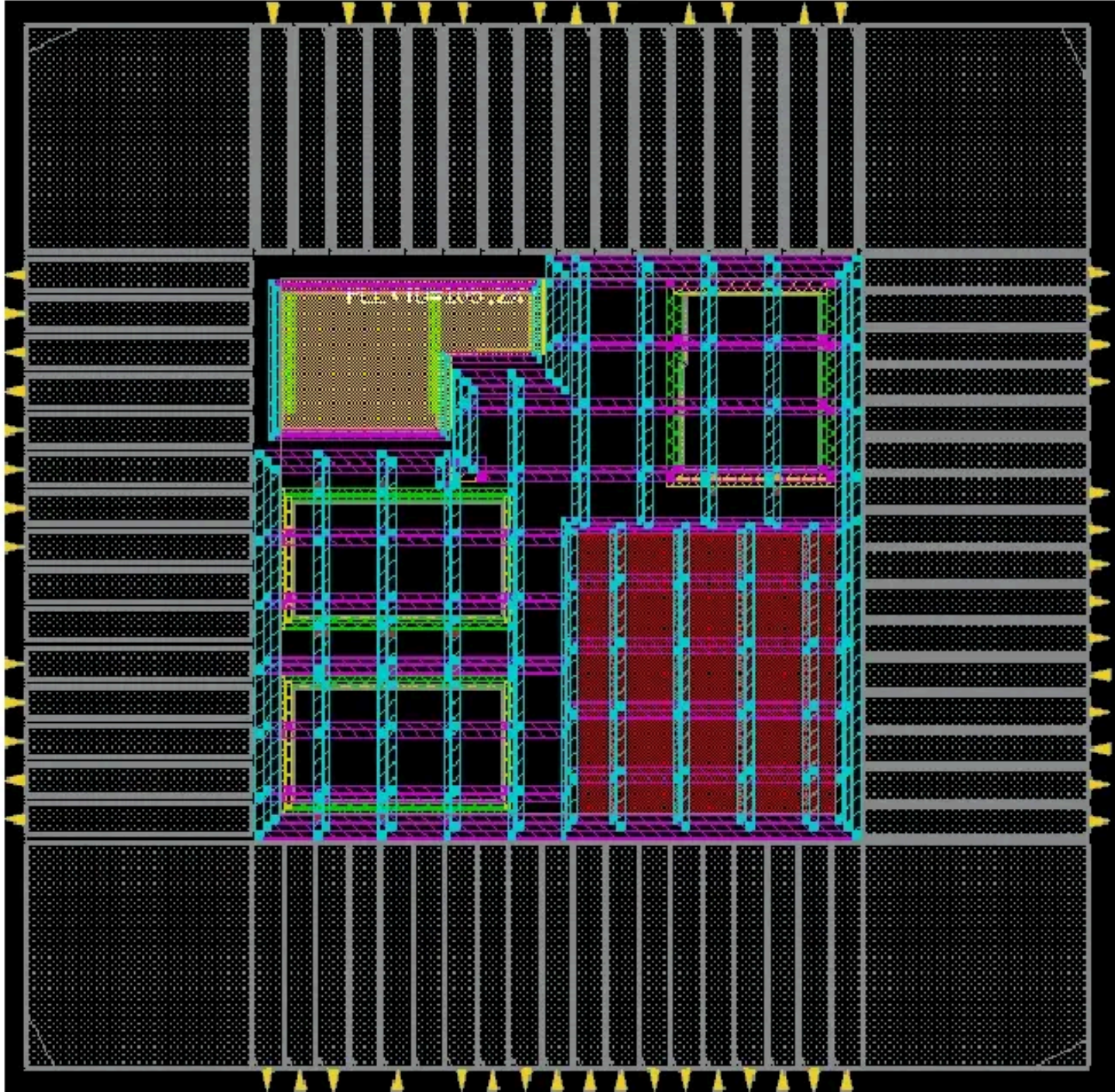
addStripe -over_power_domain 1 -nets {vdd_lp_s vss} -direction horizontal -
layer METAL7 -width 8 -set_to_set_distance 70 -start_from bottom -
```



```
start_offset 34.46 -spacing 1
```

```
addStripe -nets {vdd_lp_s vss} -over_power_domain 1 -layer METAL8 -width 8 -  
set_to_set_distance 70 -start_from left -start_offset 34.46 -spacing 1
```

- In the figure below, stripes are added in the core area for each different layer:



You need to run the following commands for this example:

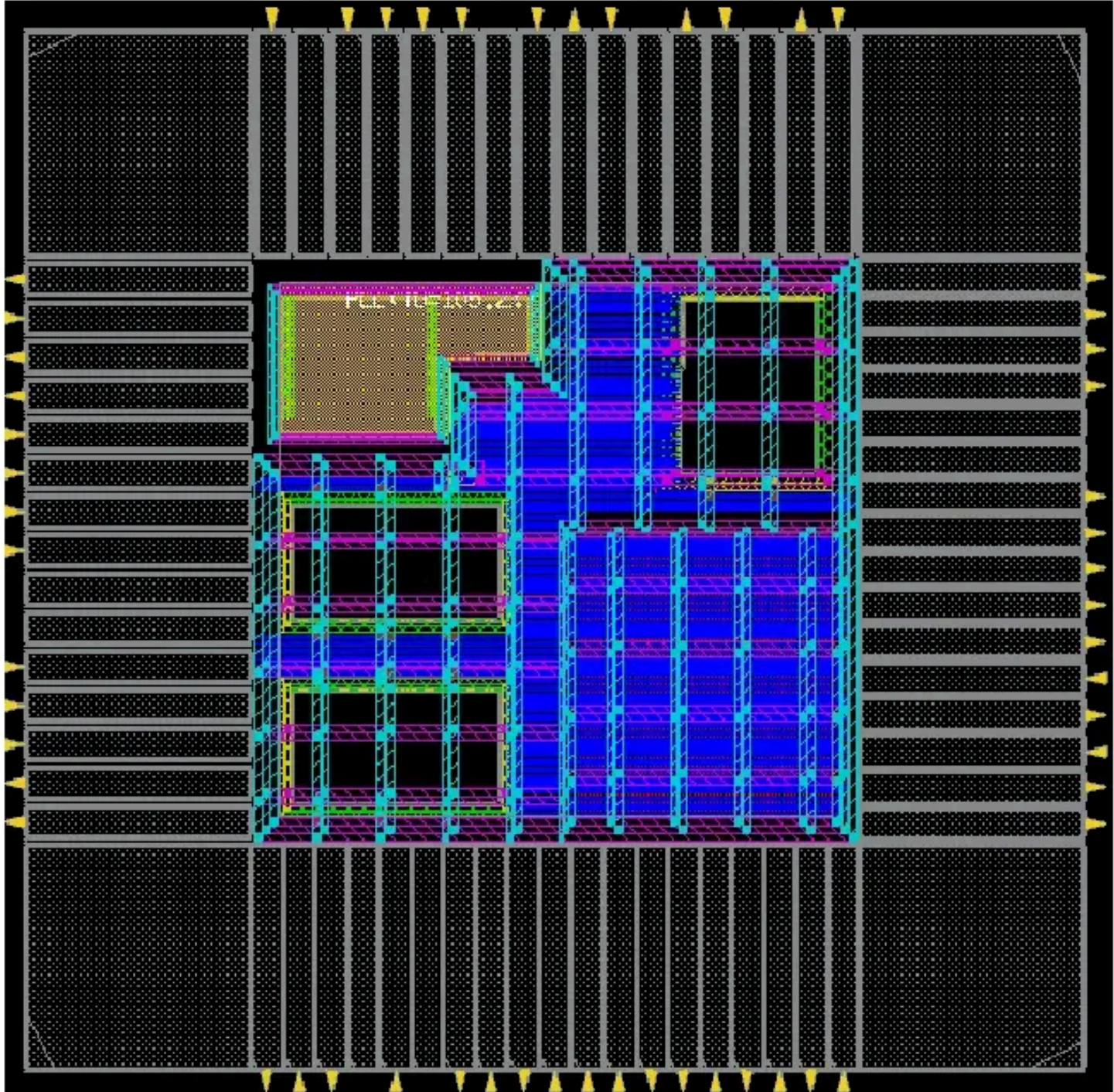
```
addStripe -nets {vdd vss} -layer METAL4 -width 8 -set_to_set_distance 70 -  
xleft_offset 37.9 -spacing 1
```

```
addStripe -nets {vdd vss} -direction horizontal -layer METAL7 -width 8 -  
set_to_set_distance 70 -ybottom_offset 14 -spacing 1
```



```
addStripe -nets {vdd vss} -layer METAL8 -width 8 -set_to_set_distance 70 -  
xleft_offset 37.9 -spacing 1
```

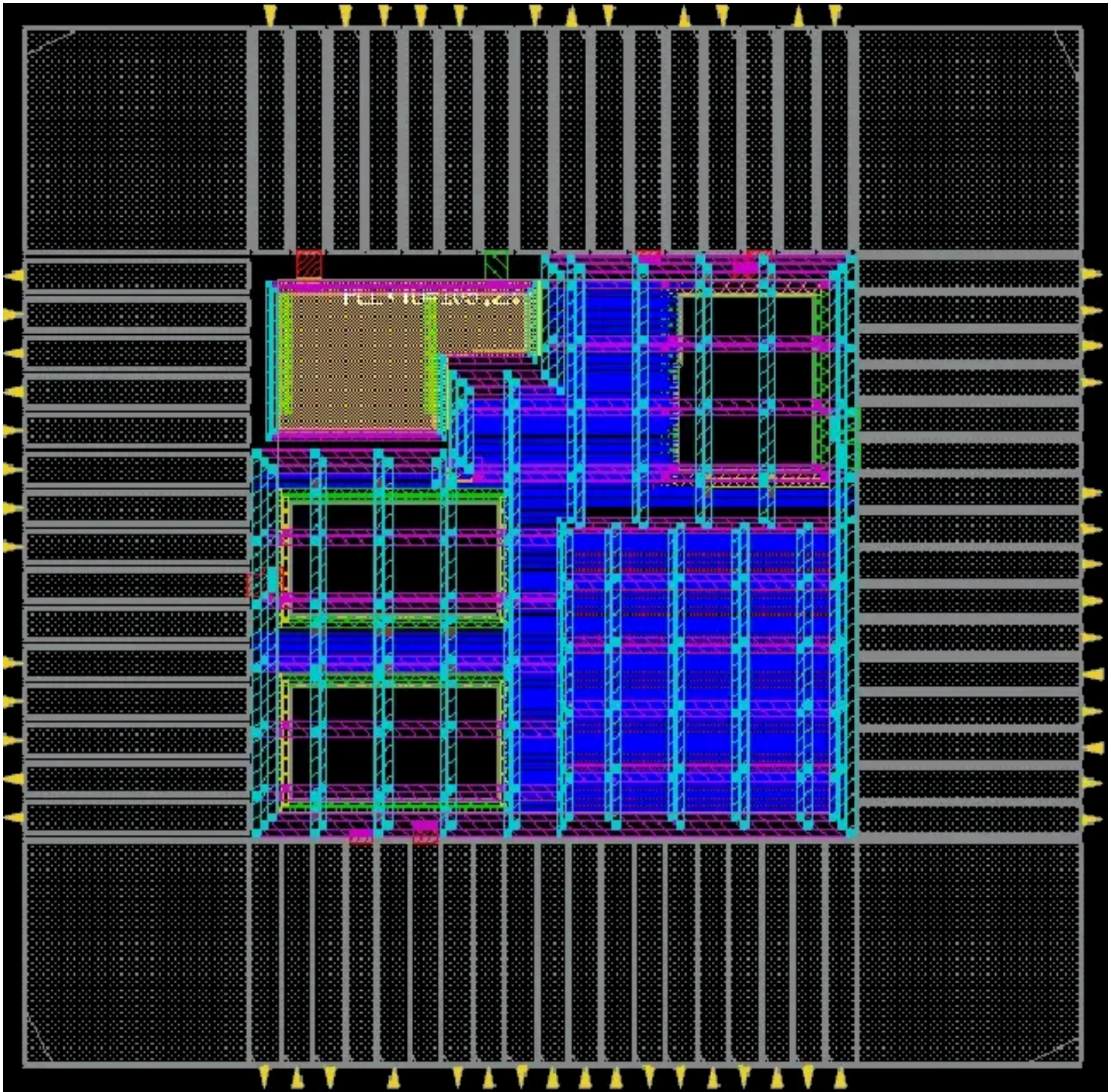
- In the figure below, standard cell pins are connected to form a followpin:



You need to run the following command for this example:

```
sroute -connect corePin
```

- In the figure below, padpins are connected to rings:



You need to run the following command for this example:

```
sroute -connect padPin
```

- The following command is an example of using the `saveDesign` command:

```
saveDesign lab/dtmf_pwr.enc
```

Related Information

- [Generating Special Power Vias Using Viagen](#)
- [Generating Default Special Via](#)

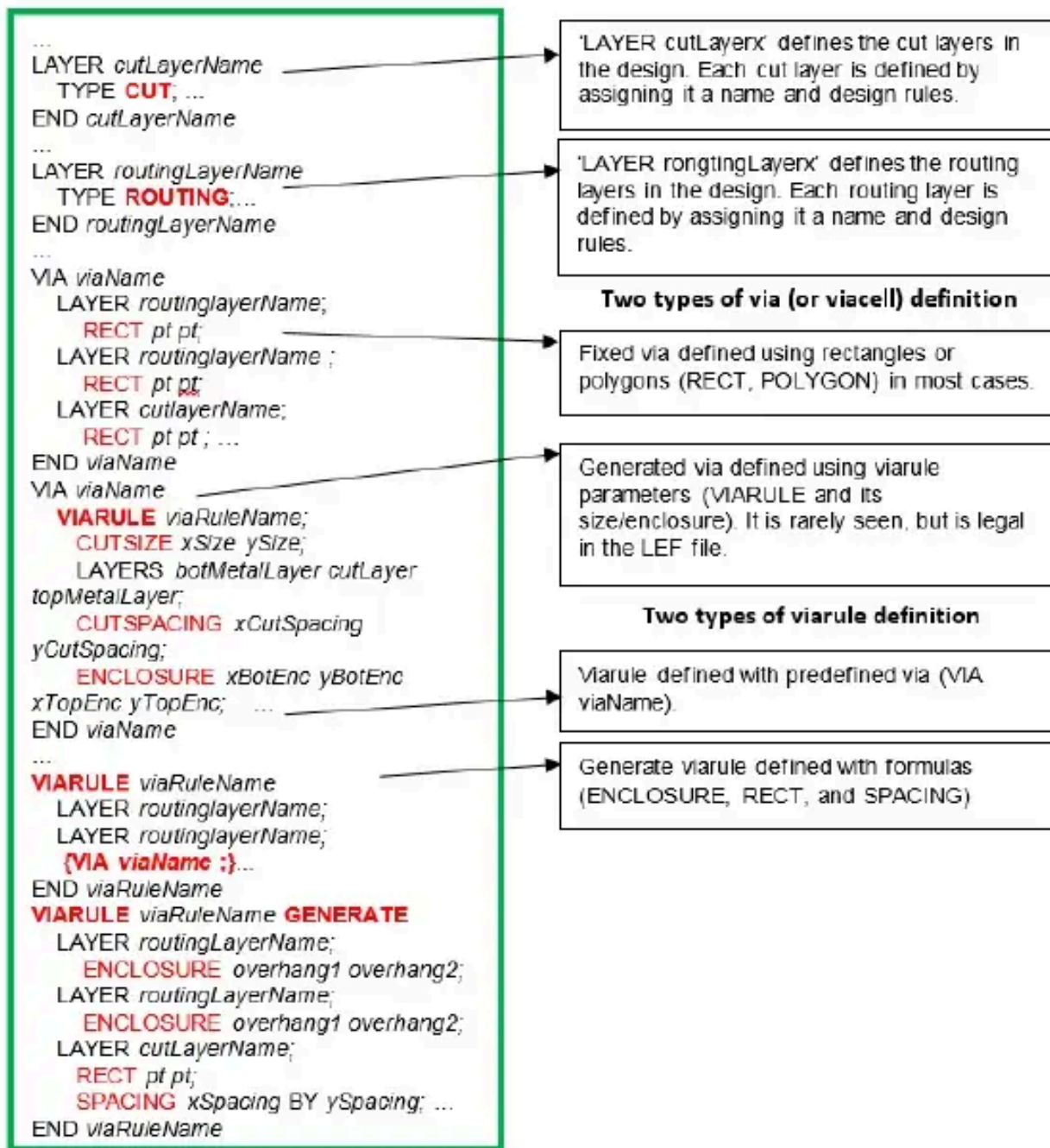
- [Inserting Vias with a Specific Cutclass](#)
- [Inserting a Via from Specific Viarule](#)
- [Trimming Redundant PG Stripes and Vias](#)
- [Flash PG](#)

Generating Special Power Vias Using Viagen

Defining Via and Viarule in LEF File

Refer to the following figure for understanding the definition of via and viarule in the LEF file. The via and viarule definition is presented in the green outline box in the following figure. The LEF section presented here does not contain the complete syntax, but helps in understanding the necessary syntax including the following definitions:

- Cut and routing layers
- Fixed and generated vias
- Predefined and generated viarules



Creating Auto Viarule

The special via engine, viagen, automatically creates 'auto viarule' internally for all the cutclasses based on the cut layer information available in the LEF file. Mostly, there are two (SINGLE and BAR), or at times three (SINGLE, BAR, and LARGE) cutclasses in each cut layer.

Following is an example of creating an auto viarule:

Cut layer definition:

```
PROPERTY LEF58_CUTCLASS "
    CUTCLASS VSINGLECUT WIDTH 0.020 LENGTH 0.020 CUTS 1 ;
    CUTCLASS VDOUBLECUT WIDTH 0.020 LENGTH 0.050 CUTS 2 ;"
```

```
PROPERTY LEF58_SPACINGTABLE "
```

```
    SPACINGTABLE    PRL  -0.022    MAXXY

    CUTCLASS          VSINGLECUT          VDOUBLECUT SIDE

    VSINGLECUT          0.044 0.044          0.044 0.044

    VDOUBLECUT SIDE 0.044 0.044          0.044 0.044
```

```
PROPERTY LEF58_ENCLOSURE "
```

```
    ENCLOSURE CUTCLASS VSINGLECUT BELOW 0.020 0.000 ;

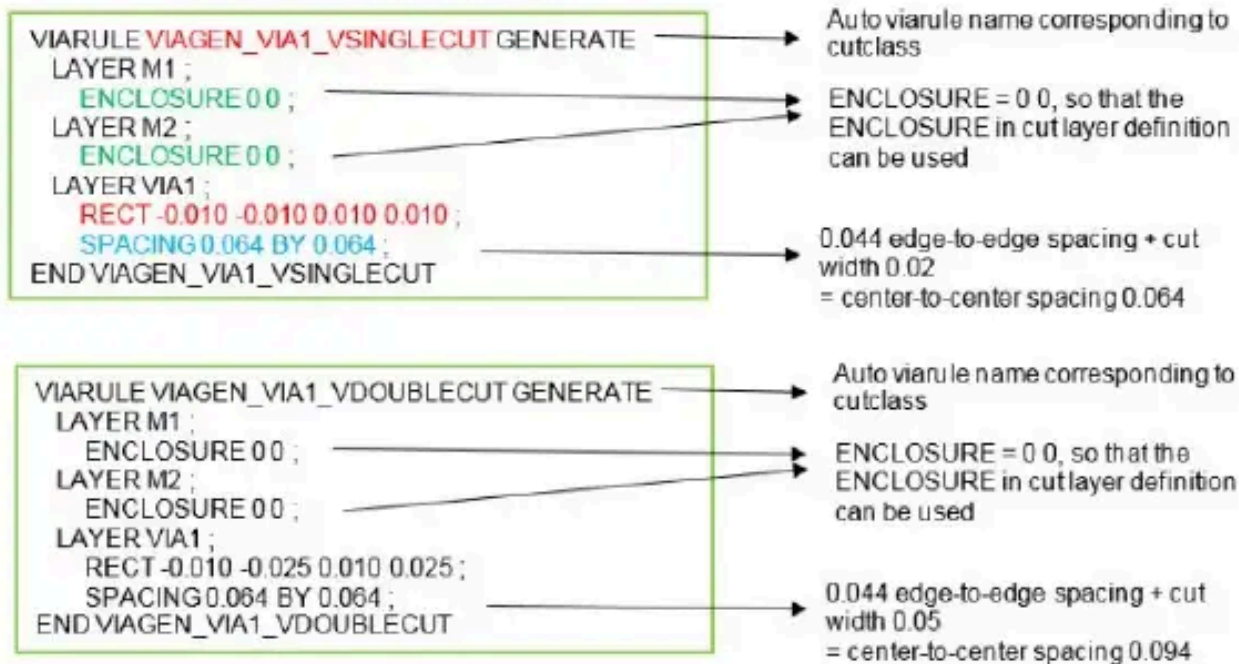
    ENCLOSURE CUTCLASS VSINGLECUT ABOVE 0.030 0.000 ;

    ENCLOSURE CUTCLASS VDOUBLECUT BELOW END 0.030 SIDE 0.000 ;

    ENCLOSURE CUTCLASS VDOUBLECUT BELOW END 0.030 SIDE 0.003 WIDTH 0.026 ;

    ...
```

Auto viarules are created for VSINGLECUT and VDOUBLECUT, as shown below:



The database order of viarules is as follows:

1. Viarules in the LEF file's order
2. Auto viarules; larger cut area first (cutclasses from LARGE to SINGLE)

Related Information

[Power Planning and Routing](#)

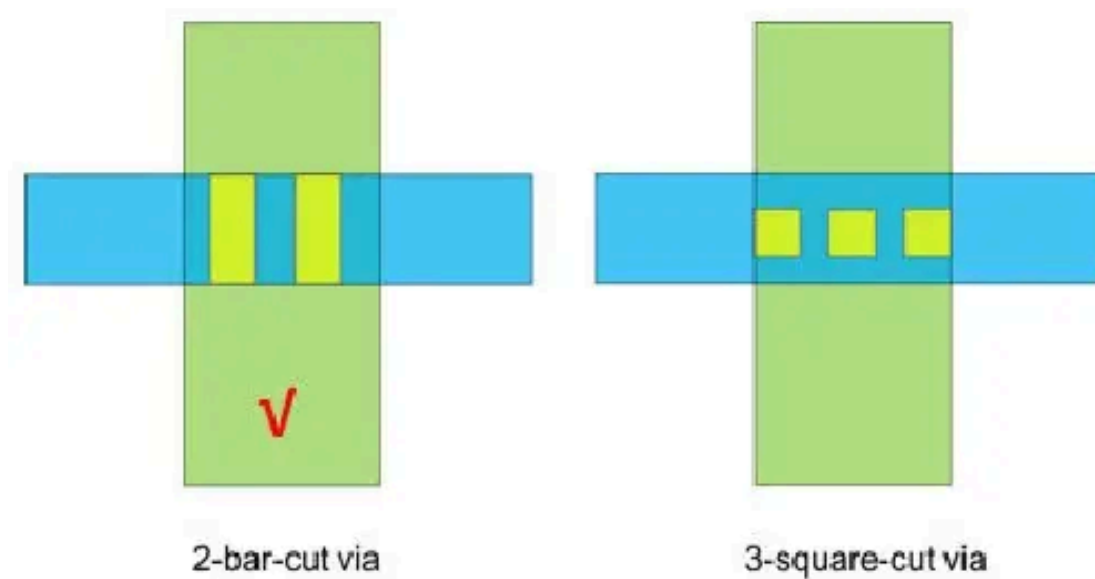
Generating Default Special Via

By default, the special via generation engine, viagen, uses only the viarule (including viarule available in the LEF file and internally generated auto viarule) to generate vias. The new viarule pickup criteria depends on the maximum total cut area for attaining the best connection and lowest IR drop.

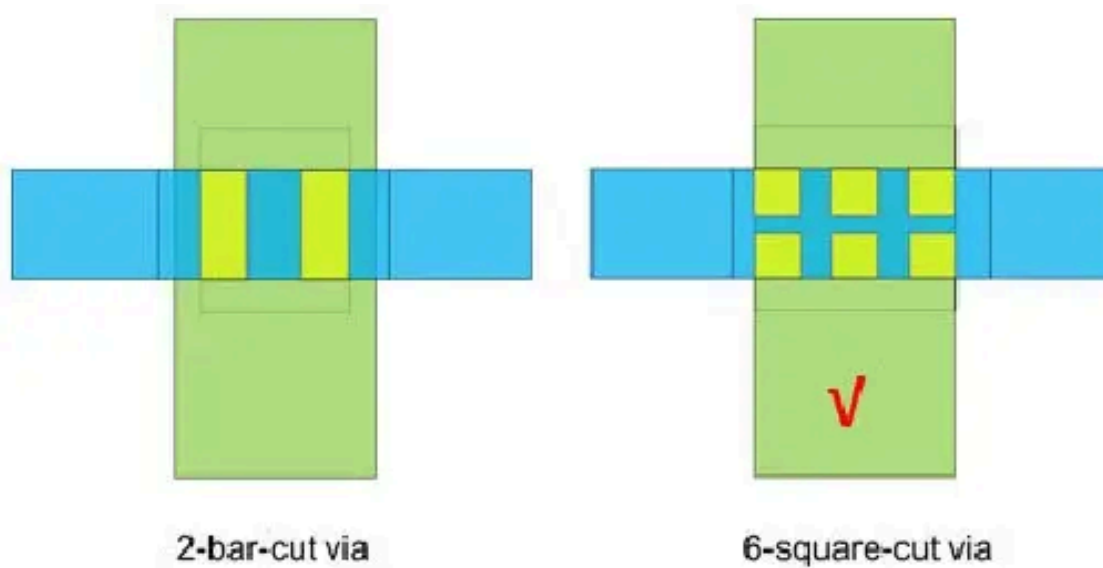
To generate a default special via:

1. Find the crossover of two same-net special segments.
2. Go through all the vias generated by the various viarules generated in the database, including the LEF file viarule and auto viarule.
3. Pick up a viarule that generates a via with the maximum total cut area, and use it. Refer to the following examples:

Example 1: non-cross via, 2-bar-cut via wins 3-square-cut via

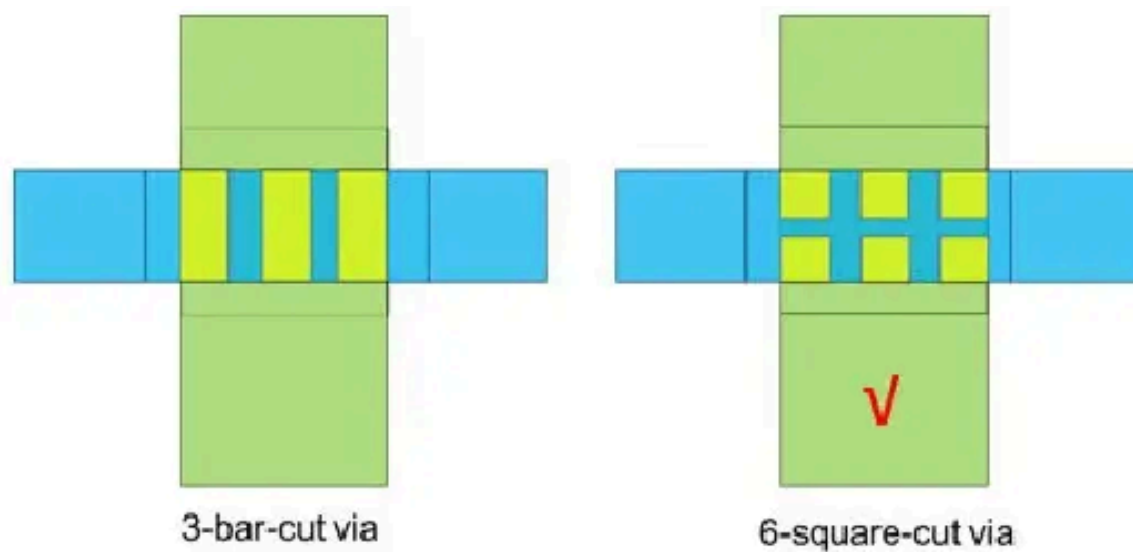


Example 2: cross via, 6-square-cut via wins 2-bar-cut via



If different viarules generate same cut-area via at a specific intersection, the viarule in the front of the database is used. So as per the LEF file order, the LEF viarule is used first, followed by auto viarule (LARGE-> BAR -> SQUARE). Refer to the following example:

Example 3: A via with various cutclasses has the same total cut area, the 6-square-cut via from viarule list over the 3-bar-cut via from viarule list, as presented below:



4. Once a viarule is determined, viagen gets parameters from the viarule as a soft guide, and the technical definition in cut/metal layer as the hard guide for generating vias. By default, the enclosure defined in the LEF viarule is ignored and the cut layer enclosure value is used. This option is controlled by running the following command:

```
setViaGenMode -ignore_viarule_enclosure {true | false}
```

Default: true

Related Information

Power Planning and Routing

Inserting Vias with a Specific Cutclass

Mostly, there are two (SINGLE and BAR), or at times three (SINGLE, BAR, and LARGE) cutclasses in each cut layer; refer to the LEF syntax. The following topics are covered under this section:

- Inserting Vias with the Preferred Cutclass
- Distinguishing Cutclasses Internally

Inserting Vias with the Preferred Cutclass

Use the following command parameter to insert a via with a specific cutclass:

```
setViaGenMode -cutclass_preference {default | {[square] [bar] [large]}}
| cutclass_name_list      | file_name }
```

Default: default

The following table illustrates the behavior of each of the above mentioned parameter values. If a specified cutclass cannot generate a DRC-clean via, the intersection remains OPEN.

Parameter Value	Description	Next Preference, if Violation Exists
default	Specifies the viarule with the cutclass that has the maximum total cut area	Next maximum total cut area
square	Uses the viarules with the SINGLE cutclass (the cut width and length are the same and the smallest in that layer, for example, 0.032 by 0.032).	OPEN
bar	Uses the viarules with the DOUBLE cutclass (the cut width is the smallest; For example, 0.032 by 0.08, is equal to two SINGLE cutclasses in the LEF definition).	

large	Uses the viarules with LARGE cutclass (neither the cut width, nor the length, are smallest; For example, 0.08 by 0.08, is equal to 4 SINGLE cutclasses in the LEF definition).	
cutclass_list	Uses the viarules with the specified cutclass, by the order of the bigger to smaller total cut area.	OPEN
file_name	Uses the viarules with the cutclass written in a specified file, by the order of bigger to smaller total cut area. Various arguments can be separated by space or line break.	OPEN

The option setting does not support incremental specification. The last cutclass preference setting is used.

Distinguishing Cutclasses Internally

To differentiate cutclasses, the area of cut is used. For example, there are three cutclasses, the minimum size cut is 'square', the medium size is 'bar', and the maximum size is 'large'; There are two cutclasses, the smaller size cut is 'square', the bigger size cut is 'bar'.

When you specify '{[square] [bar] [large]}' , there should be two or three cutclasses defined in the LEF file. If the tool detects that there are more than three or less than two cutclasses, the setting is ignored.

Related Information

[Power Planning and Routing](#)

Inserting a Via from Specific Viarule

To insert the exact via or viarule defined in the LEF file, run the following command before the commands used for inserting an issued via:

```
setViaGenMode -viarule_preference {default | predefined | generated | list of via
rule/cell names | file name }
```

Default: default

The above command changes the priority of via or viarule to be considered by viagen, as mentioned in the following table. If a specified via or viarule cannot generate a DRC-clean via, the intersection remains OPEN.

Parameter Value	Description	Next Preference, if Violation Exists
default	Specifies the viarule that generates vias with the maximum total cut area.	Next maximum total cut area
predefined	Uses predefined via rules, by the order of total cut area (viarule without keyword).	OPEN
generated	Uses generated via rules (viarule with keyword GENERATE).	OPEN
list of via rule/cell names	Uses the list of specific via rules and/or via cells specified in the parameter. If via cells and rules are listed, viagen ranks vias with higher priority and uses the one-fit principle; ranks viarules with lower priority than vias, and uses the maximum total cut area principle. Refer to the following example: <pre>setViaGenMode -viarule_preference {via23_1 viarule23_a via23_2 viarule23_b}</pre> In this case, the specified vias and viarules are arranged in the following priority: 1. Priority: via23_1 > via23_2 > viarule23_a > viarule23_b 2. The tool tries via23_1 first; If no violation occurs, it uses via23_1 (one-fit principle). 3. If it fails, it tries via23_2 ; If no violation occurs, it uses via23_2 ; 4. If it fails, it tries the viarule between viarule23_a and viarule23_b , which generates the via with the bigger total cut area. 5. If it fails, it tries the viarule between viarule23_a and viarule23_b , which generates the via with the smaller total cut area. 6. If it fails, the intersection remains OPEN.	OPEN

file name	Uses the vias and/or via rules listed in the LEF file. It uses the same principle as that used for the list of via rule/cell names.	OPEN
-----------	---	------

The option setting supports incremental specification. All the specified vias and viarules in various commands are ranked with vias with higher priority. Refer to the following example:

```
setViaGenMode -viarule_preference {via23_1 viarule23_a via23_2 viarule23_b}
setViaGenMode -viarule_preference {viarule23_c via23_3}
```

In this case, the specified vias and viarules are arranged in the following priority:

1. via23_1 > via23_2 > via23_3 > viarule23_a > viarule23_b > viarule23_c
2. The tool tries via23_1 via23_2 via23_3 with the one-fit principle.
3. If all the vias fail, it tries viarule23_a viarule23_b viarule23_c in the order of generating the bigger total cut area first, followed by the smallest total cut area at the last.
4. If all the vias and viarules fail, the intersection remains OPEN.

Note: It is not recommended to set both the `-cutclass_preference` and `-viarule_preference` parameters at the same time. Still if it happens, `-cutclass_preference` has higher priority. It means that the specified cutclass in `-viarule_preference` is attempted first, then the specified cutclass in LEF viarules and auto viarules; and then other vias and viarules in `-viarule_preference`.

Related Information

[Power Planning and Routing](#)

Trimming Redundant PG Stripes and Vias

You may trim redundant PG stripes and vias using an existing power grid pattern based on IR drop analysis reports. You the following command to trim the redundant PG stripes and vias:

```
trim_pg
```

This eliminates the need to manually adjust the power structure to reduce power grid pessimism for area/timing/power improvement in a design.

To trim a stripe:

1. Use `trim_pg -type stripe`.

2. Specify only one LEF layer name with the `-layer` parameter.
3. Specify only one string with the `-pattern` parameter. If more than one values are specified, the command displays an error.

For example, the following command trims 50% of the `VDD` stripes on a `M6` metal layer inside a `{0 0 500 500}` box:

```
trim_pg -net VDD -type stripe -layer M6 -area {0 0 500 500} -pattern 10
```

To trim a via (stack):

1. Use `trim_pg -type via` .
2. Specify two LEF layer names with the `-layer` parameter. The first layer stripe is used as the pattern reference.
3. Specify one or more strings with the `-pattern` parameter to indicate the via trip pattern.

For example, the following command trims a via (stack) along with half of the `M9 VDD` stripes, and one-third of its stack vias between the `M9` and `M6` metal layers:

```
trim_pg -net VDD -type via -layer {M9 M6} -area {0 0 500 500} -pattern {110 1}
```

Trimming Stripes

You need to specify a box for trimming a stripe. In some cases, the specifies box area also contains some local PG grid. You may keep the local grid or trim them separately using a different pattern with a combination of the following three parameters:

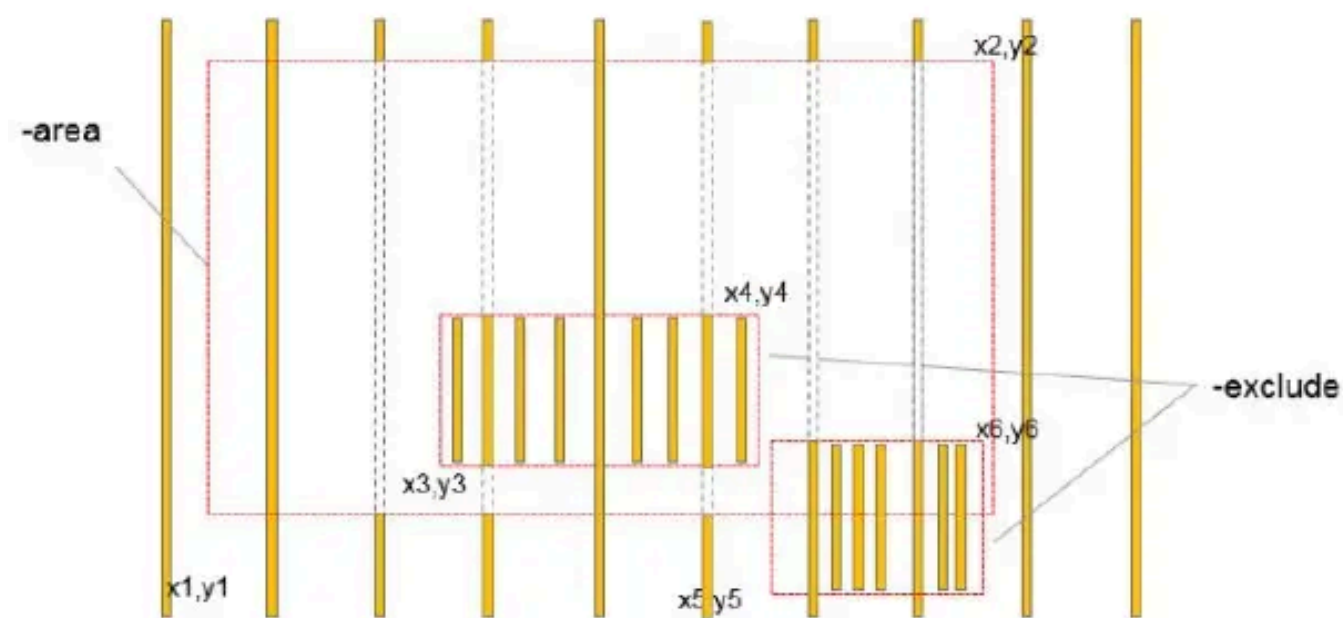
- `-area { x1 y1 x2 y2 }`: Specifies the trimming area. It defines the stripes and vias to be trimmed. The stripe overlapped with the box defined by `-area` is trimmed. By default, the entire core area is trimmed.
- `-exclude {{ lx1 ly1 ux1 uy1 } { lx2 ly2 ux2 uy2 } ...}`: Excludes the box areas for the stripes and vias in which the part of stripes need not to be trimmed. This parameter is used along with the `-area` parameter to specify the exclusion areas. If a stripe partially overlaps with the exclusion area, the overlapping segment of the stripe is not trimmed. The exclusion box can partially overlap the `-area` box.
- `-exclude_local_grid {{ lx1 ly1 ux1 uy1 } { lx2 ly2 ux2 uy2 } ...}`: Excludes the areas in which the whole stripes are skipped for trimming. If a stripe is completely inside one of the `-exclude_local_grid` box (can touch the boundary), it is excluded from the trim candidates and is not counted when applying the trim pattern. It is useful when trimming is done on a large

area but there are local stripes in this area, which could affect the global trim pattern. The exclusion box can partially overlap the `-area` box.

For example, if you want to trim 50% of the stripes inside an area. But there are some local stripes over a hard macro in this area. If you use `-exclude`, the local stripes are be trimmed. But they are counted when applying the `10` pattern. Through this, the long stripes are not trimmed. Refer to the command below:

```
trim_pg -net VDD -type stripe -layer M6 -area {x1 y1 x2 y2}
\
-exclude {x3 y3 x4 y4 x5 y5 x6 y6} -pattern 10
```

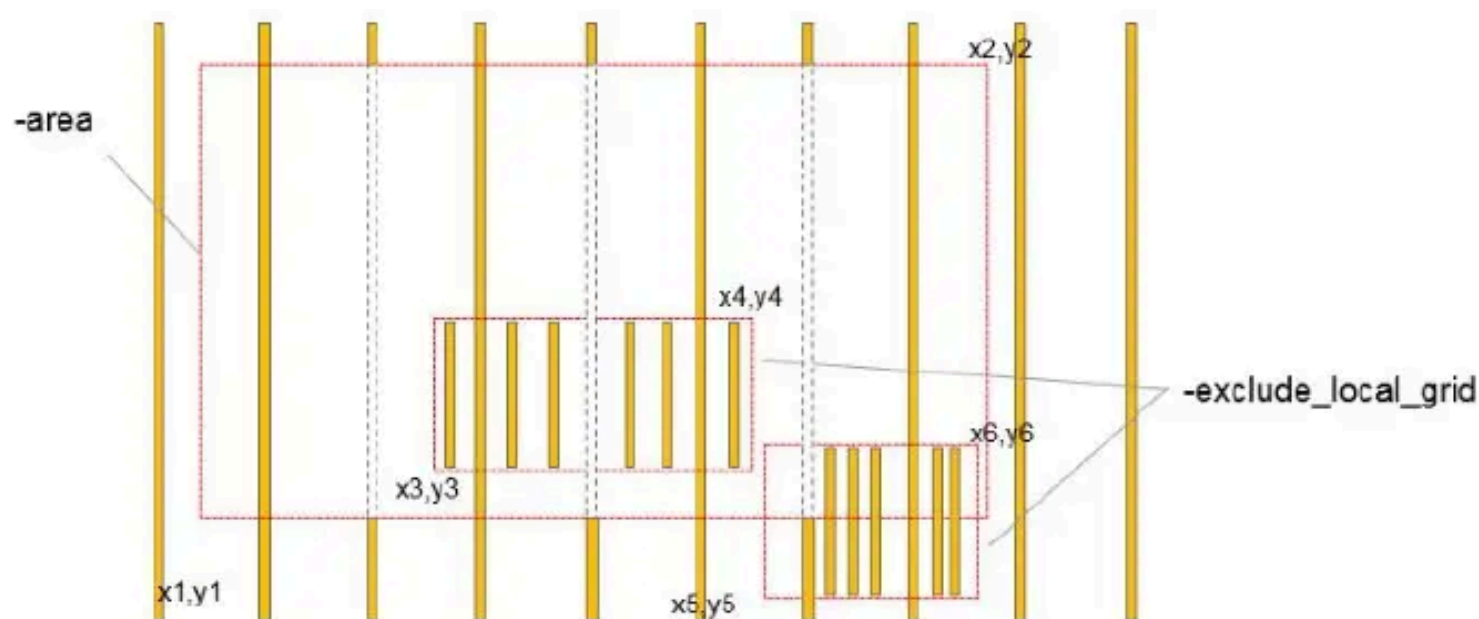
Following is the output:



For trimming the long stripes, use the `-exclude_local_grid` parameter. It prevents the local stripes from being counted in pattern and ensure that the desired pattern applied to the global stripes. You may also use it with the `-area` and `-exclude` parameters, as shown below:

```
trim_pg -net VDD -type stripe -layer M6 -area {x1 y1 x2 y2} \ -exclude_local_grid {x3
y3 x4 y4 x5 y5 x6 y6} -pattern 10
```

Following is the output:

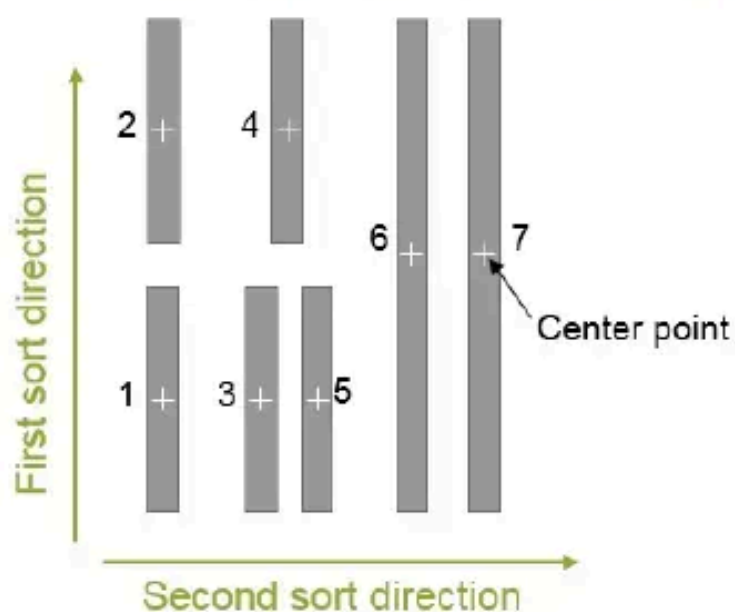


Sorting Stripes

You may sort stripes before trimming a pattern. You may select stripes as trimming candidates by sorting the `-area` and `-exclude_local_grid` parameters using the following steps:

1. Determine the running direction of most of the stripes by comparing the length of the vertical and horizontal edges of the stripe (vertical direction in the example below).
2. Sort the stripes based on their center point, by the direction derived from the previous step.
3. Sort stripes in the orthogonal direction (horizontal direction in the example below).

Refer to the example below:



Trimming a Pattern

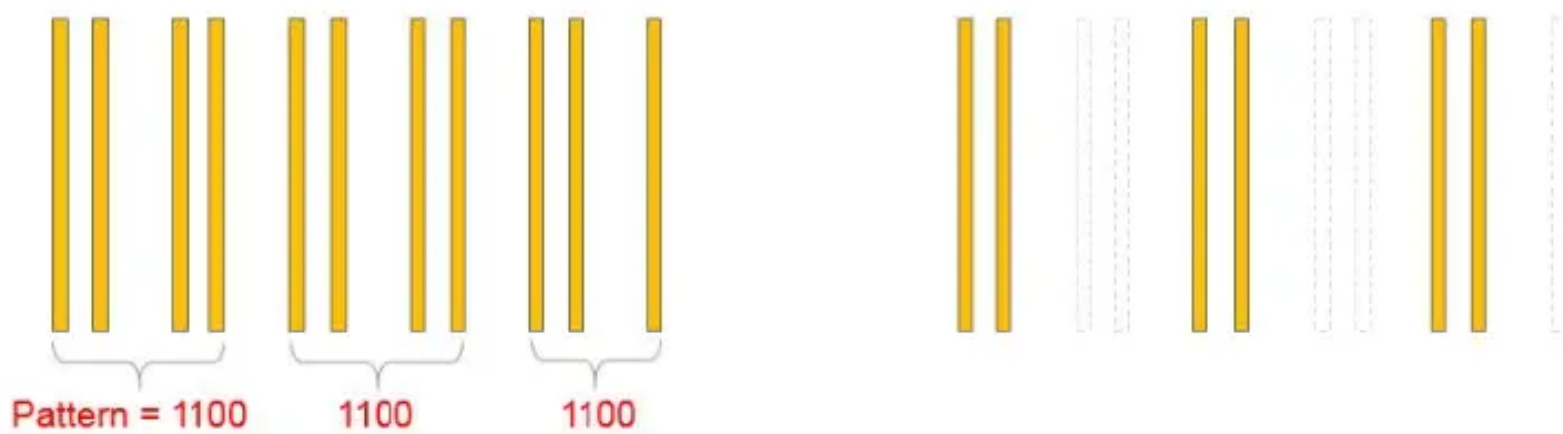
You may trim a pattern using a sequence of 0 and 1, where 0 signifies to trim the stripe and 1 to keep the stripe. Follow the steps below:

1. Starting from the first stripe in the sorted queue, trim or keep it according to the pattern.
2. Repeat the pattern till the last stripe in the queue.

Refer to the example below:

```
trim_pg -net VDD -type stripe -layer M6 -area {0 0 500 500} -pattern 1100
```

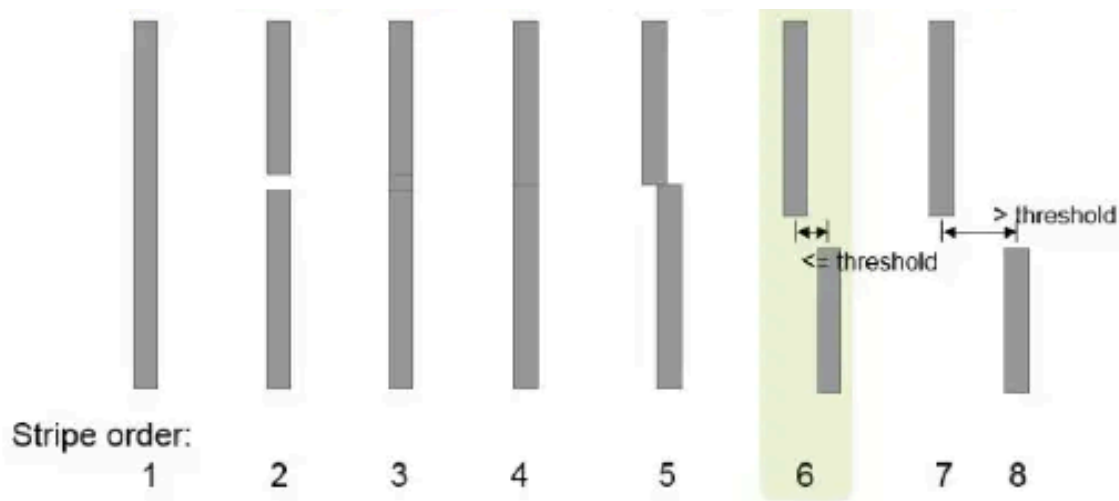
Following is the output:



Setting-up Threshold for Stripe Merging

In some cases, a long stripe is broken-up in to multiple segments such as through hard macro and power domain. The complexity of a real design power grid can also have the abut, overlapping, or slightly misaligned stripes. While trimming a PG pattern, you may consider it as a single stripe rather than multiple segments. You may define a spacing threshold along with the secondary sort direction. If the centerline spacing between two or more stripes is smaller than the value provided by the `-threshold` parameter, they are treated as a single stripe while trimming or keeping the stripe.

Refer to the example below:



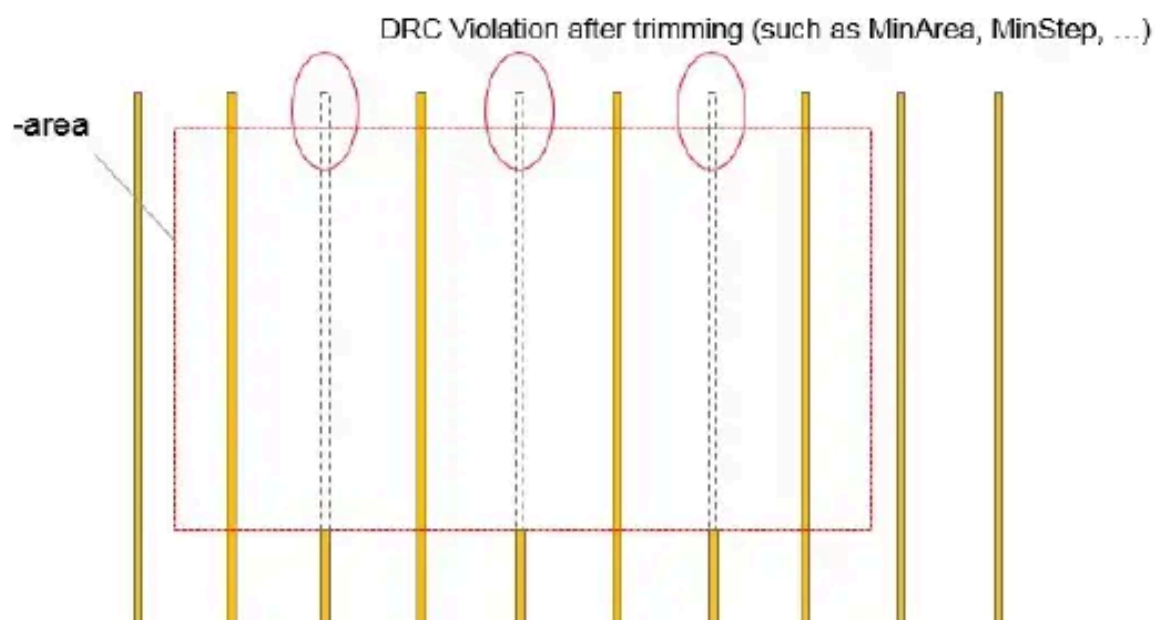
You may sort the spacing between the adjacent stripes as below:

$$-d6 < d3 < d5 < \text{Threshold} < d7 = d4 < d2 < d1 = d8$$

While grouping, start from two adjacent stripes with the smallest spacing, if those are less than the threshold, group them. In this example, the smallest spacing is $d6$ between C and H, and is less than the threshold. So C and H are grouped. Then continue with the next smallest spacing. In this example, it is $d5$. But C is already grouped with H, measure the spacing between G and H. However $d5+d6$ is greater than the threshold. So G will not be grouped with C and H. Refer to the final grouping below:

Avoiding New DRCs

While trimming stripes, you need to avoid creation of new DRCs. Otherwise, trimming will be continued on the new DRCs created. For example, the stripe segments highlighted in the red circles below are also trimmed, if new DRCs are created after trimming the stripe segments in the specified area:

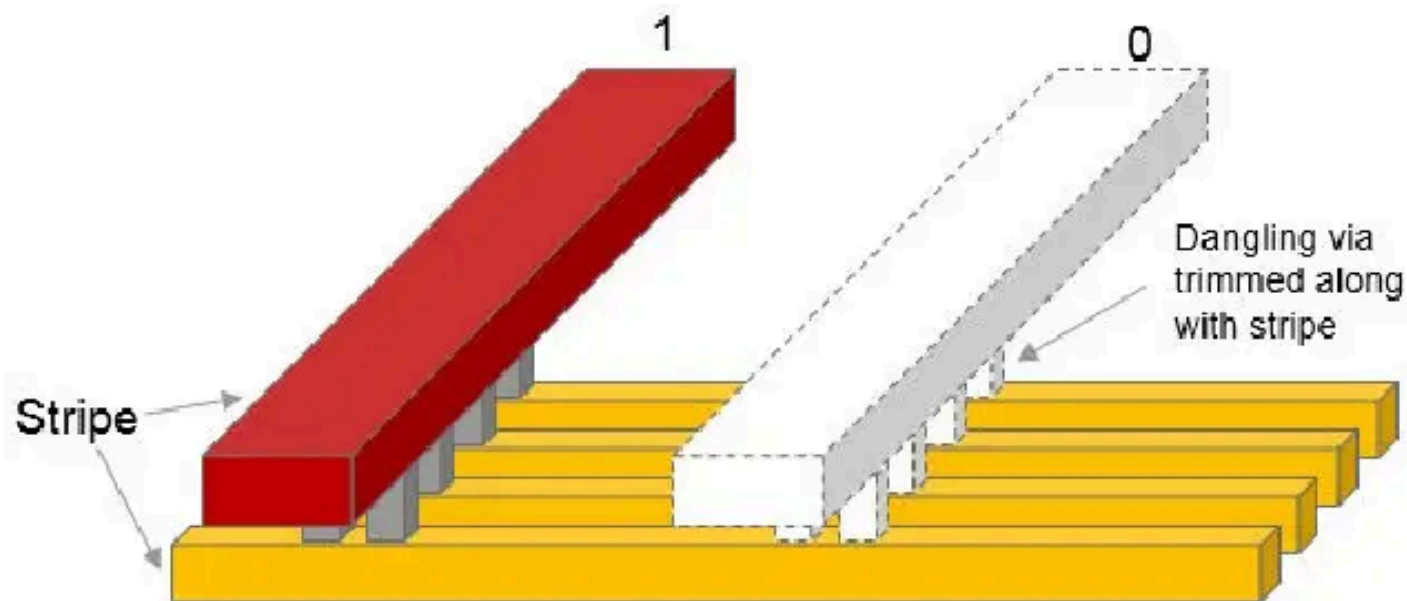


Keeping Dangling Vias While Trimming Stripes

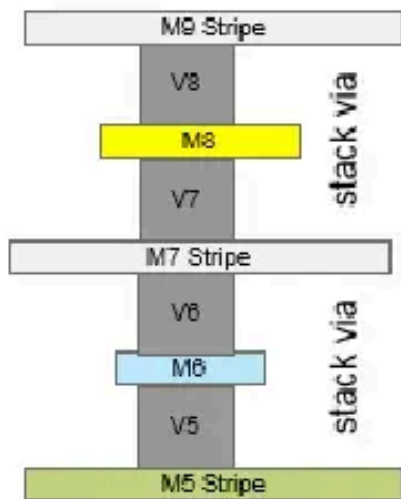
While trimming a stripe, if the connecting via (including stack via) becomes dangling, it is also be trimmed, by default. Use the `-keep_dangling_via` parameter to keep the dangling vias while trimming stripes. It searches the stack vias before trimming stripe. For example, while trimming the following pattern:

M7 stripe - V7/V8 - M9 stripe - V9 - M10 stripe

After trimming the M9 stripe, M7 and M10 are still connected through the V7/V8/V9 stack. But as this stack is actually formed by the previous V7/V8 and V9, so you need to check dangling for V7/V8 and V9 respectively.



The stack via formed after trimming is not treated as a stack via. For example, after trimming the M9 stripe, M5 and M9 are still connected through the V5~V8 stack. But keeping dangling vias while trimming checks the stack vias before trimming. So the V7/V8 and V5/V6 stacks will be removed by default.



Trimming Vias

Power vias, especially the stack vias, connecting low and high layer stripes often occupy lots of routing tracks, and brings congestions and routing difficulties to the signal routing. So you need to remove some of the stack vias on the power grid to save more routing resource. In such cases, trim the power vias and keep the stripes.

Use the `trim_pg -layer` parameter to accept one or two metal layers for stripe and via trimming. While trimming a stripe, specify `layer_name_1` on which the stripes are to be trimmed. While trimming via, specify `layer_name_2` between which the stack vias are to be trimmed. The first layer is used as the reference while applying the via trim pattern. For instance, `trim_pg -layer M9` means to trim `M9` stripes.

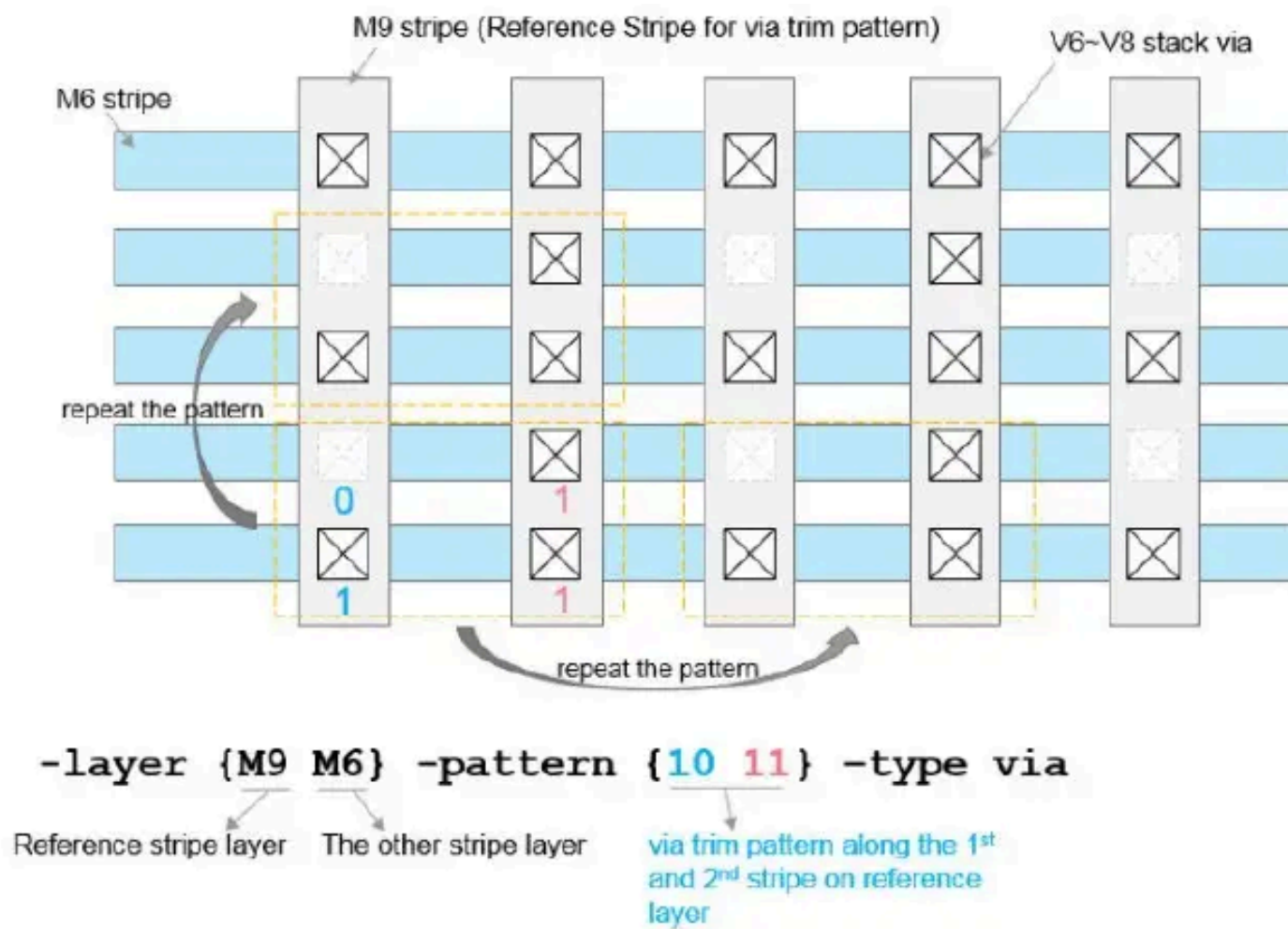
When two layer names specified, it means to trim via that connects the stripe on the two layers. The first layer stripe is used as the pattern reference, because two-dimensional pattern is needed for via. For instance, `trim_pg -layer {M9 M6}` means to trim stack via between the `M9` and `M6` stripes, with `M9` stripe as the pattern reference.

Use the `trim_pg -pattern` parameter to accept one or multiple strings.

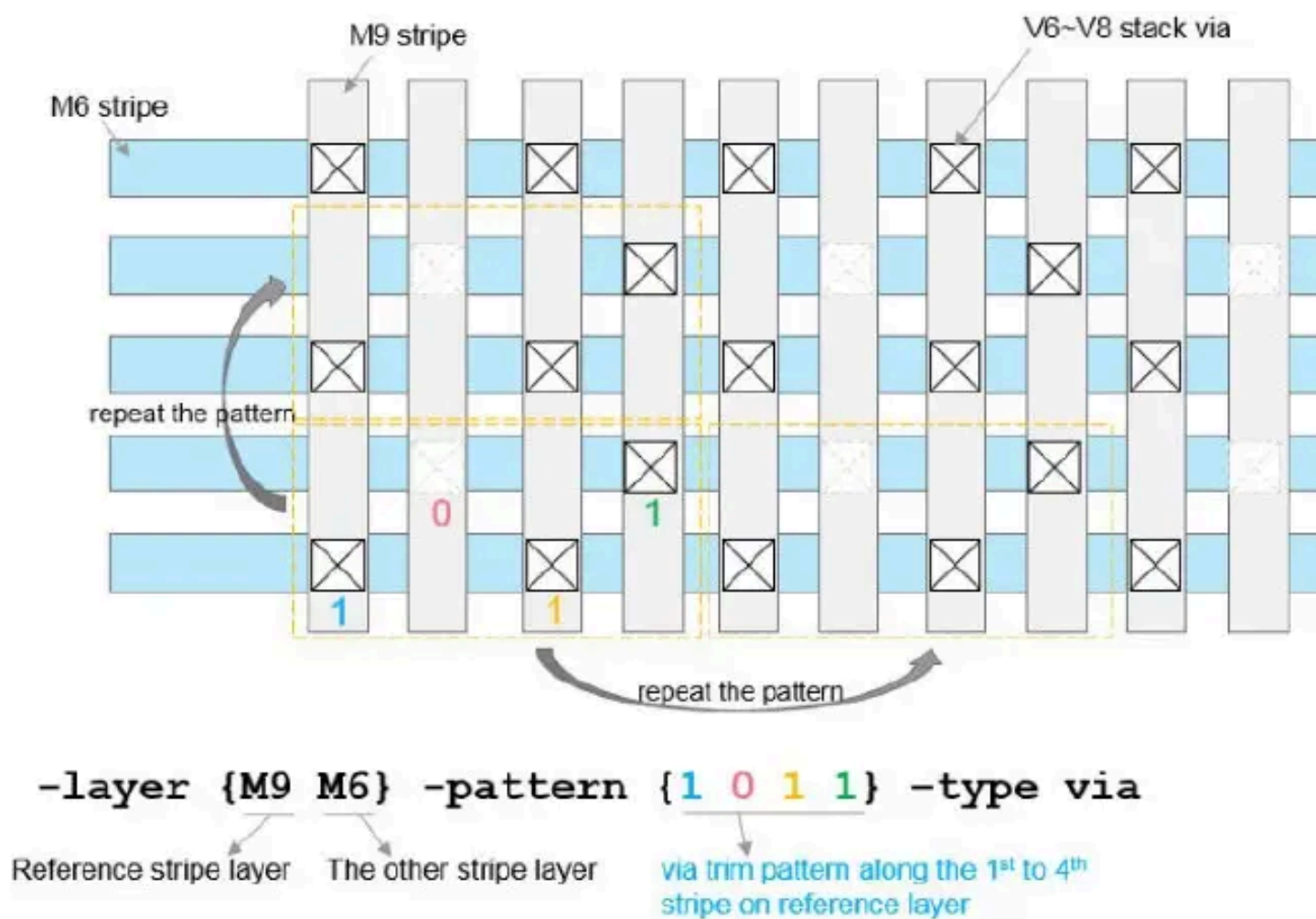
- While trimming one string pattern, `trim_pg -pattern 10` means to trim every other stripe.
- While trimming via, both one and multiple string patterns are allowed. `trim_pg -pattern {10 11}` means to trim every other via along number `1, 3, 5, 7, ...` of the reference stripe, and keep all the vias along `2, 4, 6, 8, ...` stripes.

Examples

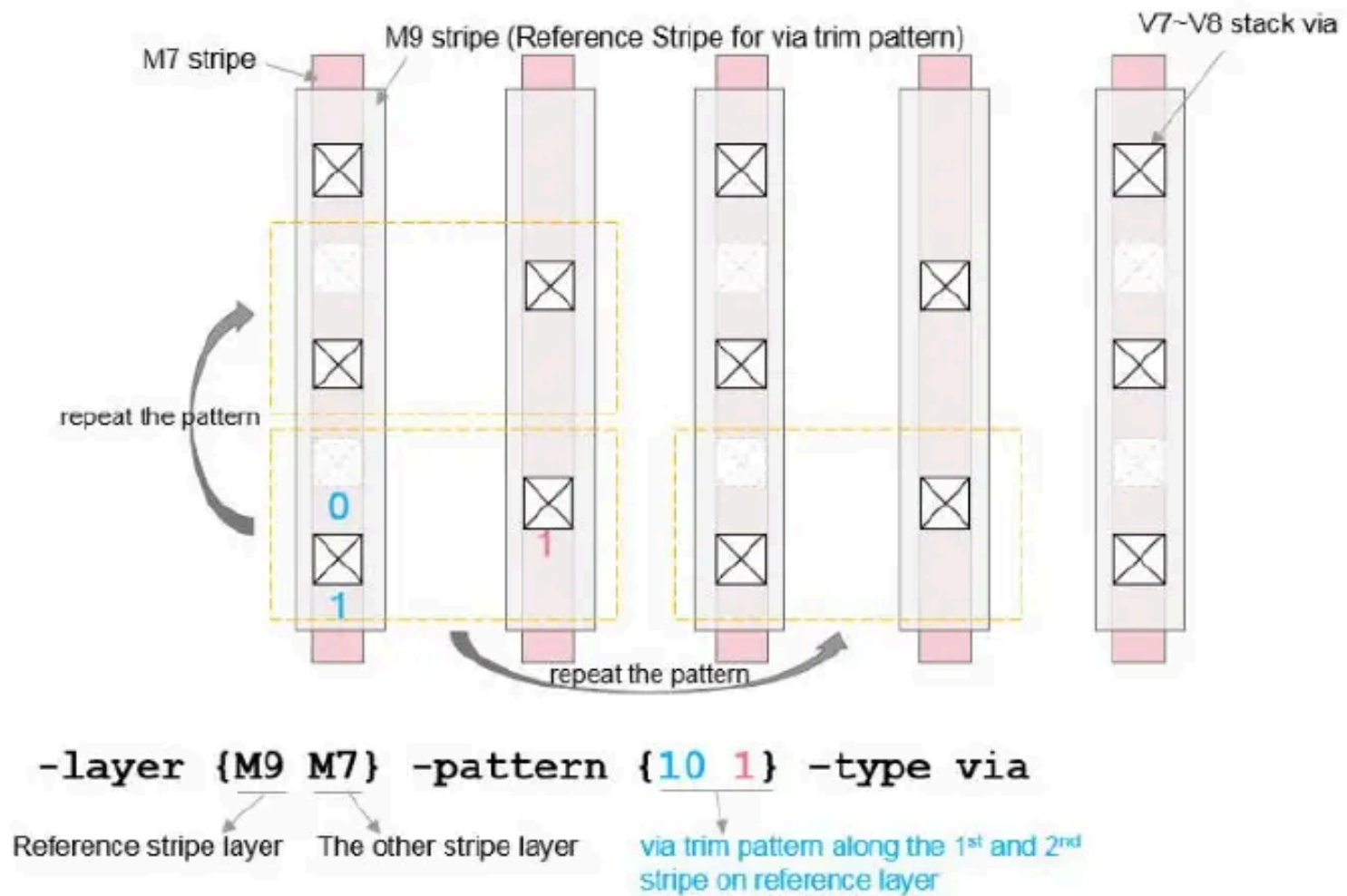
- Following is an example of trimming stack via between the `M9` and `M6` stripes:



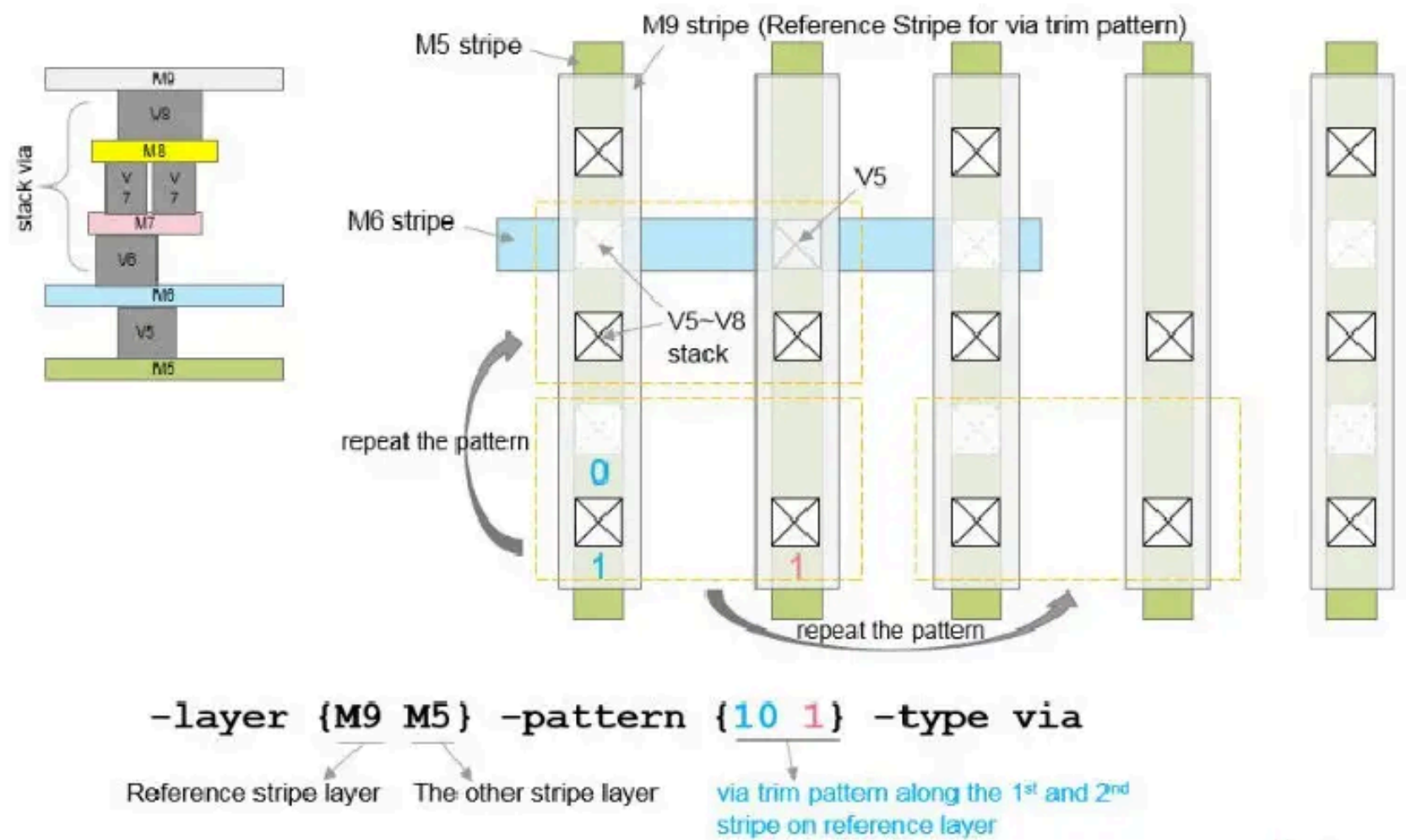
- Following is an example of trimming stack via between the M9 and M6 stripes with checker board pattern:



- Following is an example of trimming stack via between parallel stripes on M7 and M9 :



- Following is an example of trimming only the stack via that connects the stripes on the two specified layers, which are the candidates for trimming (here, V5~V8 stack are candidates, the V5 via that connects the M5 and M6 stripes, is not the candidate):



Related Information

[Power Planning and Routing](#)

Flash PG

Flash PG is a Power Planning functionality of Innovus, which generates the Power Grid (PG) structure of a design. Flash PG enhances the portability of the PG structure with the help of PG Structure Description Language (PSDL) to capture the design intent. It also simplifies the PG route creation (during the floorplanning phase) using a single command, [route_pg](#) .

Flash PG uses the PG Structure Description Language (PSDL) to capture the design intent along with the relation between the patterns and layers in a single PSDL file. The PG engineer directly uses this file with the help of the `route_pg` command to generate the PG structure, and this process saves a significant amount of time. The PSDL file can be used to map the PG structure specifications in the design.

Following are the benefits of using Flash PG for generating a PG routing flow:

- Quick PG generation in the Fast PG mode for design exploration.
- Building of the PG structures just using a single `route_pg` command. This eliminates duplicate runtime overhead and efforts for data initialization.
- Reduced output database size through database compression and multi-threading.
- Integration with the DRC checking engine for accurate DRC check in lesser time.
- Integration with the Common Coloring Engine to provide correct metal coloring.
- Integration with Nanoroute routing engine to allow localized surgical routing.
- Availability of a complete overview of the PG structures, which enable PG generation engine to do fast DRC/LVS check. You can describe the desired PG routing patterns to be instantiated for each metal layer and the arrangement between different instantiated patterns on different layers with minimal effort. This makes it easy to be used for various floorplans and design sizes.
- Automatic extraction of design rule data from the tech LEF file .

Following are the two main components of Flash PG:

- [PG Structure Description Language](#)
- [PG Network Synthesis](#)

Related Topics

- [route_pg](#)
- [Components of a PSDL File](#)

PG Structure Description Language

PG Structure Description Language (PSDL) describes a complete specification of the entire PG mesh structure with relation between wires on different layers. Flash PG uses a text-based description file written in PSDL to save these specifications.

PSDL enables you to describe the desired routing patterns for each layer, and the way these patterns are laid out in the design with respect to the other patterns from a different layer. PSDL also enables a flexible alignment of metal patterns, especially while focusing on tracking misalignments. This reduces or eliminates the attempts of identifying errors for calculating offsets and distances to maintain pattern alignment.

PSDL does not have a complete dependency on the design size and/or the floorplan. Hence, you can transfer the required PG structure between designs just with slight updates to the PSDL field values.

PSDL presents a comprehensive overview of the whole PG structure by describing the desired PG routing patterns for each layer. It also presents the relation between these patterns and various layers. PSDL has the following two main components:

- **Pattern:** Specifies the master pattern template to be used for creating the PG structure for every layer. You can use the same pattern multiple times for various layers, if the constraints are same.
- **Region:** Specifies the desired area where the PG structure will be created.

PG Structure Description Language Format

The PG Structure Description Language (PSDL) format presents an overview of the pattern, region, layer, via, and metal syntax.

Following is the format of a PSDL file showing its essential requirements:

```
{ PATTERN statement } ... # contains its name, type, direction, width, spacing, pitch,
length, which will be applied to any metal
{ PATTERN  patternName
  { TYPE ( FOLLOWPIN | STRIPE | STAPLE ) }
  [ { DIRECTION ( HORIZONTAL | VERTICAL ) } ]
  [ { WIDTH ( widthi | DEFAULT | nonDefaultRuleName ) ... } ]
  [ { SPACING ( spacingi | MINSPACING | { PITCH numPitches } ) ... } ]
  [ { LENGTH minLength maxLength } ]
}
{ REGION statement } ... # contains PG mesh area/area exclusion, layer statement to
```

```
construct PG mesh
{ REGION
    [ { AREA ( areaSetId | { xL1 yL1 xH1 yH1 } ... ) } ] [ { EXCEPTAREA statement } ]
    [ { CELL cellMasterName ... } | { BLOCK ( * | blockName ... ) } | { INSTANCE
instanceName ... ]
    [ { POWERDOMAIN powerDomainName ... } ]
    [ { DESIGNAREA | COREAREA | COREIO } ]
    [ { EXCEPTAREA exceptAreaId } ]
    [ { RING statement } ]
    { LAYER layerName [ { SNAPT0 GRID } ] [ { VIA { VIARULE ... } TO { .. } } ]
        { METAL statement } ...
    } ...
}
```

The **PATTERN Statement** part in the PSDL file format above defines the pattern name. You always need to name a pattern, to recall it and use in a metal statement, wherever needed. After defining a pattern, you need to place it in the design. **PATTERN Statement** also includes the type of the pattern, which can be a followpin, stripe, or staple.

 Staple is a short segment, which can be replicated.

The **PATTERN Statement** part also explains the direction of the pattern, which can be either horizontal or vertical. Additionally, it defines the pattern width, spacing, and length. The width between any two patterns can be varied.

The **REGION Statement** part defines the PG mesh included or excluded area. **METAL Statement** defines the offsets and the distance between the placement of various patterns. Following is the format of **METAL Statement** :

```
{ METAL metalPatternId patternName
    { NET net1 net2 ... netn }
    [ { SNAPT0 statement } ]
    [ { SIZE ( * | xnum ) ( * | ynum ) } ]
    [ { OFFSET x0 y0 [ FROM ( REGION | ZERO | CORE | DESIGN ) ] [ TO ... ] } ]
    [ { STEPDISTANCE xdist ydist [ CENTERTOCENTER ... ] } ]
    [ { FOLLOW syntax } ]
    [ { ALIGNMENT syntax } ]
    [ { UDA udaName } ]
}
```

Other than the components defined above, PSDL allows you to use some Tcl variables, along with some numerical expressions, for numeric calculations. Additionally, you can create user grids,

custom areas, and exception areas.

You can write a PSDL statement of a pattern in any order except the following:

- Define a statement to be used as a reference, before the statement that uses/calls it.
- A PG structure is generated in the same order as they appear in the PSDL file. If there are two metal statements that are trying to occupy one same track, the metal statement, which is first defined in the PSDL file, will take precedence in filling up the track with the PG wire, while the second one will fill in the remaining space.

The following notations are used in a PSDL file:

- {syntax } : Curly braces indicate the beginning and the end of a syntax or a statement.
- [syntax] : Square brackets indicate optional parameter/syntax. When this parameter/syntax is omitted, its default value is used.
- (syntax1 | syntax2) : Parentheses indicate options for the parameter/syntax. Do not use these in PSDL unless you want to represent variables or in a numeric expression.
- ... : Ellipsis indicate preceding parameter/syntax. You can use them multiple times in a PSDL file.

Related Topics

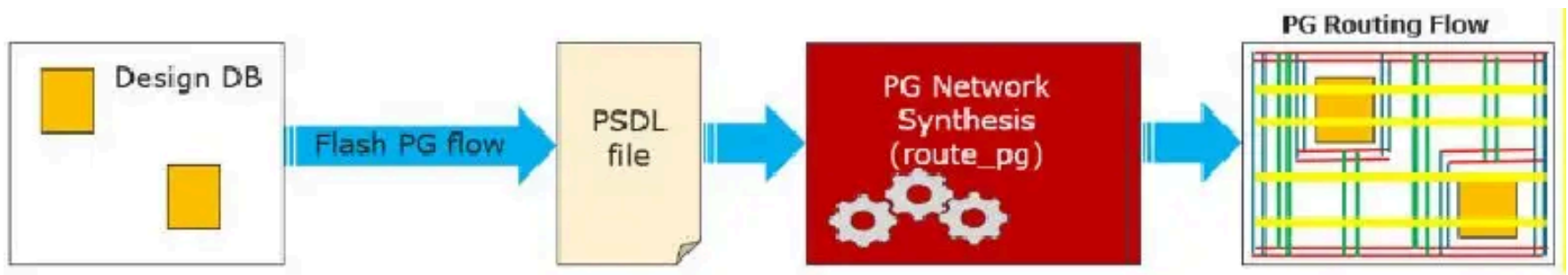
- [Components of a PSDL File](#)
- [PSDL Examples](#)

PG Network Synthesis

PG Network Synthesis (PGNS) represents a new PG routing structure generation engine that constructs a complete PG mesh based on the PSDL input. PGNS creates the PG structure of all the layers using a single Tcl command, `route_pg`.

The `route_pg` command uses the Innovus database, builds the region with the help of PSDL, where the PG structure is to be created, and parses it. This command also uses all the syntax in PSDL to build the PG structure.

The following figure presents the process of developing a PG routing flow using PGNS:

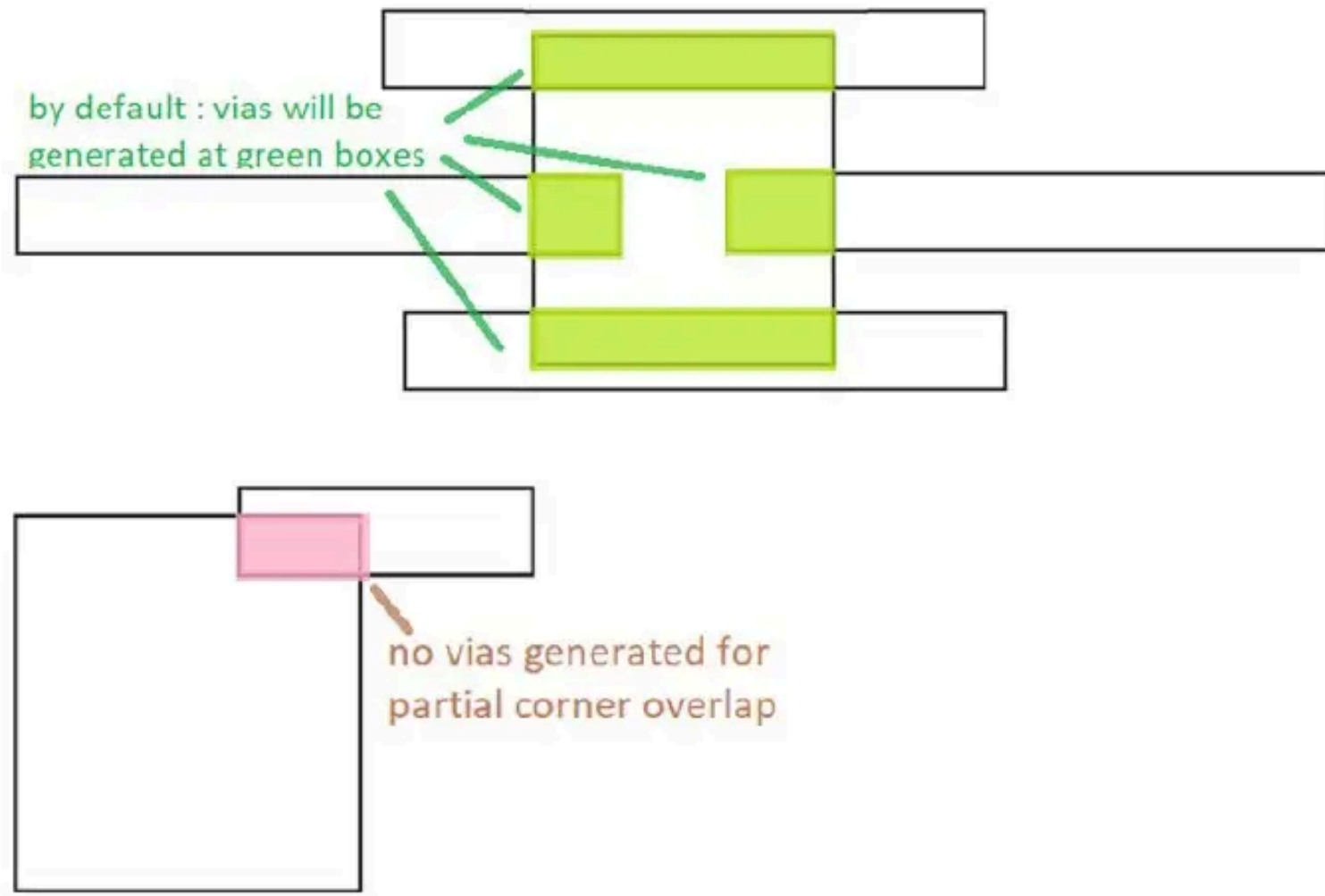


Following is the syntax of the `route_pg` command:

```
route_pg
[-help]
[-check_psdل_only]
[-delete_floating]
[-fast]
[-psdl_file file_name ]
[-skip_block_pin]
[-skip_partial_overlap_via]
[-skip_via]
[-write_psdل output_file ]
```

Following is the description of the various parameters of the `route_pg` command:

- `-help` : Prints out the command usage.
- `-check_psdل_only` : Performs sanity check only on the PSDL input file (bool, optional).
- `-delete_floating` : Removes floating wires (bool, optional).
- `-fast` : Generates PG routing flow in the Fast PG mode (bool, optional).
- `-psdl_file file_name` : Specifies the PSDL input file (string, optional).
- `-skip_block_pin` : Skips the formation of connection to the block pins (bool, optional).
- `-skip_partial_overlap_via` : Skips building of via on partial overlap intersection (bool, optional).



- `-skip_via` : Skips building via object (bool, optional).
- `-write_psdL` : Writes a PSDL file to ensure that the input numerical/string Tcl variable is correctly calculated and applied.

See the examples below:

- The following command checks the `-psdL_file` syntax only (without running the `route_pg` command):

```
route_pg -fast -psdL_file filename -check_psdL_only
```
- The following command removes a floating wire:

```
route_pg -fast -psdL_file psdL_file -delete_floating
```
- The following command does not generate via while running the `route_pg` command:

```
route_pg -fast -psdL_file psdL_file -skip_via
```
- The following command skips partial generated vias:

```
route_pg -fast -psdL_file psdL_file -skip_partial_overlap_via
```
- The following command generates a PG structure in the Fast PG mode and writes a PSDL file

during its run:

```
route_pg -fast -psdl_file      file_name  -write_psd1  output_file
```

- The above command generates a PG structure in the Complete PG mode and writes a PSDL file during its run:

```
route_pg -psdl_file      file_name  -write_psd1  output_file
```

- The above command checks the PSDL syntax and writes a PSDL file without generating the PG structure:

```
route_pg -psdl_file      file_name  -check_psd1_only -write_psd1      output_file
```

Flash PG Integration with PGNS

Flash PG with the PGNS is natively integrated with:

- FGC check for a clarified DRC output.
- Common Coloring Engine (CCE) for wire/via coloring.
- Nanoroute for detail surgical routing.
- Compressed database for data compression.
- NC protection for encryption/decryption of PSDL file. It can be used in the cases where you want to share a PSDL file only with the specific users, you can encrypt it. Run the following commands in sequence to encrypt a PSDL file:

```
encrypt psdl_file -tcl encrypted_psd1_file
```

```
route_pg -fast -psdl_file      encrypted_psd1_file
```

PGNS provides two modes for routing PG:

- [Fast PG Mode](#)
- [Complete PG Mode](#)

Fast PG Mode

Fast PG mode is a quick generation of PG mesh, which skips comprehensive DRC check and fixing. It performs an internal DRC check, and creates PG mesh with over 90% DRC clean, because the structure itself is DRC clean. The probable DRC violations can be in the cases where some complicated rules for which handling was not done, or in case there is a DRC against a fixed object. You can use it for trial-and-error run (run and fix the identified errors). Run the following command to

run PGNS in the Fast PG mode:

```
route_pg -fast -psdl_file file_name
```

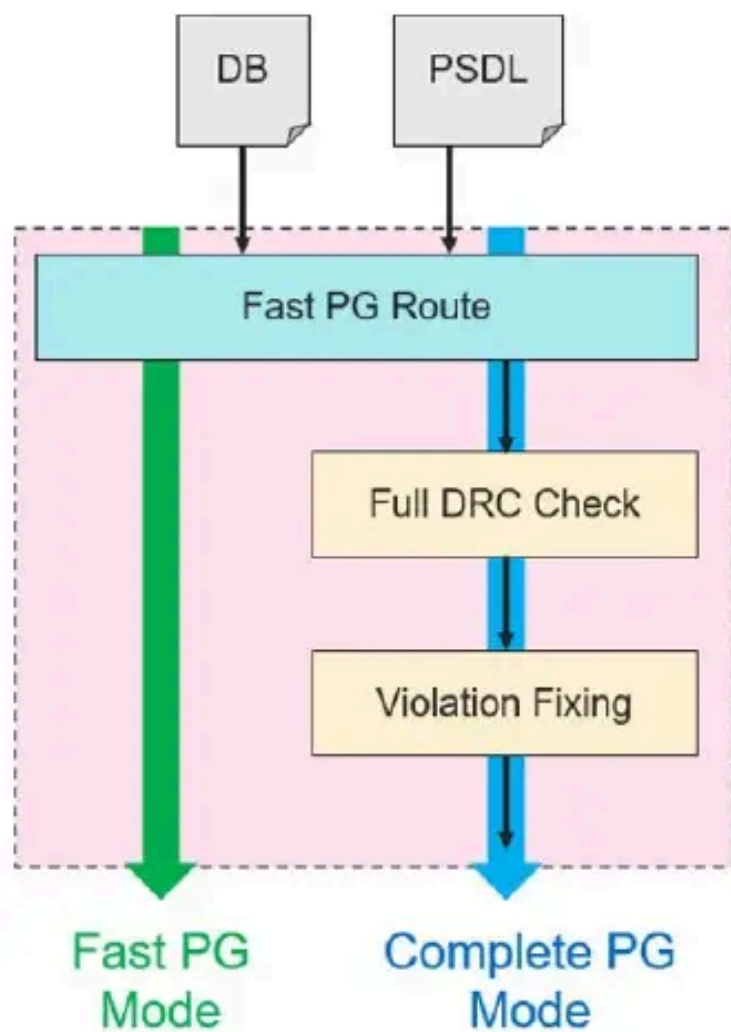
Complete PG Mode

Complete PG mode uses the `route_pg` command to generate a PG mesh. It first uses the fast PG mode of PGNS, and then runs a full DRC check for fixing the identified DRC violations. The PG mesh generated using complete PG mode is always free from all the DRC violations. This mode of PG generation is recommended when the PG structure is stable, since the runtime is longer to guarantees a DRC clean PG mesh.

Run the following command to run PGNS in the Complete PG mode:

```
route_pg -psdl_file file_name
```

The following figure presents the flows involved in the Fast PG and Complete PG modes of PGNS:



Related Topics

- [route_pg](#)
- [Flash PG](#)

Components of a PSDL File

Following is the primary format of a PSDL file:

```
[ { VARIABLE statement } ... ]  
{ PATTERN statement } ...  
[ { USERGRID statement } ... ]  
[ { AREA statement } ... ]  
[ { EXCEPTAREA statement } ... ]  
{ REGION statement } ...
```

Related Topics

- [VARIABLE Statement and Numerical Expressions](#)
- [PATTERN Statement](#)
- [USERGRID and SNAPTO Statements](#)
- [AREA Statement](#)
- [EXCEPTAREA Statement](#)
- [REGION Statement](#)
- [PG Structure Description Language](#)
- [PSDL Examples](#)

VARIABLE Statement and Numerical Expressions

Following is the VARIABLE statement in a PSDL file:

```
{ VARIABLE {   varName   ( value | { TCLVAR   tclVarName   } ) } ... }
```

Here:

- `varName` : Specifies any valid 'C-style' variable name.
- `value` : Specifies any numeric value or a numerical expression.

- `tclVarName` : Specifies a Tcl variable name, where the value is obtained.

You can replace any coordinate value with a variable in PSDL by using the following syntax:

```
... ( varName ) ...
```

You can replace any numerical value with a numerical expression in PSDL using the following syntax:

```
( numericalExpression | varName )
```

The value of the replaced variable can be recalled by using the `varName` enclosed by parenthesis. Ensure to define a variable before recalling it in an expression. See the examples below:

- Following is an example of defining a numerical expression in a PSDL file:

```
{ VARIABLE { x1 0.048 } { x2 1.025 } { y 0.110 } }
{ METAL met1 pat1 { NET vdd } { OFFSET ((x2) + 2 * (x1)) (y) } ... }
{ METAL met2 pat2 { NET vss } { OFFSET (x2) (y) } ... }
```

- Following is an example of defining a Tcl variable in the Tcl file and reading it inside the PSDL file. You can apply this Tcl variable only for a single value.

```
restoreDesign test.enc test
set abc 0.040
route_pg -fast -psdl_file test.psdL # reading the abc variable inside PSDL
{ VARIABLE { x 0.02 } { y (0.104 * 2) } { pitch 0.560 } { a {TCLVAR abc} } }
{ METAL met1 pat1 { NET vdd } { OFFSET ((x) + 2 * (a)) (y) } ... }
```

- Following is an example of defining a Tcl variable as a list of rectangles/polygons, which can be used in an AREA statement:

```
restoreDesign test.enc test
set boxes_list { {61 45 89 68} {61 20 77 39} }
route_pg -fast -psdl_file test.psdL # reading the boxes_list variable inside PSDL
{ REGION { AREA abcd { TCLVAR boxes_list } }
  { LAYER METAL1 { EXCEPTAREA { BLOCK * } }
    { METAL m1_fpin_vdd m1_followpin { NET {TCLVAR vdd_net} } }
    { METAL m1_fpin_vss m1_followpin { NET {TCLVAR gnd_net} } }
  } ...
}
```

- Following is an example of defining a Tcl variable as a list of names for CELL/BLOCK/INSTANCE/POWERDOMAIN, which can be used in an AREA statement:

```
restoreDesign test.enc test
set newInst { test5 test6 }
route_pg -fast -psdl_file test.psdl          # reading the abc variable inside PSDL
{ REGION  { AREA ps_cell { INSTANCE {TCLVAR newInst } } }
{ LAYER METAL2 { SNAPT0 GRID }
{ METAL m2_ps m2_staple { NET VDD } { OFFSET 2.4 0.8 } { STEPDISTANCE 2.2 2.7 } }
} ...}
```

- Following is an example of defining a Tcl string variable in a Tcl file, which can be read inside a PSDL file:

```
restoreDesign test.enc test
set gnd_net [get_db [get_db nets -if {.is_ground == true }] .name]
set vdd_net [dbGet [dbGet top.pgNets.name VDD -p].name]
route_pg -fast -psdl_file test.psdl          # reading the vdd/gnd_net string variable
inside PSDL
{ REGION  { COREAREA }
  { LAYER METAL1 { EXCEPTAREA { BLOCK * } }
    { METAL m1_fpin_vdd m1_followpin { NET {TCLVAR vdd_net} } }
    { METAL m1_fpin_vss m1_followpin { NET {TCLVAR gnd_net} } }
  } ...
}
```

Related Topics

- [route_pg](#)
- [Components of a PSDL File](#)
- [PG Structure Description Language](#)
- [PSDL Examples](#)

PATTERN Statement

- [PATTERN FOLLOWPIN Statement](#)
- [PATTERN STRIPE Statement](#)

- **PATTERN STAPLE Statement**

The PATTERN statement defines the master pattern template to be used for constructing the PG structure for each layer. You can use the same pattern multiple times for different layers, if the constraints are the same. Following is the PATTERN statement in a PSDL file:

```
{ PATTERN patternName
  { TYPE ( FOLLOWPIN | STRIPE | STAPLE ) }
  [ { DIRECTION ( HORIZONTAL | VERTICAL ) } ]
  [ { WIDTH ( width _i | DEFAULT | TRACKWIDTH ) ... } ]
  [ { SPACING ( spacing _i | MINSPACING | { PITCH numPitches } ) ... } ]
  [ { LENGTH [ minLength ] ( maxLength | AUTO ) [ HARD ] } ]
}
```

Following are the various arguments used in the PATTERN statement:

- **TYPE:** Indicates the pattern type as FOLLOWPIN, STRIPE, or STAPLE.
- **DIRECTION:** Indicates the direction of the metal as HORIZONTAL or VERTICAL. If this option is not specified, the routing direction of the layer is used.
- **WIDTH:** Specifies the number of metals to be placed in the current pattern as well as the width for each metal in the pattern. If the DEFAULT value is specified (instead of actual width value), then the default metal width of the given layer is used. Similarly, non-default width is used if the specified NONDEFAULTRULE name is specified. The TRACKWIDTH option uses the track-defined width as the value of the wire width.
- **SPACING:** Specifies the spacing required between two adjacent metals. This argument is optional if there is a single metal. The value of the spacing can be an absolute spacing value or one of the followings:
 - **TRACK numTracks** : Specifies that the spacing is defined by the number of routing tracks away from the previous metal.
 - **PITCH numPitches** : Specifies that the spacing is defined by the number of routing pitches away from the previous metal (measured from center-to-center).



The PITCH spacing value is retrieved from the tech LEF file, which is the center-to-center distance between two wires.

- **MINSPACING:** Uses minimum spacing defined in the library for the current layer based on the specified wire width and length. Spacing is the closest edge measurement between two adjacent metals.

- LENGTH: Specifies the minimum and maximum metal length for the STAPLE type.
 - When AUTO is specified, the staple length is calculated based on the minimum area rule (MAR) or vias and its enclosure. This attribute is ignored for FOLLOWPIN or STRIPES.
 - When [HARD] is specified, Flash PG honors user-specified staple length, with ignoring its DRC length violation, which can be fixed at a later stage. However, staple wire will be deleted when there is a DRC with an existing OBS and routing blockage.

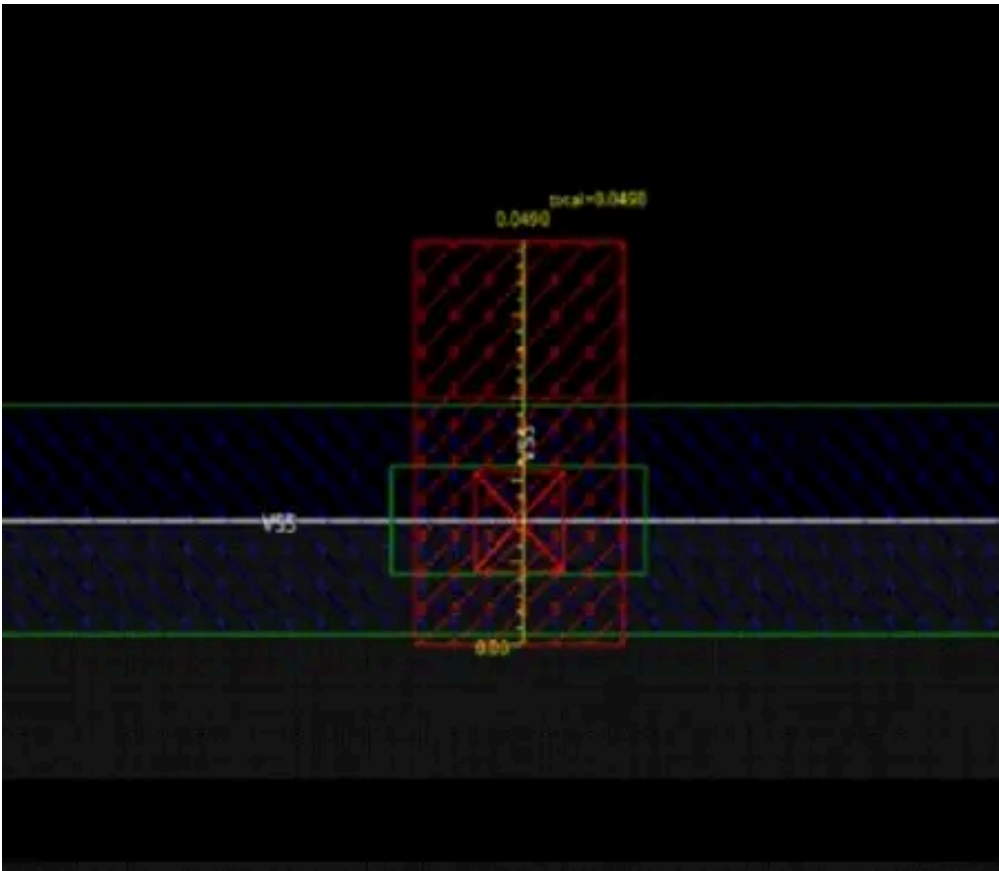
Following is an example of the PATTERN statement where Flash PG honors user-specified staple length:

```
{ PATTERN pat0 { TYPE FOLLOWPIN } }  
{ PATTERN pat1 { TYPE STAPLE } { DIRECTION VERTICAL } { WIDTH 0.29 } { LENGTH 0.049  
HARD } }  
{ REGION { COREAREA }  
  { LAYER M0  
    { METAL m0_vdd pat0 { NET VDD } }  
    { METAL m0_vss pat0 { NET VSS } }  
  }  
  { LAYER METAL2  
    { METAL m2_vdd pat1 { NET VDD } { OFFSET 4.435 5.74 } { STEPDISTANCE 8.550 5.74  
  } }  
    { METAL m2_vss pat2 { NET VSS } { OFFSET 4.435 2.87 } { STEPDISTANCE 8.550 5.74  
  } }  
  }  
}
```



Since [HARD] is specified, METAL2 pat1 staple length is kept at 0.049, and not adjusted to 0.060 DRC clean value.

Following is the output of the above example:



PATTERN FOLLOWPIN Statement

Following is the PATTERN FOLLOWPIN statement in a PSDL file:

```
{ PATTERN  patternName
  { TYPE  FOLLOWPIN }
} # width is determined by the std-cells' PG pin
```

Following is an example of the PATTERN FOLLOWPIN statement:

```
{ PATTERN m1_followpin
  { TYPE FOLLOWPIN }
} # width is determined by the std-cells' PG pin
```

PATTERN STRIPE Statement

Following is the PATTERN STRIPE statement in a PSDL file:

```
{ PATTERN patternName
  { TYPE  STRIPE }
  [ { DIRECTION ( HORIZONTAL | VERTICAL ) } ]
  { WIDTH  ( width  _i | DEFAULT | TRACKWIDTH ) ... }
  [ { SPACING  ( spacing  _i | MINSPACING | { TRACK numTracks }
    | { PITCH numPitches } ) ... } ]
```

}

 The pitch SPACING value is retrieved from the tech LEF file, which is the center-to-center distance between two wires.

See the examples below:

- Following is an example of the PATTERN STRIPE statement:

```
{ PATTERN stripe_example
{ TYPE STRIPE }
  { DIRECTION HORIZONTAL }
  { WIDTH DEFAULT DEFAULT }
  { SPACING { PITCH 2 } }
}
```

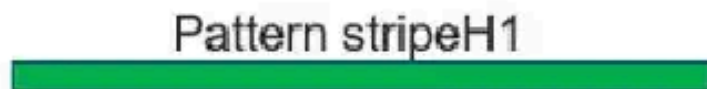
Following is the output of the above example:



- Following is an example of creating a horizontal global stripe with 0.36 width:

```
{ PATTERN stripeH1 { TYPE STRIPE } { DIRECTION HORIZONTAL } { WIDTH 0.36 } }
```

Following is the output of the above example:

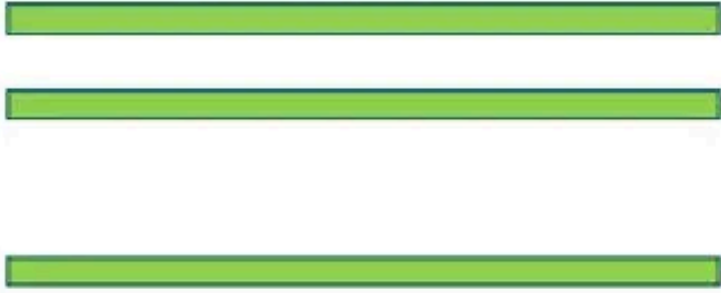


- Following is an example of creating a group of three horizontal global stripes with 0.36 width each and different spacing:

```
{ PATTERN stripeH2 { TYPE STRIPE } { DIRECTION HORIZONTAL } { WIDTH 0.36 0.36 0.36 }
{ SPACING 0.5 0.3 } }
```

Following is the output of the above example:

Pattern stripeH2



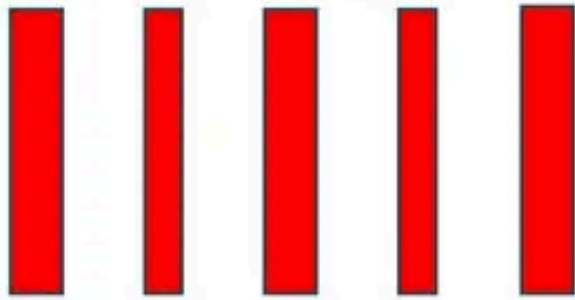
- Following is an example of creating a group of five vertical global stripes with different widths and same spacing:

```
{ PATTERN stripeV1 { TYPE STRIPE } { DIRECTION VERTICAL } { WIDTH 0.36 0.30 0.36
0.30 0.36 } { SPACING 0.5 } }
```

 The SPACING value is measured from edge to edge.

Following is the output of the above example:

Pattern stripeV1



PATTERN STAPLE Statement

Following is the PATTERN STAPLE statement in a PSDL file:

```
{ PATTERN  patternName
  { TYPE  STAPLE }
  [ { DIRECTION ( HORIZONTAL | VERTICAL ) } ]
  { WIDTH  (   widthi | DEFAULT | TRACKWIDTH )      ... }
  [ { SPACING  (   spacingi | MINSPACING | { TRACK numTracks }
                  | { PITCH  numPitches  } ) ... } ]
  [ { LENGTH [   minLength ]   ( maxLength | AUTO )   } ]
}
```

In this statement:

The PITCH SPACING value is retrieved from the tech LEF file, which is the center-to-center

distance between two wires.

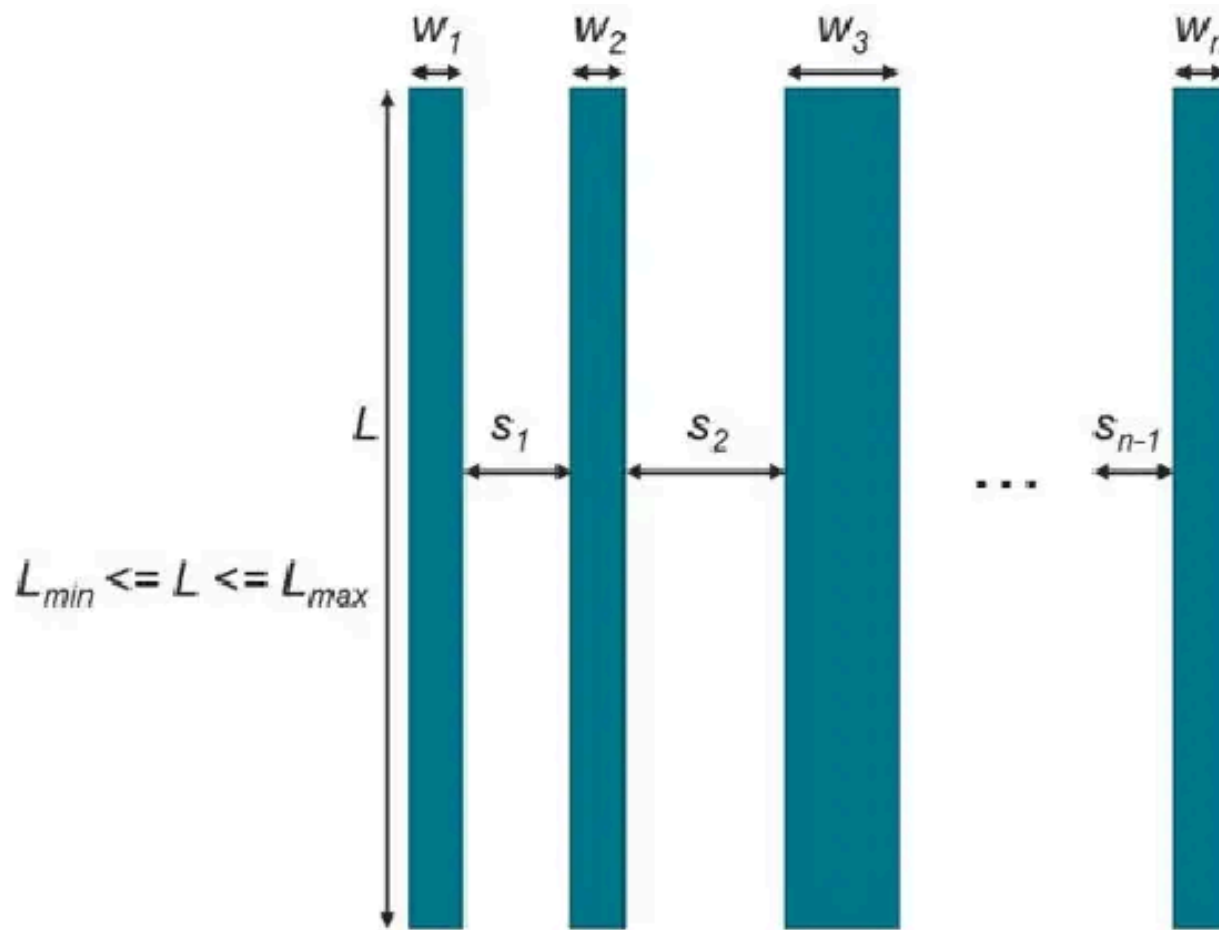
- When only maxLength is specified, both maxLength and minLength have the same value.
- When both maxLength and minLength are less than the Minimum Area Rule (MAR) value; MAR will be used for both minLength and maxLength to ensure a DRC clean result.
- When only minLength is less than MAR, then MAR will be used for minLength
- When AUTO is used, staple length is the longest value calculated based on MAR and specified wire width or vias and its closure.

See the examples below:

- Following is an example of the PATTERN STAPLE statement with different WIDTH, SPACING, and LENGTH values:

```
{ PATTERN  staple_example
{ TYPE  STAPLE  }
  { DIRECTION  VERTICAL  }
  { WIDTH  w1 w2 w3 ... wn }
  { SPACING  s1 s2 ... sn-1 }
  { LENGTH  Lmin Lmax }
}
```

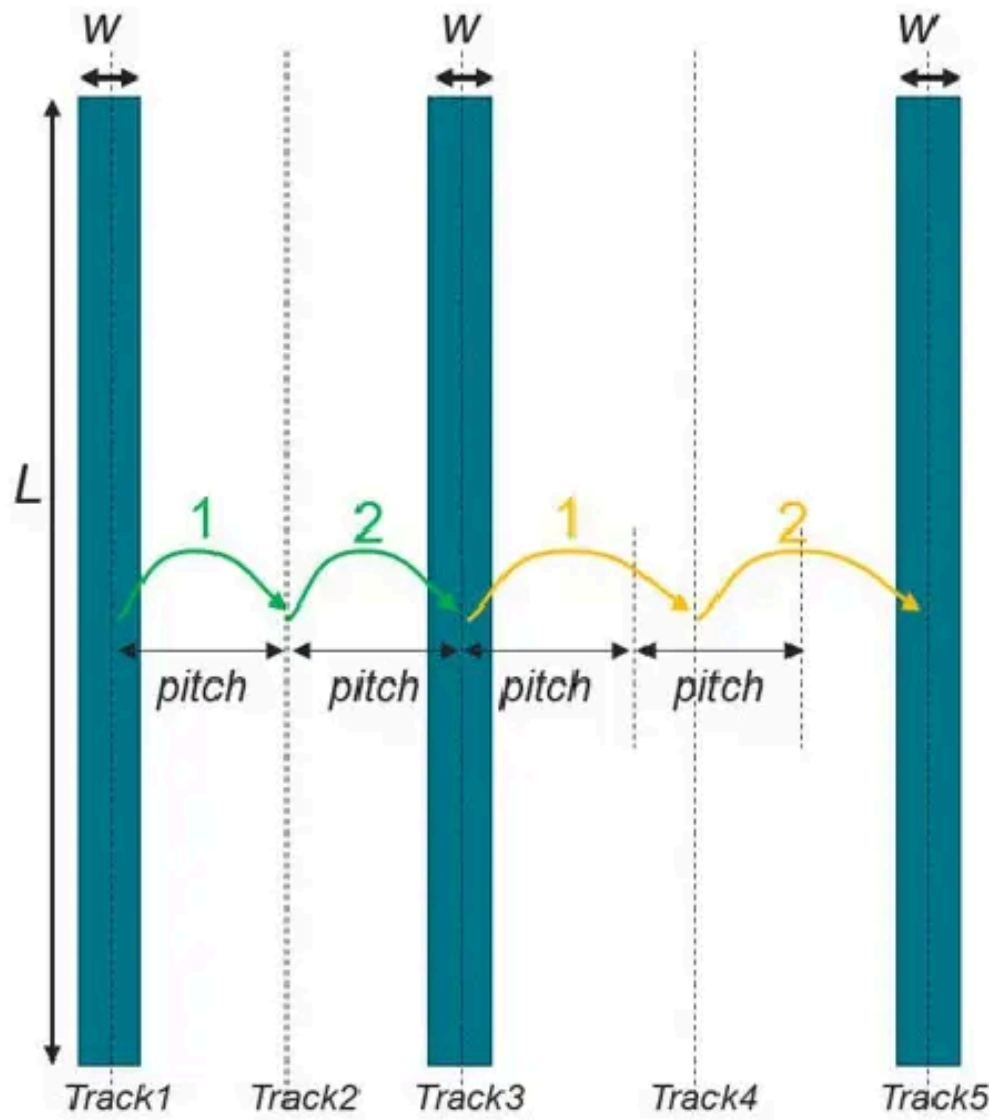
Following is the output of the above example:



- Following is an example of the PATTERN STAPLE statement with vertical DIRECTION, default WIDTH, SPACING as TRACK 2, and LENGTH as L:

```
{ PATTERN  track_example
{ TYPE  STAPLE  }
  { DIRECTION  VERTICAL  }
  { WIDTH  DEFAULT DEFAULT DEFAULT  }
  { SPACING { TRACK 2 } { TRACK 2 } }
  { LENGTH  L L  }
}
```

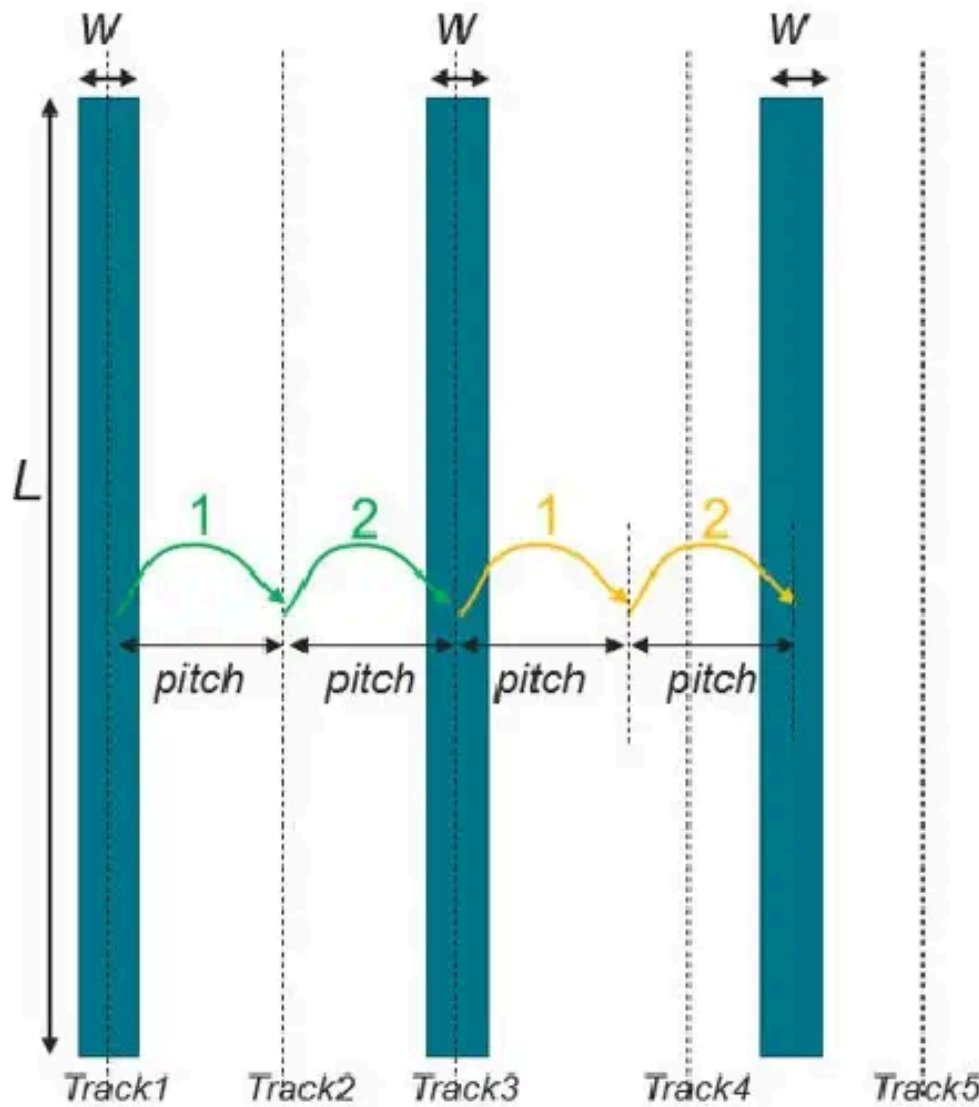
Following is the output of the above example:



- Following is an example of the PATTERN STAPLE statement with vertical DIRECTION, default WIDTH, SPACING as PITCH 2, and LENGTH as L:

```
{ PATTERN pitch_example
{ TYPE STAPLE }
  { DIRECTION VERTICAL }
  { WIDTH DEFAULT DEFAULT DEFAULT }
  { SPACING { PITCH 2 } { PITCH 2 } }
  { LENGTH L L }
}
```

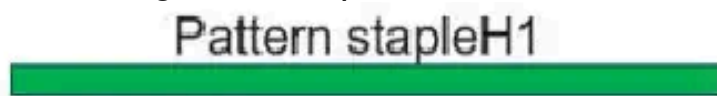
Following is the output of the above example:



- Following is an example of creating a vertical staple stripe of 0.36 width and 1.7 fixed length:

```
{ PATTERN stapleH1 { TYPE STAPLE } { DIRECTION HORIZONTAL } { WIDTH 0.36 } {  
LENGTH 1.70 1.70} }
```

Following is the output of the above example:



- Following is an example of creating a group of three staple stripes with 0.36 width each, different spacing values, 1.7 fixed length. When there are OBS/DRC violations on certain locations, staples will not be generated.

```
{ PATTERN stapleH2 { TYPE STAPLE } { DIRECTION HORIZONTAL } { WIDTH 0.36 0.36 0.36  
}  
{ SPACING 0.5 3 } { LENGTH 1.70 1.70} }
```

Following is the output of the above example:

Pattern stapleH2



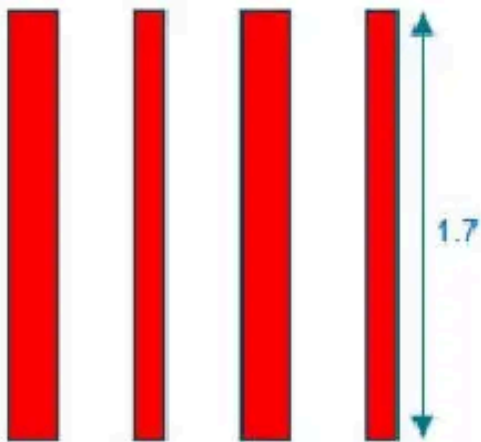
- Following is an example of creating a group of four vertical global stripes with different widths, same spacing, 1.7 maxLength, and 1.5 minLength (when there is OBS/DRC):

```
{ PATTERN stapleV1 { TYPE STAPLE } { DIRECTION VERTICAL } }  
{ WIDTH 0.36 0.30 0.36 0.30 } { SPACING 0.5 } { LENGTH 1.5 1.7 }
```

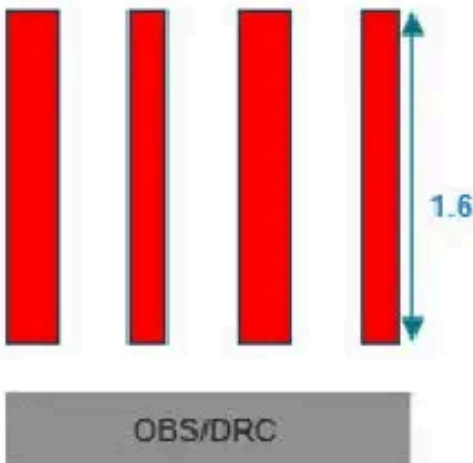
 The SPACING value is measured from edge to edge.

Following is the output of the above example:

Pattern stripeV1
(without OBS or DRC)



Pattern stripeV1
(with OBS/DRC)



Related Topics

- [route_pg](#)
- [Components of a PSDL File](#)
- [PG Structure Description Language](#)
- [PSDL Examples](#)

USERGRID and SNAPTO Statements

The USERGRID statement specifies a user-defined uniform grid line for snapping PG wires to the grid or for using as the metal pattern reference location. This grid is independent of the routing the TRACK definition specified in the design. Following is the USERGRID statement in a PSDL file:

```
{ USERGRID  gridName  startOffset  stepDistance  }
```

Here:

- **gridName** : Specifies a unique identifier to represent the specified user grid.
- **startOffset** : Specifies the starting coordinate of the grid lines
- **stepDistance** : Specifies the distance between two grid lines.

You can also use the USERGRID statement as a reference grid in the FOLLOW statement, at the same time with snapping to its metal track, as shown below:

```
{FOLLOW ( * | xRefPatternId| USERGRID gridName) ( * | yRefPatternId| USERGRID gridName)
}
```

See the examples below:

- Following is an example of using the FOLLOW USERGRID, where the CENTER of the lowest metal will be on the left/bottom grid, when ALIGNMENT is set to LOW:

```
{ USERGRID grid1 0.100 1.500 } #creates grid1: 0.100, 1.600, 3.100, 4.600, 6.100,
...
{ REGION { COREAREA }
    { LAYER METAL3 { SNAPTO GRID } ### snapping all METAL3 to its track
        { METAL m3_vdd m3_staple { NET VDD } { FOLLOW USERGRID grid1 m1_fpin_vdd}
    { ALIGNMENT CENTER CENTER } }
    }
}
```

Following is the output of the above example:

```
grid1: 0.100, 1.600, 3.100, 4.600, 6.100, ...
```

- Following is an example of creating a snapping grid from 0.100 with 1.500 interval:

```
{ USERGRID  grid1 0. 100 1.500 }
```

Following is the output of the above example:

```
grid1: 0.100, 1.600, 3.100, 4.600, 6.100, ...
```

- Following is an example of creating a snapping grid from 20.000 with 2.200 interval:

```
{ USERGRID  grid2 20. 000 2.200 }
```

Following is the output of the above example:

```
grid2: 20.000, 22.200, 24.400, 26.600, 28.800, 31.000, ...
```

You can use a predefined USERGRID inside a SNAPTO phrase in either a LAYER or a METAL statement. This indicates the snapping grid for all wires in the corresponding LAYER or METAL respectively. See the example below of snapping all wires in the M1 layer to the grid1 routing grid:

```
{ USERGRID    grid1  0.100 1.500 }  
... { LAYER    M1 { SNAPTO USERGRID    grid1  } ... }
```

SNAPTO Statement

The SNAPTO statement defines the PG wire snapping behavior, when it is used in the design. It allows snapping of wires to the:

- Predefined TRACKs
- Half-grid in between two TRACKs
- Mask1 or Mask2 grid
- User-specified grid, as indicated in the USERGRID statement

The SNAPTO statement defined in the LAYER statement is applied to all the METALs inside the LAYER statement, unless the SNAPTO statement is also defined in the METAL statement (where one in METAL statement will override the behavior defined by the LAYER statement). Following is the SNAPTO statement in a PSDL file:

```
{ SNAPTO ( NONE | GRID | HALFGRID | MASK (1|2) | USERGRID    gridName  
          | { USERGRID    statement  } ) }
```

Following are the SNAPTO statement snapping options:

- **NONE** : Prevents snapping of a metal layer to any grid. If you specify the SNAPTO statement with the NONE option in a metal layer, it disables snapping in it.
- **GRID** : Snaps the center of PG wires to the routing tracks, as defined in the design.
- **HALFGRID** : Snaps the center of the PG wires to the center line between two routing tracks.
- **MASK 1|2** : Snaps the center of the PG wires to the center line of MASK1 and MASK2.
- **USERGRID gridName** : Snaps the center of the PG wires to the predefined user grid (gridName) that has been defined previously in the USERGRID statement.
- **{ USERGRID statement }** : Creates a user-defined grid first, as specified in the inline

USERGRID statement, and then uses this grid to snap the center of the PG wires. This newly created grid can also be used for snapping for other layers or metals, which will be defined later.

Following is an example of the SNAPTO statement in the definition of various metal layers in a PSDL file:

```
{ USERGRID grid1 0.100 1.200 }      # Creating user grid grid1 with offset 0.100 and step
1.200
{ REGION
    { LAYER M5 { SNAPTO GRID }
        { METAL m5_one ... }      # m5_one will be snapped to routing TRACKS (same as
ONGRIDONLY )
        { METAL m5_two ... { SNAPTO NONE } }      # m5_two will not be snapped to any
grid
        { METAL m5_three ... { SNAPTO USERGRID grid1 } } }      # m5_three will be
snapped to user grid grid1
    { LAYER M6
        { METAL m6_one ... { SNAPTO { USERGRID grid2 0.200 2.400 } } }      # m6_one
will be snapped to user grid grid2
        { METAL m6_two ... { SNAPTO USERGRID grid2 } } }      # m6_two will be snapped
to user grid grid2
```

Related Topics

- [route_pg](#)
- [Components of a PSDL File](#)
- [PG Structure Description Language](#)
- [PSDL Examples](#)

AREA Statement

Following is the AREA statement in a PSDL file:

```
{ AREA areaSetId
    [ { EXPAND ( allSide | leftRightSide topBottomSide | left bottom right top
        | { LEFT left } { BOTTOM bottom } { RIGHT right } { TOP top } ) } ]
    [ { MERGE distance [ (METALDIRECTION ( VERTICAL | HORIZONTAL | ALL )) | ( LAYER
layer1 ... ) } ]
    ( { xL1 yL1 xH1 yH1 } | { x1 y1 x2 y2 x3 y3 x4 y4 ... } | { CELL cellMasterName ... }
```

```

        | { BLOCK ( * |      blockName      | blockinstanceName      ...) } | { INSTANCE
instanceName ... }
        | { POWERDOMAIN ( * | powerDomainName ... ) } | {TCLVAR tclVarName} ) ...
    | { COREAREA } | { DESIGNAREA } | { PADAREA } | { COREIO } | { ROWONLY }
}

```

The MERGE option in the AREA statement can be used with boxes, poly, cell, block, instance, power domain, and tclvar to merge the area segments, when the boxes are within `mergeDistance` . The MERGE option helps you in extending the metal regions between merged REGION. The MERGE distance option specifies the distance between the nearest boxes, CELL, BLOCK, INSTANCE, or POWERDOMAIN to be merged. Here, certain metal layers, all vertical direction metals, all horizontal direction metals, or all metal layers need to be extended for boxes, CELL, BLOCK, INSTANCE, or POWERDOMAIN meeting the merge distance. The default value of the METALDIRECTION option is ALL.

See the examples below:

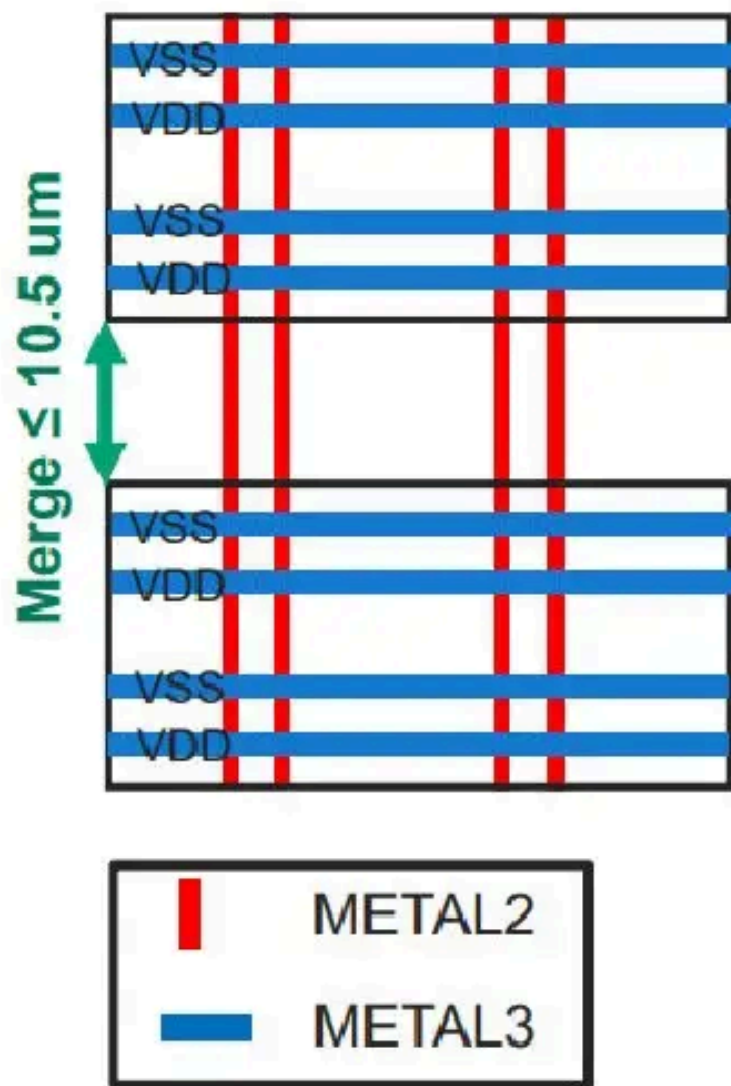
- Following is an example of merging two METAL patterns within a VERTICAL distance of 10.5 um:

```

{ REGION
    { AREA m6_m17_area {30 60 60 80} {30 30 60 50} }
    { MERGE 10.5 ( METALDIRECTION VERTICAL ) }
    { LAYER METAL2 { SNAPTO GRID }
        { METAL m2_pat m2_stripe { NET VDD VSS } { OFFSET 4.435 * FROM REGION } {
STEPDISTANCE 8.55 * } }
    }
    { LAYER METAL3
        { METAL m3_pat m3_stripe { NET VDD VSS } { OFFSET * 1.0 FROM REGION } {
STEPDISTANCE * 1.5 } }
    }
}

```

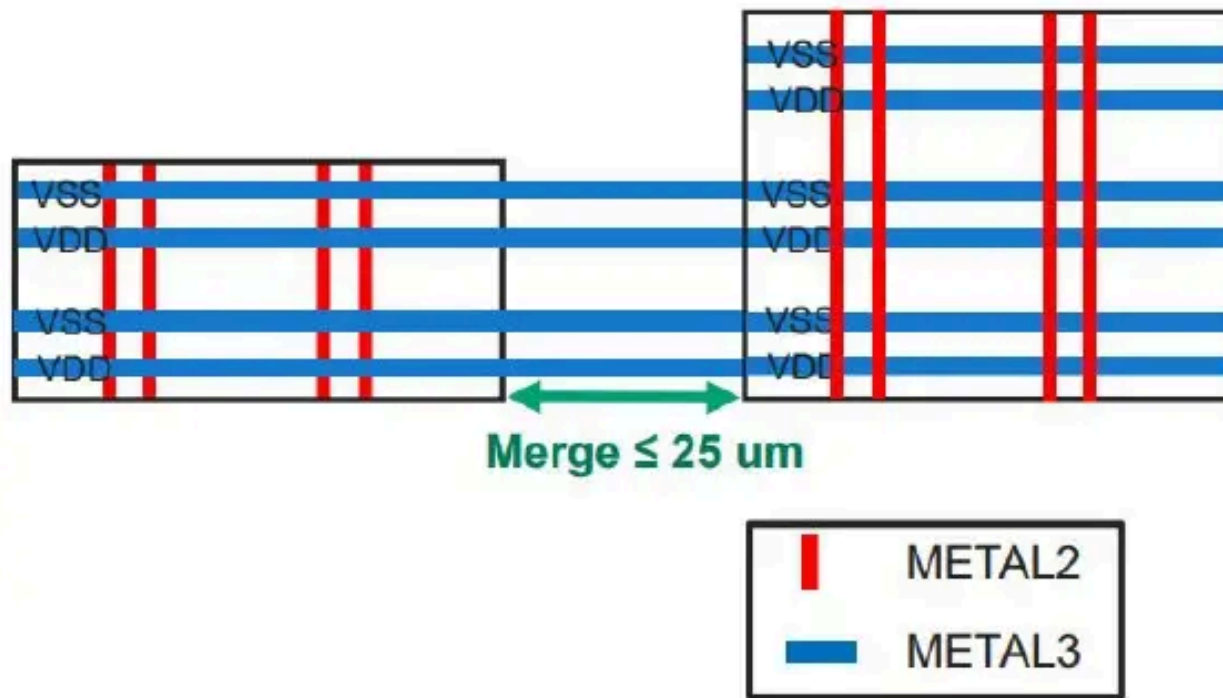
Following is the output of the above example:



- Following is an example of merging two METAL patterns within a HORIZONTAL distance of 25 um:

```
{ REGION
  { AREA m6_m17_area {30 60 60 80} {50 60 80 100} }
  { MERGE 25 ( LAYER METAL3 ) }
  { LAYER METAL2 { SNAPT0 GRID }
    { METAL m2_pat m2_stripe { NET VDD VSS } { OFFSET 4.435 * FROM REGION } {
STEPDISTANCE 8.55 * } }
  }
  { LAYER METAL3
    { METAL m3_pat m3_stripe { NET VDD VSS } { OFFSET * 1.0 FROM REGION } {
STEPDISTANCE * 1.5 } }
  }
}
```

Following is the output of the above example:



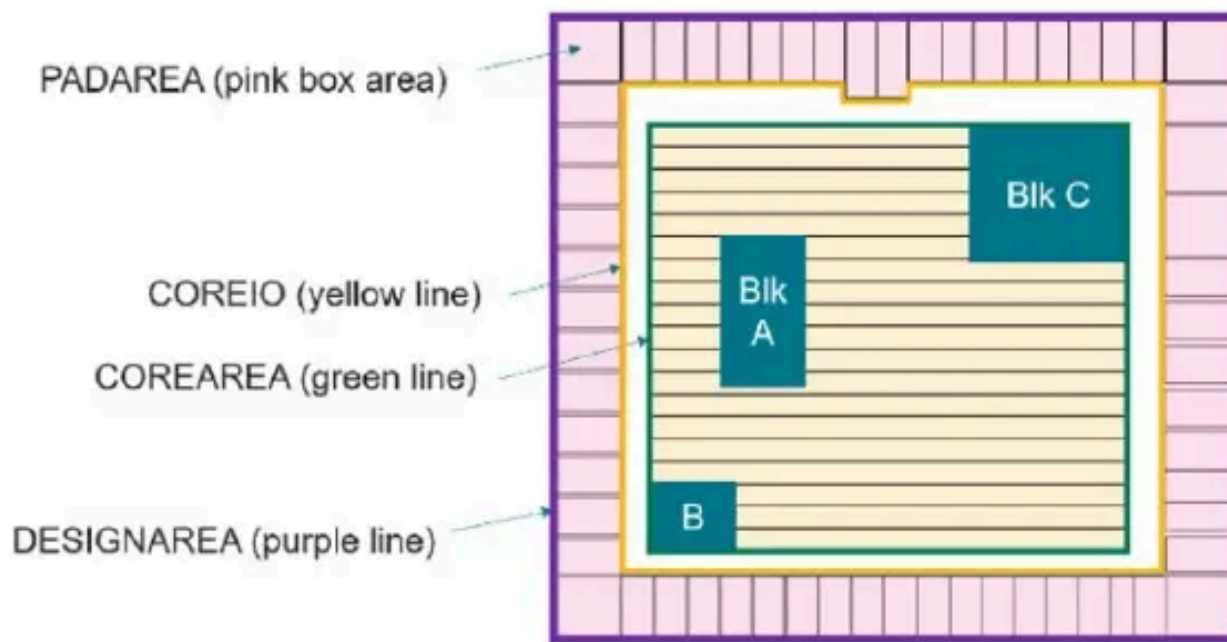
Following are the rectilinear area source options used in the AREA statement:

- { `xL1 yL1 xH1 yH1` } : Indicates the coordinates of a rectangle specified by its two corner points.
- { `x1 y1 x2 y2 x3 y3 x4 y4 ...` } : Indicates a rectilinear polygon specified by a list of points tracing the polygon boundary.
- { `CELL cellMasterName ...` } : Specifies the complete instance area whose cell master matches with `cellMasterName` in the list.
- { `BLOCK ( * | blockName | blockinstanceName ... )` } : Specifies the complete BLOCK MACRO type instance that matches with `blockName` in the list, or with all BLOCK MACROs, if `*` is specified.
- { `INSTANCE instanceName ...` } : Specifies the complete instance area that matches with the `instanceName` in the list.
- { `POWERDOMAIN ( * | powerDomainName ... )` } : Specifies the complete power domain region that matches with the `powerDomainName` in the list, or with all the non-default power domains, if `*` is specified.
- { `TCLVAR tclVarName` } : Specifies that the value of the `tclVarName` Tcl variable will be used to define the rectilinear area for the specified AREA statement. The Tcl variable value must be set according to one of the following formats:
 - `set tclVarName { xL1 yL1 xH1 yH1 }` : Indicates setting a simple rectangular area

defined by $(x_{L1}, y_{L1}) - (x_{H1}, y_{H1})$.

- `set tclVarName { x1 y1 x2 y2 x3 y3 x4 y4 ... }` : Indicates setting a single rectilinear polygon area defined by (x_1, y_1) , (x_2, y_2) , (x_3, y_3) , (x_4, y_4) ,
- `set tclVarName { ( { xL1 yL1 xH1 yH1 } | { x1 y1 x2 y2 x3 y3 x4 y4 ... } ) ... }` : Indicates setting an area that is defined by the union of multiple rectangles and/or rectilinear polygons.

The following figure presents the special area definition for the REGION statement:



Following is the AREA Rectangle/Polygon statement in a PSDL file:

```
{ AREA areaSetId
  [ { EXPAND ( allSide | leftRightSide topBottomSide | left bottom right top
               | { LEFT left } { BOTTOM bottom } { RIGHT right } { TOP top } ) } ]
  ( { xL1 yL1 xH1 yH1 } | { x1 y1 x2 y2 x3 y3 x4 y4 ... } ) ... )
}
```

Following is the AREA CELL/BLOCK/INSTANCE statement in a PSDL file:

```
{ AREA areaSetId
  [ { EXPAND ( allSide | leftRightSide topBottomSide | left bottom right top
               | { LEFT left } { BOTTOM bottom } { RIGHT right } { TOP top } ) } ]
  ( { CELL cellMasterName ... }
    | { BLOCK ( * | blockName | blockinstanceName ... ) }
    | { INSTANCE instanceName ... }
  )
}
```

Following is the AREA POWERDOMAIN statement in a PSDL file:

```
{ AREA    areaSetId
    [ { EXPAND (    allSide    | leftRightSide topBottomSide    | left bottom right top
                | { LEFT    left } { BOTTOM    bottom } { RIGHT    right } { TOP    top } ) } ]
    { POWERDOMAIN ( * |    powerDomainName    ... ) }
}
```

Related Topics

- [route_pg](#)
- [Components of a PSDL File](#)
- [PG Structure Description Language](#)
- [PSDL Examples](#)

EXCEPTAREA Statement

Following is the EXCEPTAREA statement in a PSDL file:

```
{ EXCEPTAREA    exceptAreaId
    ( { AREA    areaSetId    } | { AREA    statement    }
      | { CELL    cellMasterName    ... }
      | { BLOCK ( * |    blockName    ... ) }
      | { INSTANCE    instanceName    ... }
      | { POWERDOMAIN ( * |    powerDomainName    ... ) } )
}
```

Following are the exclusion area definition options used in the EXCEPTAREA statement:

- `{ AREA areaSetId }`: Uses the predefined area specified by the `areaSetId` option in the AREA statement.
- `{ AREA statement }`: Defines a new AREA statement and uses it for the EXCEPTAREA statement.
- `{ CELL cellMasterName ... }`: Excludes all the instance area whose cell master matches with `cellMasterName` in the list.
- `{ BLOCK ( * | blockName ... ) }`: Excludes all the BLOCK MACRO type instance that matches with `blockName` in the list, or with all BLOCK MACROs, if * is specified.
- `{ INSTANCE instanceName ... }`: Excludes all the instance area that matches with `instanceName` in the list.

- { POWERDOMAIN (* | powerDomainName ...) } : Excludes all the power domain region that matches with powerDomainName in the list, or with all non-default power domains, if * is specified.

Following is the EXCEPTAREA statement with named area definition:

```
{ EXCEPTAREA  exceptAreaId
  { AREA  areaSetId
    [ { EXPAND (  allSide  | leftRightSide topBottomSide  | left bottom right top
      | { LEFT  left  } { BOTTOM  bottom  } { RIGHT  right  } { TOP  top  } ) } ]
    ( ( {  xL1 yL1 xH1 yH1  } | {  x1 y1 x2 y2 x3 y3 x4 y4 ...  } | { CELL
cellMasterName  ...  }
      | { BLOCK ( * |  blockName  ...  ) }
      | { INSTANCE  instanceName  ...  }
      | { POWERDOMAIN ( * |  powerDomainName  ...  ) } ) ...  )
    }
  }
}
```

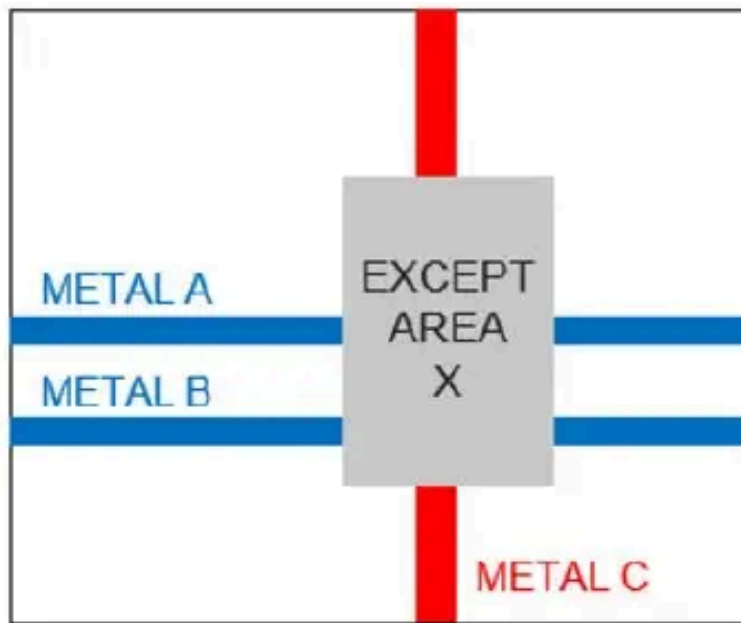
Scope of EXCEPTAREA Statement

You can apply EXCEPTAREA in the REGION, LAYER, or METAL statements. The validity scope of the EXCEPTAREA depends on the statement where it is applied. The following examples show the different behaviors of EXCEPTAREA based on where it is defined:

- The following example presents keeping EXCEPTAREA X above all the metal stripes:

```
{ REGION
{ EXCEPTAREA X }
  { LAYER M1 { METAL A }
    { METAL B } }
  { LAYER M2 { METAL C } }
}
```

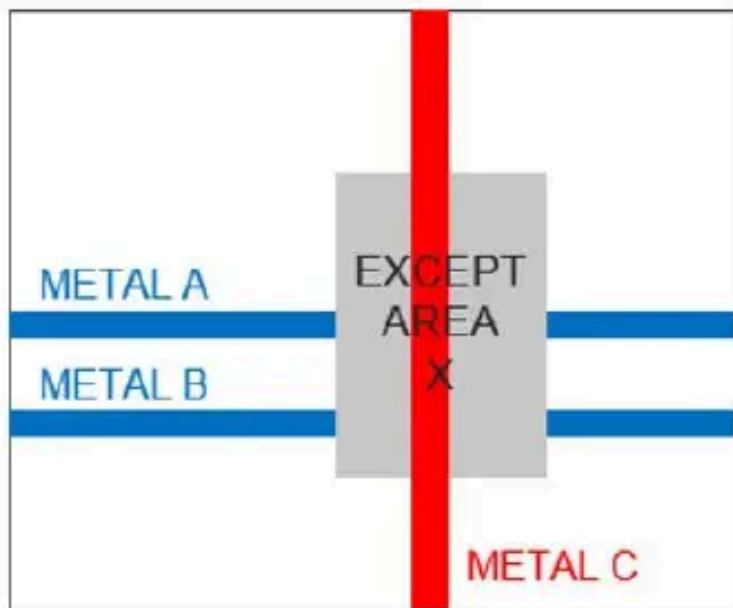
Following is the output region of the above example:



- The following example presents keeping EXCEPTAREA X above the METAL B metal stripe and below the METAL A and METAL C metal stripes:

```
{ REGION
{ LAYER M1 { EXCEPTAREA X }
  { METAL A }
  { METAL B } }
{ LAYER M2 { METAL C } }
}
```

Following is the output region of the above example:



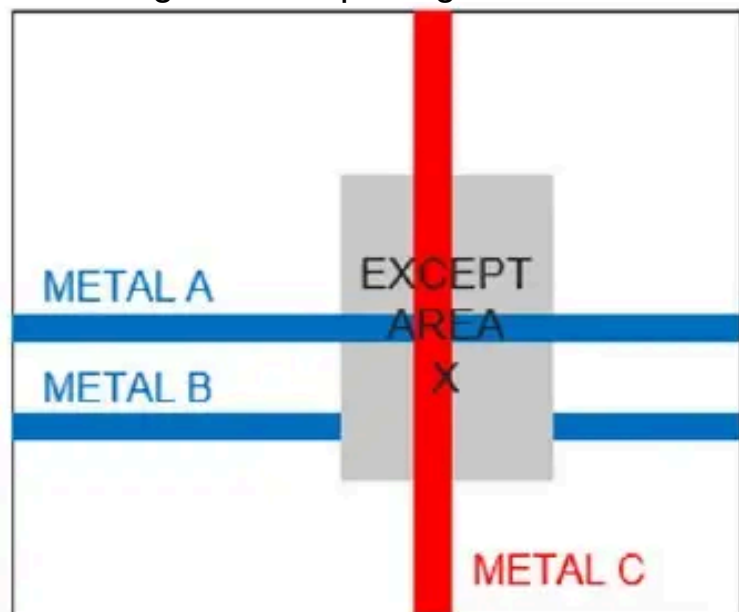
- The following example presents keeping EXCEPTAREA X above the METAL A and METAL B metal stripes and below the METAL C metal stripe:

```
{ REGION
```



```
{ LAYER M1 { METAL A }  
    { METAL B { EXCEPTAREA X } }  
}  
{ LAYER M2 { METAL C } }
```

Following is the output region of the above example:



Related Topics

- [route_pg](#)
- [Components of a PSDL File](#)
- [PG Structure Description Language](#)
- [PSDL Examples](#)

REGION Statement

- [RING Statement](#)
- [LAYER Statement](#)
 - [METAL Statement](#)
 - [FOLLOW and ALIGNMENT Statements](#)
 - [VIA Statement](#)

- [UDA Statement](#)
 - [Related Topics](#)

Following is the REGION statement in a PSDL file:

```
{ REGION
    [ ( { AREA    areaSetId    } | { AREA    statement  } | { CELL    cellMasterName    ... }
      | { BLOCK ( * |    blockName | blockinstanceName    ... ) } | { INSTANCE
instanceName    ... }
      | { POWERDOMAIN ( * |    powerDomainName ...    ) }
      | { DESIGNAREA | COREAREA | PADAREA } ) ]
    [ { EXCEPTAREA    statement    } ]
    [ { UDA    udaName    } ]
    [ { RING    statements    } ... ]
    [ { LAYER    statements    } ... ]
}
```

Following are the area definition options used in the REGION statement:

- { AREA areaSetId } : Uses the predefined area specified as areaSetId from the AREA statement.
- { AREA statement } : Defines a new AREA statement and uses it for the REGION area.
- { CELL cellMasterName ... } : Specifies the REGION instance area consisting of all the instance area regions, which has its cell master, which matches with cellMasterName .
- { BLOCK (* | blockName | blockinstanceName ...) } : Specifies the REGION area consisting of all the BLOCK MACRO type instances, which matches with blockName or blockinstanceName in the list, or all BLOCK MACROS, if * is specified.
- { INSTANCE instanceName ... } : Specifies the REGION area consisting of all the instance area regions, which matches with instanceName in the list.
- { POWERDOMAIN (* | powerDomainName ...) } : Specifies the REGION area consisting of all the power domain regions, which matches with the powerDomainName in the list, or all non-default power domains, if * is specified.

RING Statement

Following is the RING statement in the REGION statement:

```
{ RING ringName
    { INNER | OUTER } [ { UDA    udaName    } ]
```

```

{ NET [ USEGROUP ]    net1 net2 ... netn }

{ EXTENDWIRE ( AUTO | NONE ) }

{ LAYER ( { LEFT    layerLeft    } { BOTTOM  layerBottom  } { RIGHT  layerRight  } { TOP
layerTop  } | layerAll  ) }

{ WIDTH (  allSideWidth  | leftRightWidth topBottomWidth      | left bottom right top
          | { LEFT  left  } { BOTTOM  bottom  } { RIGHT  right  } { TOP  top  } ) }

{ OFFSET (  allSideOffset | leftRightOffset topBottomOffset    | left bottom right top
          | { LEFT  left  } { BOTTOM  bottom  } { RIGHT  right  } { TOP  top  } ) }

{ SPACING (  allSideSpacing | leftRightSpacing topBottomSpacing | left bottom right
top
          | { LEFT  left  } { BOTTOM  bottom  } { RIGHT  right  } { TOP  top  } ) }

}

```

 EXTENDWIRE NONE does not extend any PG wires to the PG rings, unless specified in the METAL statement.

See the examples below:

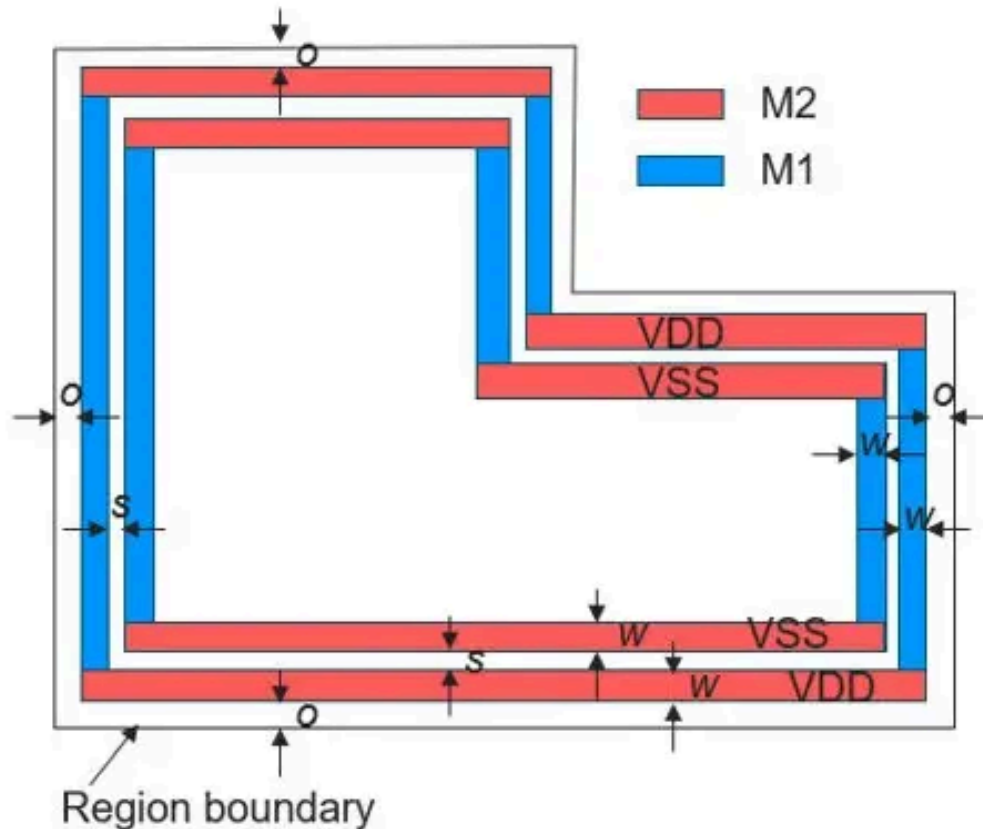
- Following is an example of the INNER RING usage:

```

{ RING { INNER } { LAYER M2 M2 M1 M1 }
{ NET VDD VSS } { WIDTH      w } { OFFSET  o } { SPACING  s } }

```

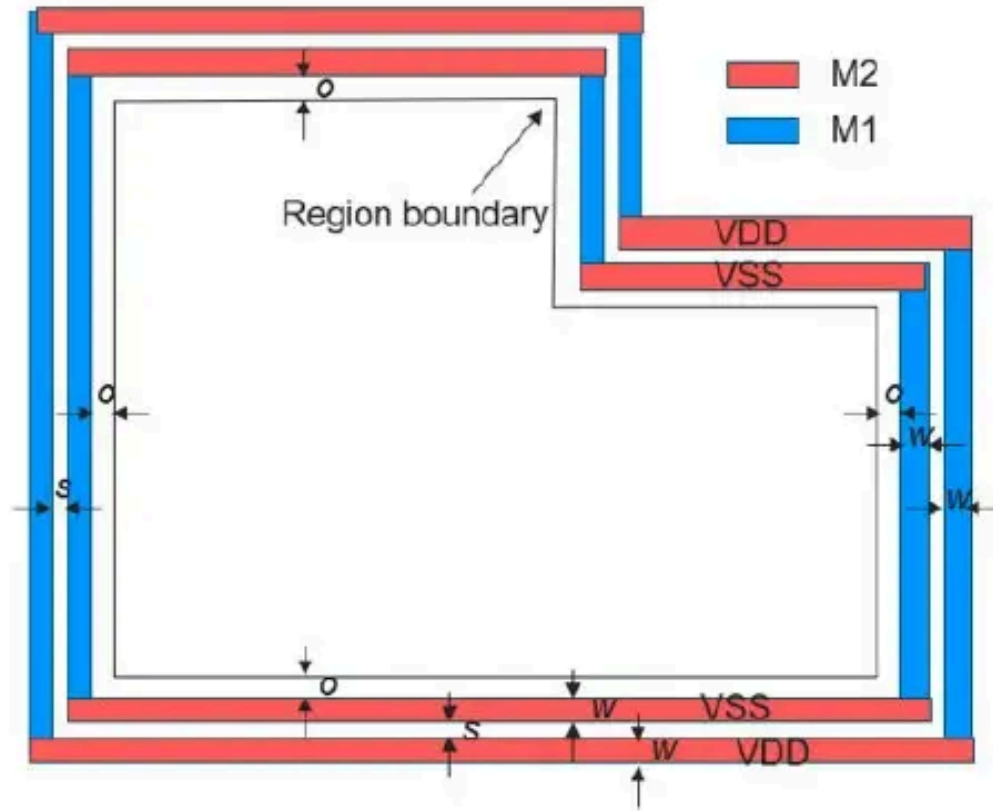
Following is the output of the above example:



- Following is an example of the OUTER RING usage:

```
{ RING { OUTER }
{ LAYER M2 M2 M1 M1 }
  { NET VDD VSS } { WIDTH      w }
  { OFFSET  o } { SPACING   s } }
```

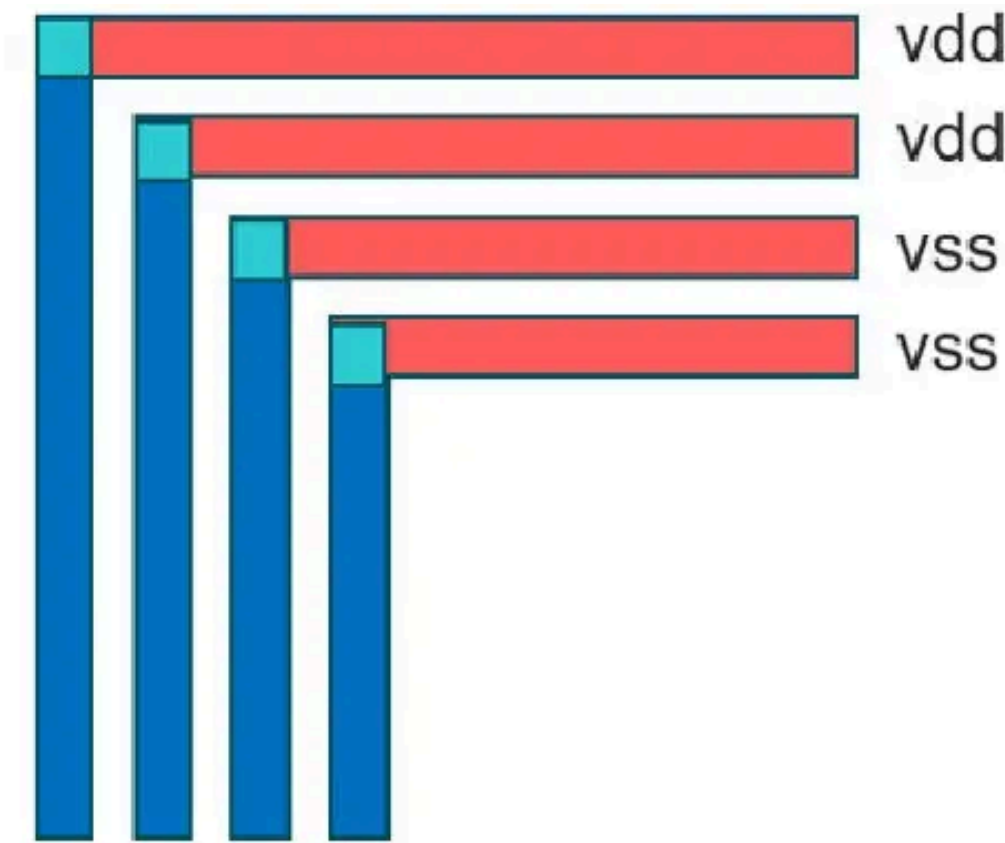
Following is the output of the above example:



- Following are the examples of controlling RING NET group at the corners:

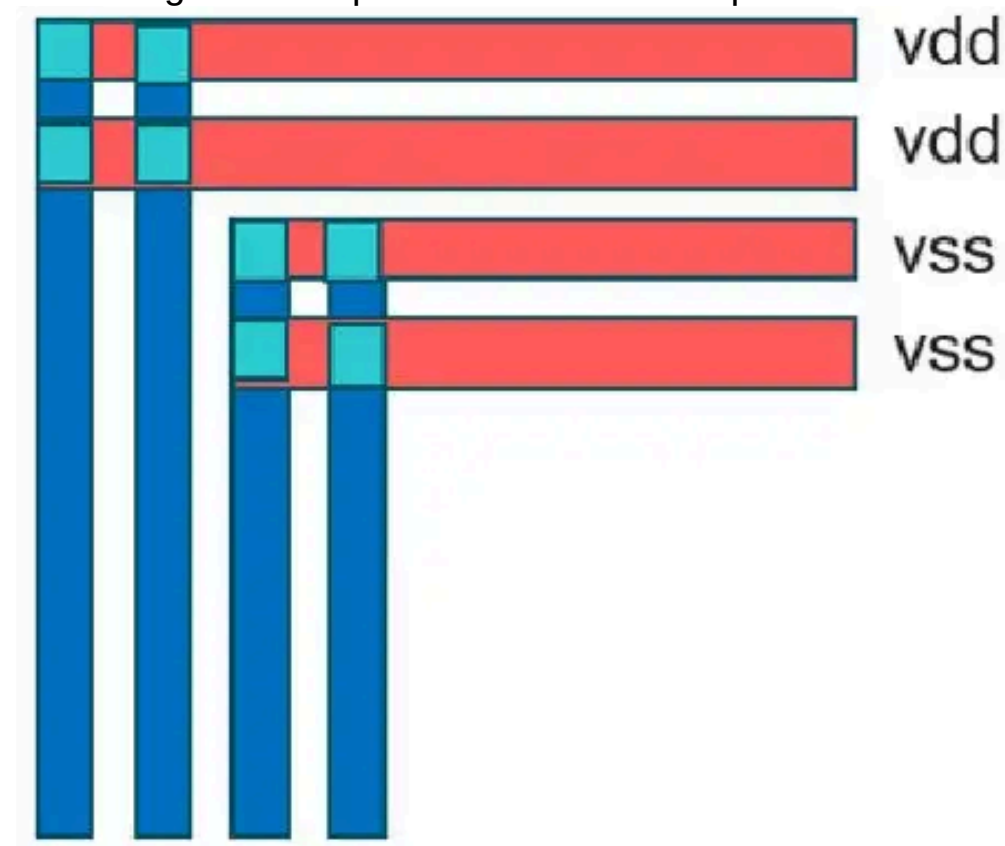
```
◦ { RING ...
  { NET vdd vdd vss vss }
}
```

Following is the output of the above example:



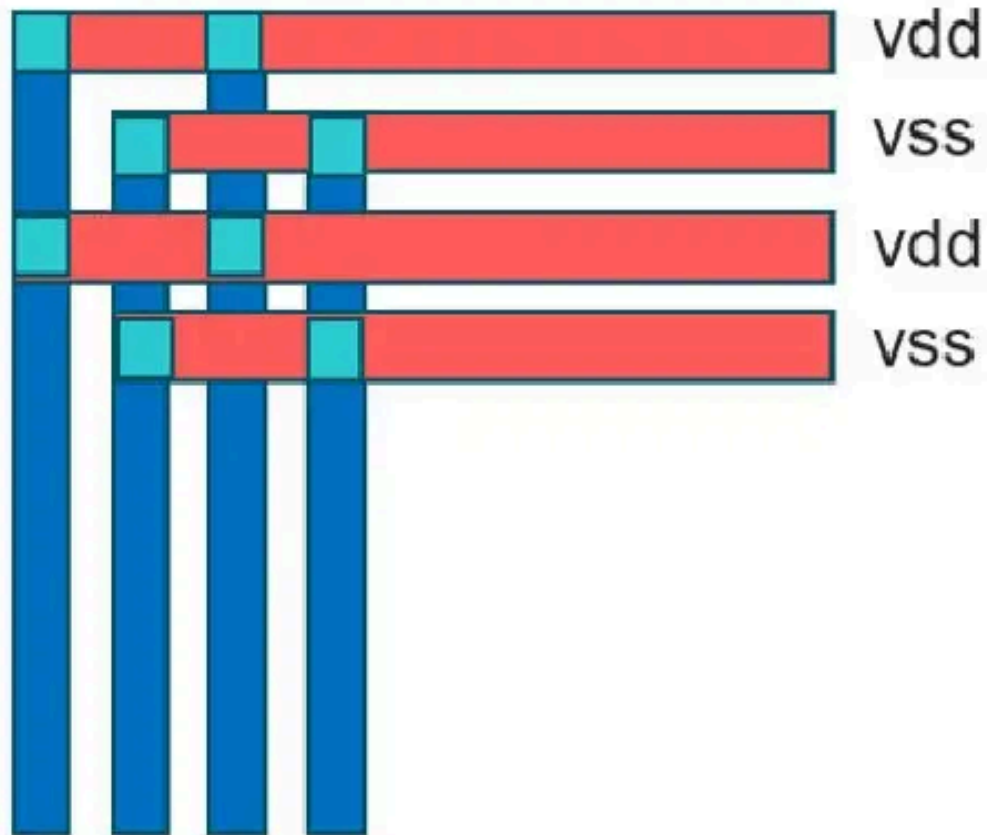
- { RING ...
 { NET USEGROUP vdd vdd vss vss }
}

Following is the output of the above example:



- { RING ...
{ NET USEGROUP vdd vss vdd vss }
}

Following is the output of the above example:



LAYER Statement

Following is the LAYER statement in the REGION statement:

```
{ LAYER  layerName
    [ { ONGRIDONLY } | { SNAPT0      statement  } ]
    [ { UDA    udaName  } ]
    { METAL    statements  } ...
    [ { EXCEPTAREA  statement  } ]
}
```

METAL Statement

Metal specifies the location within the region where the patterns will be used to develop the specified design. It contains the offset and distance between the various patterns within the defined region. Following is the METAL statement in the LAYER statement:

```
{ METAL    metalPatternId    patternName
```

```
{ NET net1 net2 ... netn } [ { UDA udaName } ]
[ { EXTENDTO ringName1 ringName2 ... } ]
[ { SIZE ( * | xnum ) ( * | ynum ) } ] [ { SNAPT0 statement } ] ]
[ { OFFSET x0 y0 [ FROM ( REGION | ZERO | CORE | DESIGN | CELL | REGIONBBOX ) ] [ TO (
CENTER | LOWEDGE ) ] } ]
[ { STEPDISTANCE xdist ydist [ BEFORESNAPPING ] [ { LOWEDGETOLOWEDGE |
CENTERTOCENTER | EDGETOEDGE } ] } ]
[ { FOLLOW syntax } ]
[ { ALIGNMENT syntax } ]
[ { EXCEPTAREA statement } ]
}
```

Following are the attributes used in the METAL statement:

- { NET net₁ net₂ ... net_n } : Specifies the net attached to the corresponding width in the PATTERN statement. The net set in the METAL statement is a secondary set, where the width set in the corresponding PATTERN statement is the primary set. This means that the number of nets in the set follows the number of width sets in the pattern.
- { UDA statement } : Specifies the user-defined attributes for all the PG wire objects created by the associated METAL statement. The UDA statement overrides both the LAYER UDA and REGION UDA attributes.
- { EXTENDTO ringName₁ ringName₂ ... } : Extends the PG wires to PG rings.
- { SIZE (* | x_{num}) (* | y_{num}) } : Specifies the number of times the PG wires set are replicated in the specified REGION. By default (with default value = *), the PG wires' set is replicated until it reaches the end of the REGION.
- { SNAPT0 statement } : Specifies the snapping behavior of all the PG wires generated by the METAL statement. This overrides the LAYER snapping attribute.
- { OFFSET x₀ y₀ [FROM (REGION | CORE | DESIGN | ZERO | CELL | REGIONBBOX)] [TO (CENTER | LOWEDGE)] } : Specifies the absolute starting offset for the PG wires' set. The offset value is calculated based on one or more of the following coordinates:
 - REGION's lower-left coordinate of union of multiple areas (default value)
 - Lower-left coordinate of CORE bounding box
 - Lower-left coordinate of DESIGN bounding box
 - Absolute (0,0) coordinate

- Lower-left coordinate of each CELL/instance/block
- REGIONBBOX specifies the union of bounding box
- OFFSET measurement from the lower-edge of the first metal in the pattern



- The default value of OFFSET is REGIONBBOX when no value is specified for OFFSET. This REGIONBBOX merges all the CELL, BOX, INSTANCE, or POWERDOMAIN area regions, and uses the lowest-left corner of the merged boxes as the initial OFFSET.
- The default value of OFFSET is TO LOWEDGE in all other cases. When you specify TO CENTER, the OFFSET measurement is done from the lowest-left edge of the region to the center of the pattern bounding box.
- You can measure the OFFSET from each lowest-left corner of REGION boxes by specifying the FROM REGION and FROM CELL keywords.

- { STEPDISTANCE x_{dist} y_{dist} [BEFORESNAPPING] [(LOWEDGETOLOWEDGE | CENTERTOCENTER | EDGETOEDGE)] } : Specifies the set-to-set absolute distance between the set of two PG wires. The distance is calculated based on their low-edge, default center point, or edge-to-edge spacing measurement. By default, the distance is calculated based on the first wire from the previous set after when it is snapped to the grid. If BEFORESNAPPING is specified, then the distance is calculated based on the first wire from the previous set before it was snapped to the grid.
- { EXCEPTAREA statement } : Specifies the exclusion area in which all the PG wires generated by the METAL statement need to be avoided. This exclusion region is added on the top of the EXCEPTAREA statement in both the LAYER and REGION statements.
- { FOLLOW statement } : Specifies the METAL to be used as a reference for both the starting offset and set-to-set distance.
- { ALIGNMENT statement } : Specifies the alignment behavior between the PG wires' set from the current METAL and the set in the reference METAL.

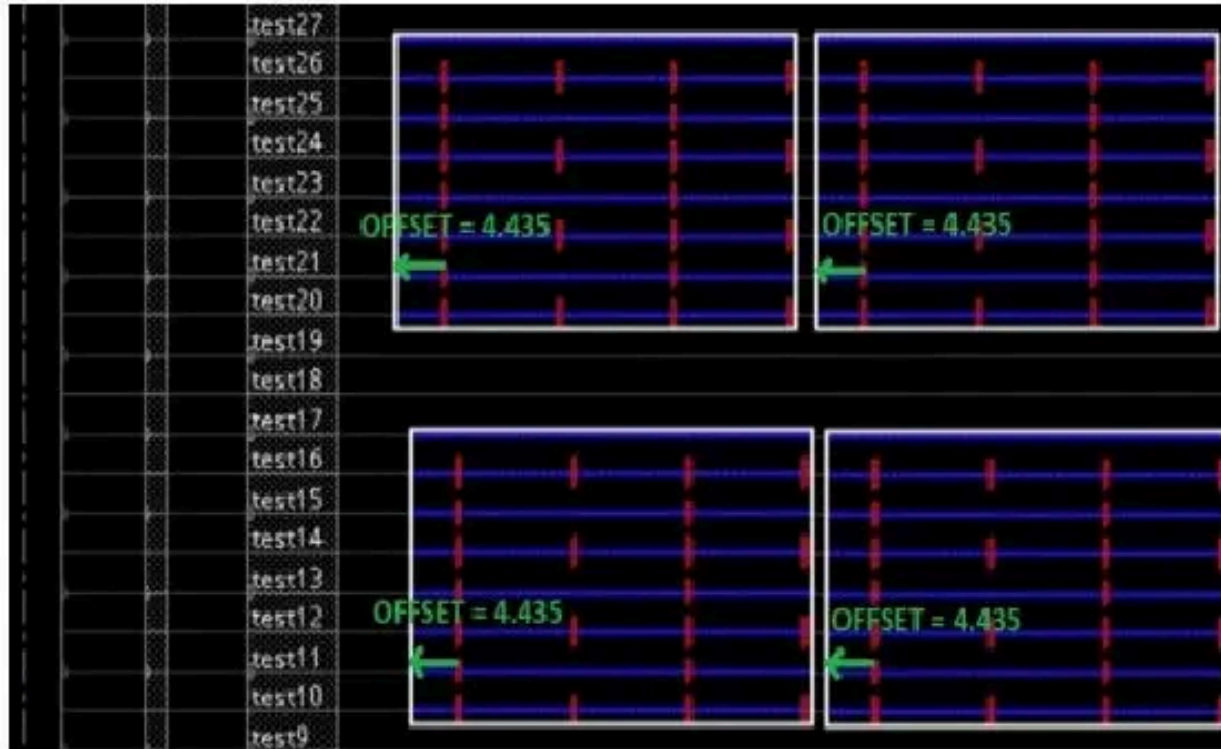
See the following examples:

- Following is an example of measuring OFFSET from each box in the REGION:

```
{ REGION
  { AREA m6_m17_area {28 60 58 82} {59 60 88 82} {29 30 59 52} {60 30 90 52} }
```

```
{ LAYER METAL1
  { METAL m1_fpin_vdd m1_followpin { NET {TCLVAR vdd_net} } }
  { METAL m1_fpin_vss m1_followpin { NET {TCLVAR gnd_net} } }
}
{ LAYER METAL2 { SNAPT0 GRID }
  { METAL m2_vdd m2_staple { NET VDD } { OFFSET 4.435 * FROM REGION } {
STEPDISTANCE 8.55 * } { FOLLOW * m1_fpin_vdd } { ALIGNMENT * CENTER } }
  { METAL m2_vss m2_staple { NET VSS } { OFFSET 4.435 * FROM REGION } {
STEPDISTANCE 17.10 * } { FOLLOW * m1_fpin_vss } { ALIGNMENT * CENTER } }
}
}
```

Following is the output of the above example:



- Following is an example of OFFSET measurement from the center of the pattern:

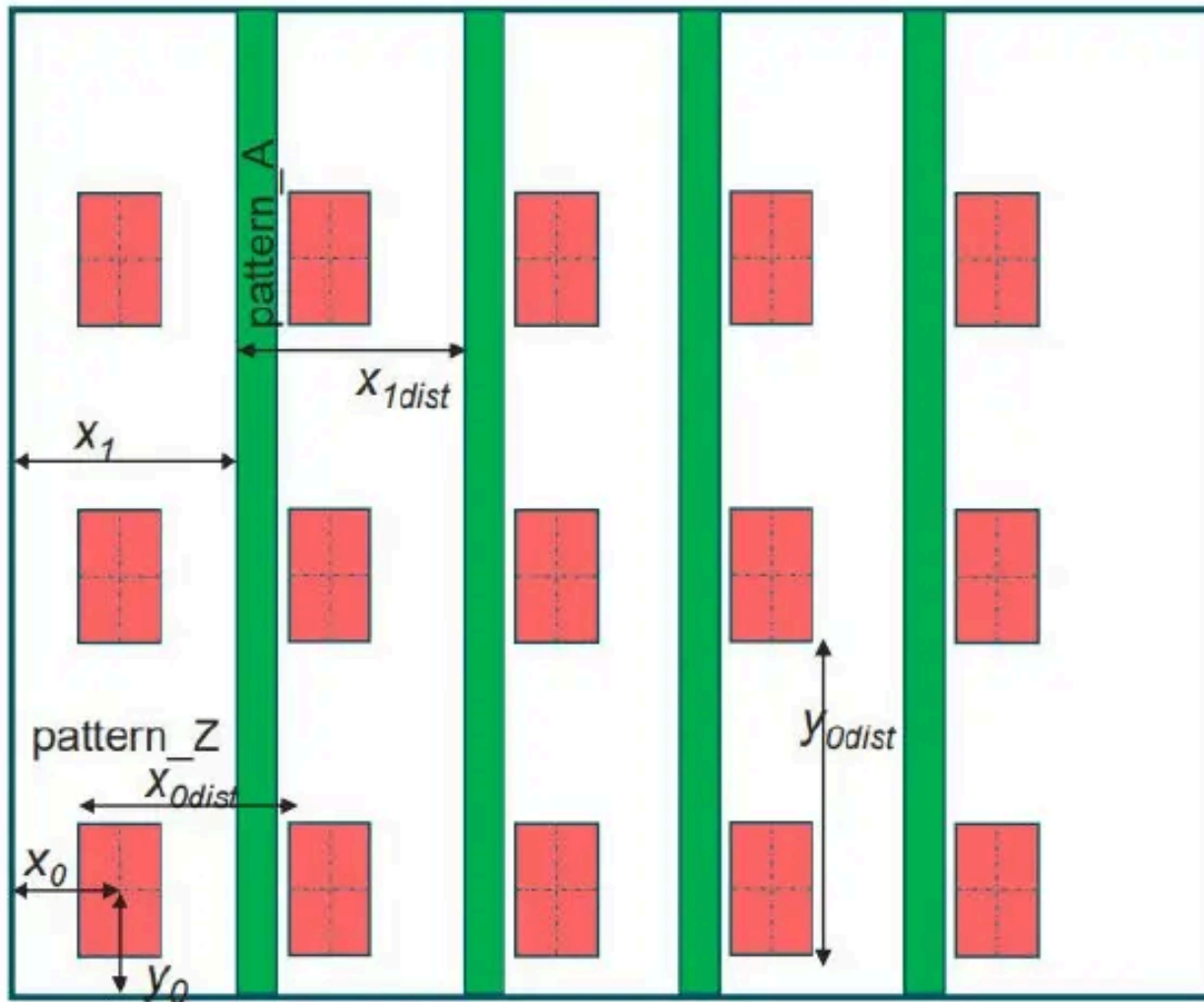
```
{ REGION
  { AREA { xL1 yL1 xH1 yH1 } }
  { LAYER M3
    { METAL example1 pattern_Z
      { NET vdd }
      { OFFSET x0 y0 TO CENTER}
      { STEPDISTANCE x0dist y0dist }
```

```

}
{ METAL example2 pattern_A
  { NET vdd }
  { OFFSET x1 * }
  { STEPDISTANCE x1dist * }
}
}
}

```

Following is the output of the above example:



- Following is an example of using one METAL statement in the REGION statement:

```

{ REGION
{ AREA { x_L1 y_L1 x_H1 y_H1 } }
  { LAYER M3
    { METAL example pattern_Z
      { NET vdd }
      { OFFSET x_0 y_0 }
    }
  }
}

```



```

    { STEPDISTANCE     $x_{dist}$   $y_{dist}$  }

    { SIZE     $x_{num}$   $y_{num}$  }

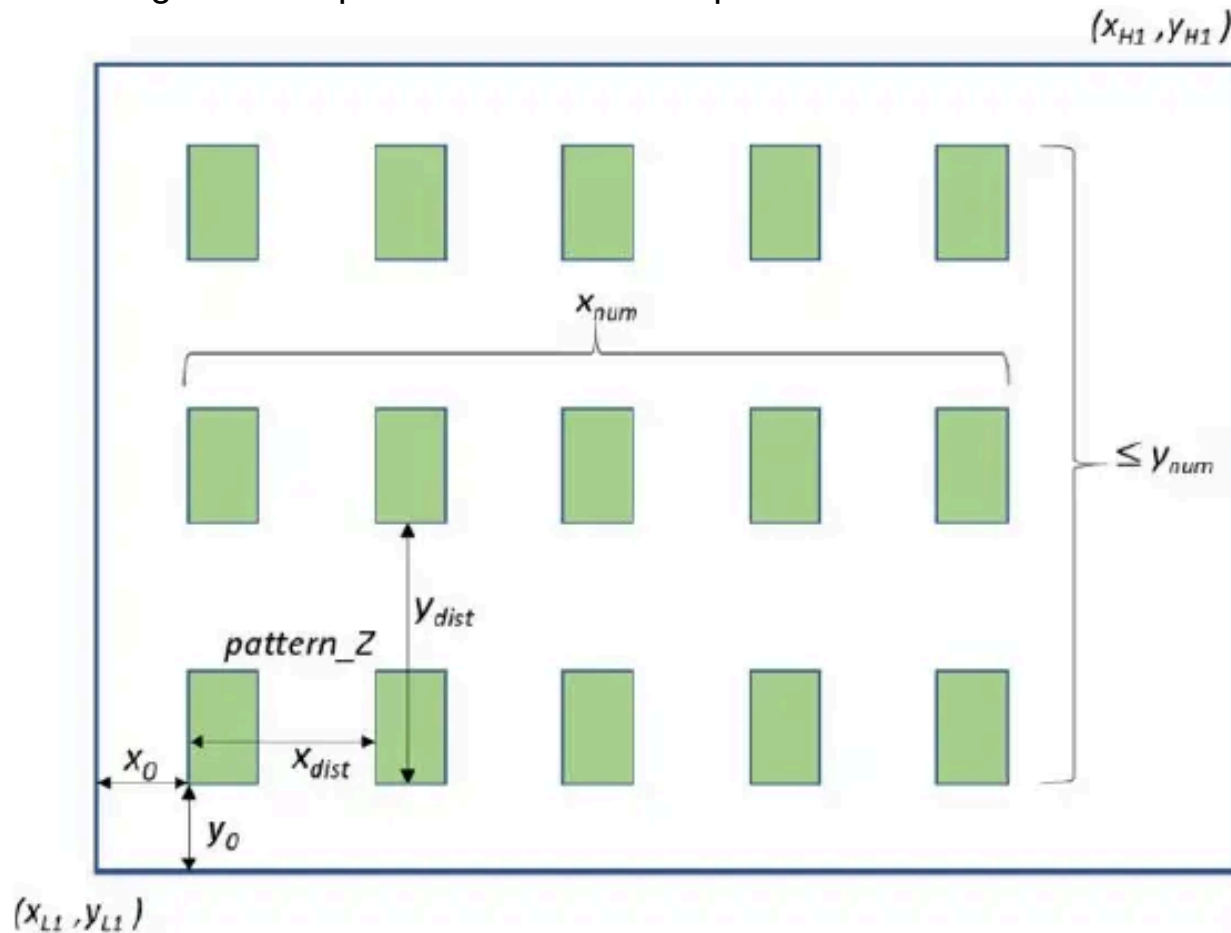
}

}

}

```

Following is the output of the above example:



- Following is an example of using two METAL statements in the REGION statement:

```

{ REGION
{ AREA {    $x_{L1}$   $y_{L1}$   $x_{H1}$   $y_{H1}$  } }
{ LAYER   M3
{ METAL   example1   pattern_Z
{ NET    vdd }
{ OFFSET   $x_0$   $y_0$  }
{ STEPDISTANCE     $x_{0dist}$   $y_{0dist}$  }
}
{ METAL   example2   pattern_A
{ NET    vdd }

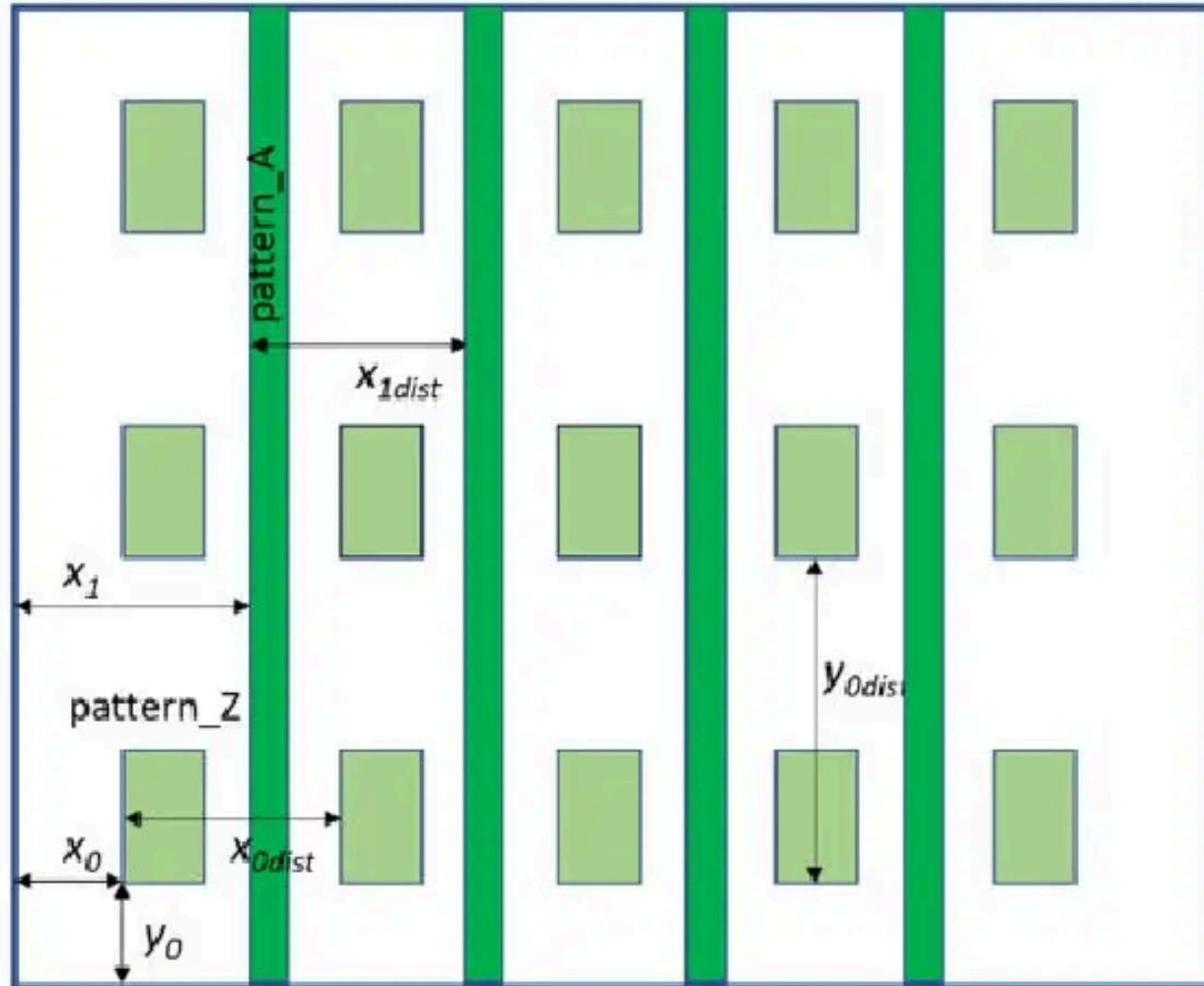
```

```

{ OFFSET    x1 * }
{ STEPDISTANCE    x1dist * }
}
}
}

```

Following is the output of the above example:



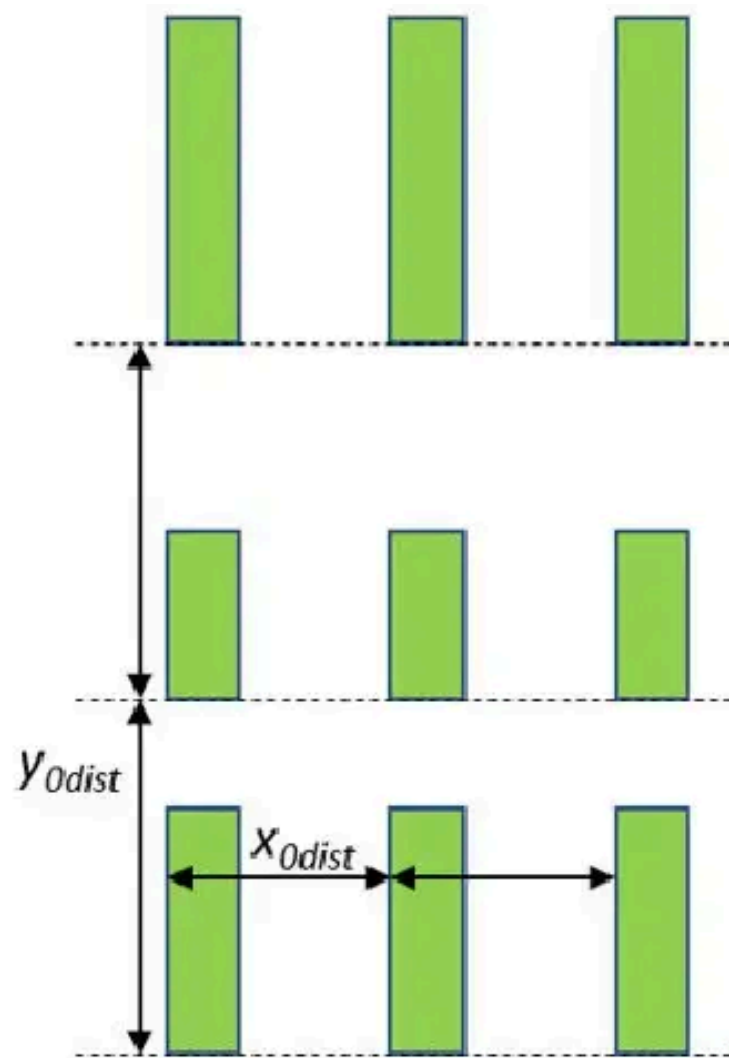
- Following is an example of the METAL statement with STEPDISTANCE value as LOWEDGETOLOWEDGE (default):

```

{ METAL "MxStaple"    pattern_Z
{ OFFSET    x0 y0 }
{ STEPDISTANCE    x0dist y0dist
LOWEDGETOLOWEDGE }
}

```

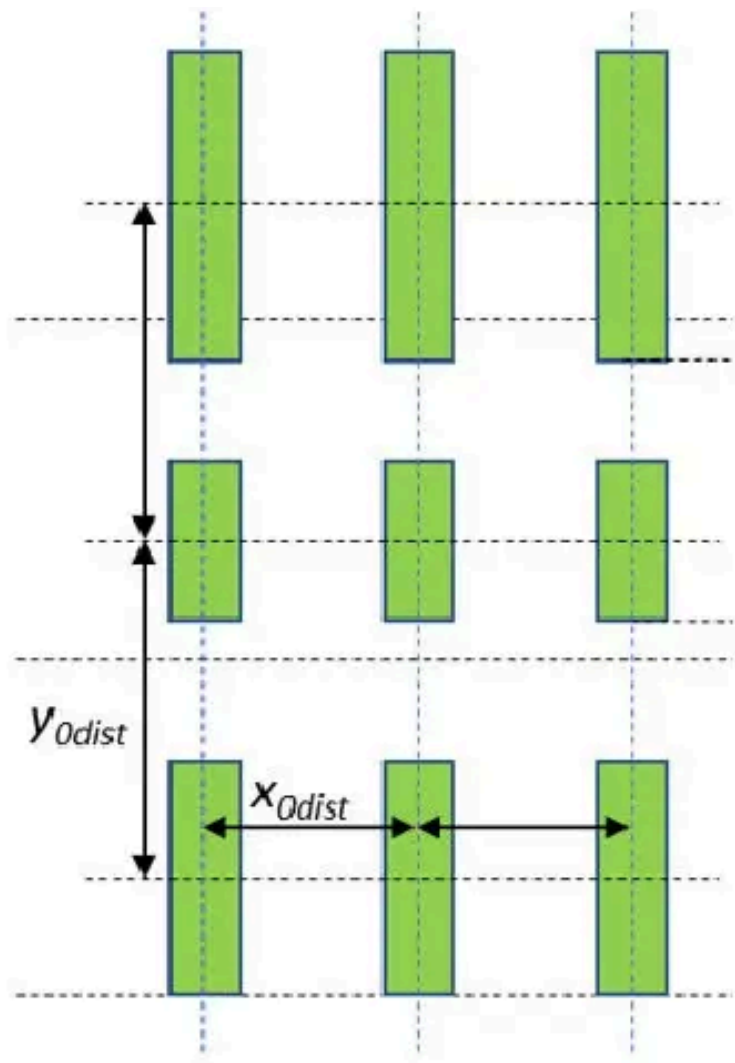
Following is the output of the above example:



- Following is an example of the METAL statement with STEPDISTANCE value as CENTERTOCENTER:

```
{ METAL "MxStaple"    pattern_Z
{ OFFSET  x_0 y_0 }
  { STEPDISTANCE  x_0dist y_0dist
    CENTERTOCENTER }
}
```

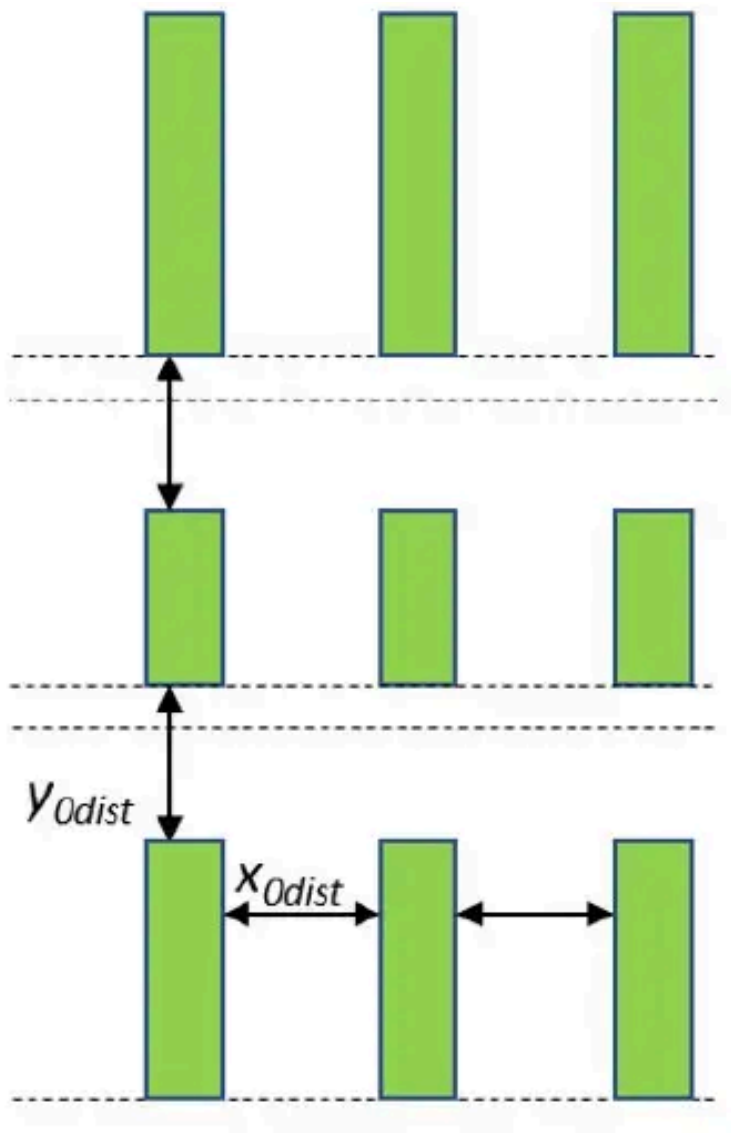
Following is the output of the above example:



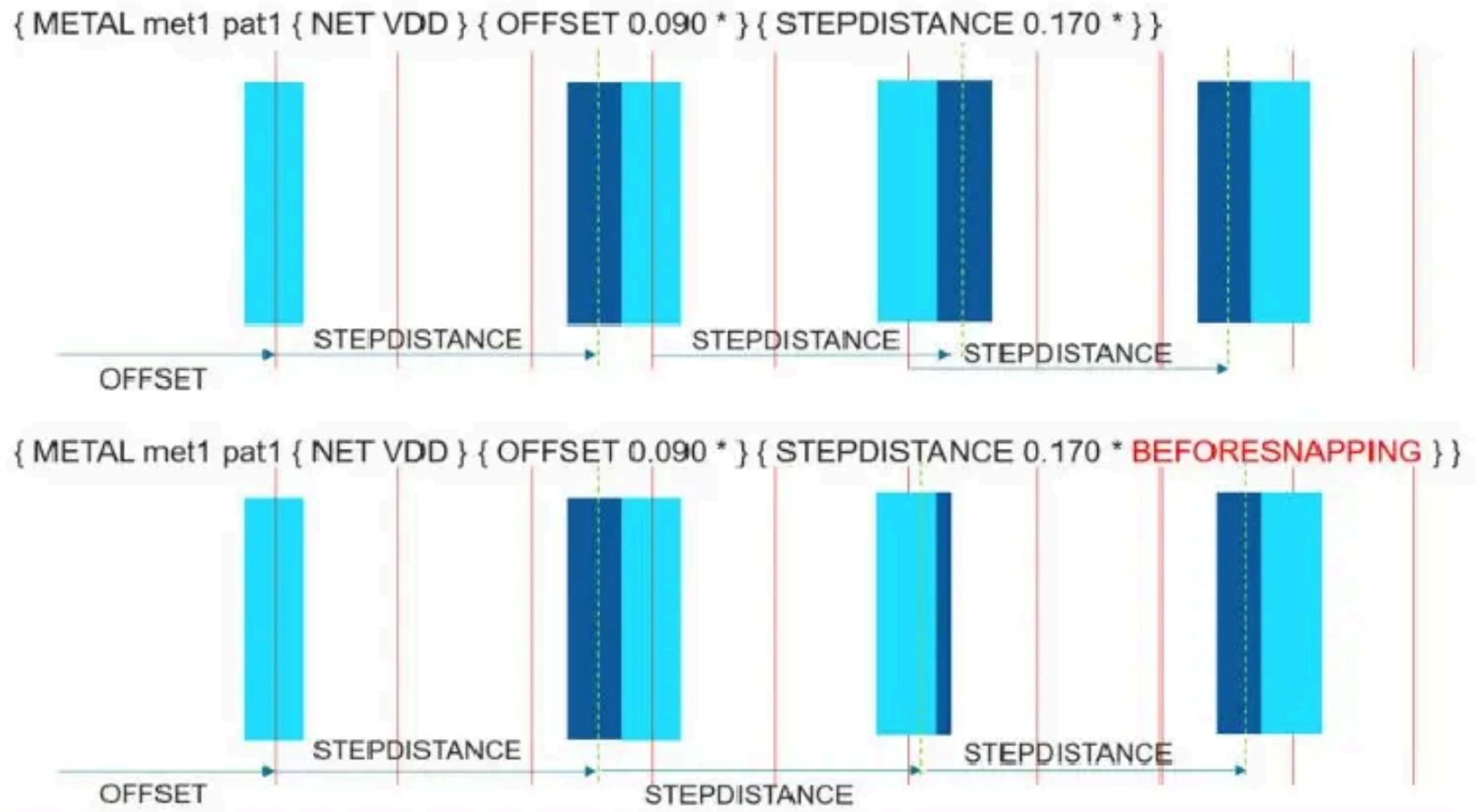
- Following is an example of the METAL statement with STEPDISTANCE value as EDGETOEDGE:

```
{ METAL "MxStaple"    pattern_Z
{ OFFSET  x_0 y_0 }
  { STEPDISTANCE  x_0dist y_0dist
    EDGETOEDGE }
}
```

Following is the output of the above example:



- Following are two examples of the METAL statement without and with using the BEFORESNAPPING option:



 Snapping is done towards the closest track. So, it can be done towards the left track (if it is closer than the right one). When the location is right from the middle of two tracks, snapping will be done to the left.

FOLLOW and ALIGNMENT Statements

Following is the syntax of the FOLLOW statement:

```
{FOLLOW ( * | xRefPatternId | USERGRID gridName ) ( * | yRefPatternId | USERGRID gridName ) }
```

See the examples below:

- Following is an example of using USERGRID as a reference grid in the FOLLOW statement:

```
{ USERGRID grid1 0.100 1.500 } #creates grid1: 0.100, 1.600, 3.100, 4.600, 6.100,
```

...

```
{ REGION { COREAREA }
```

```
  { LAYER METAL3 { SNAPT0 GRID } ### snapping all METAL3 to its' track
```

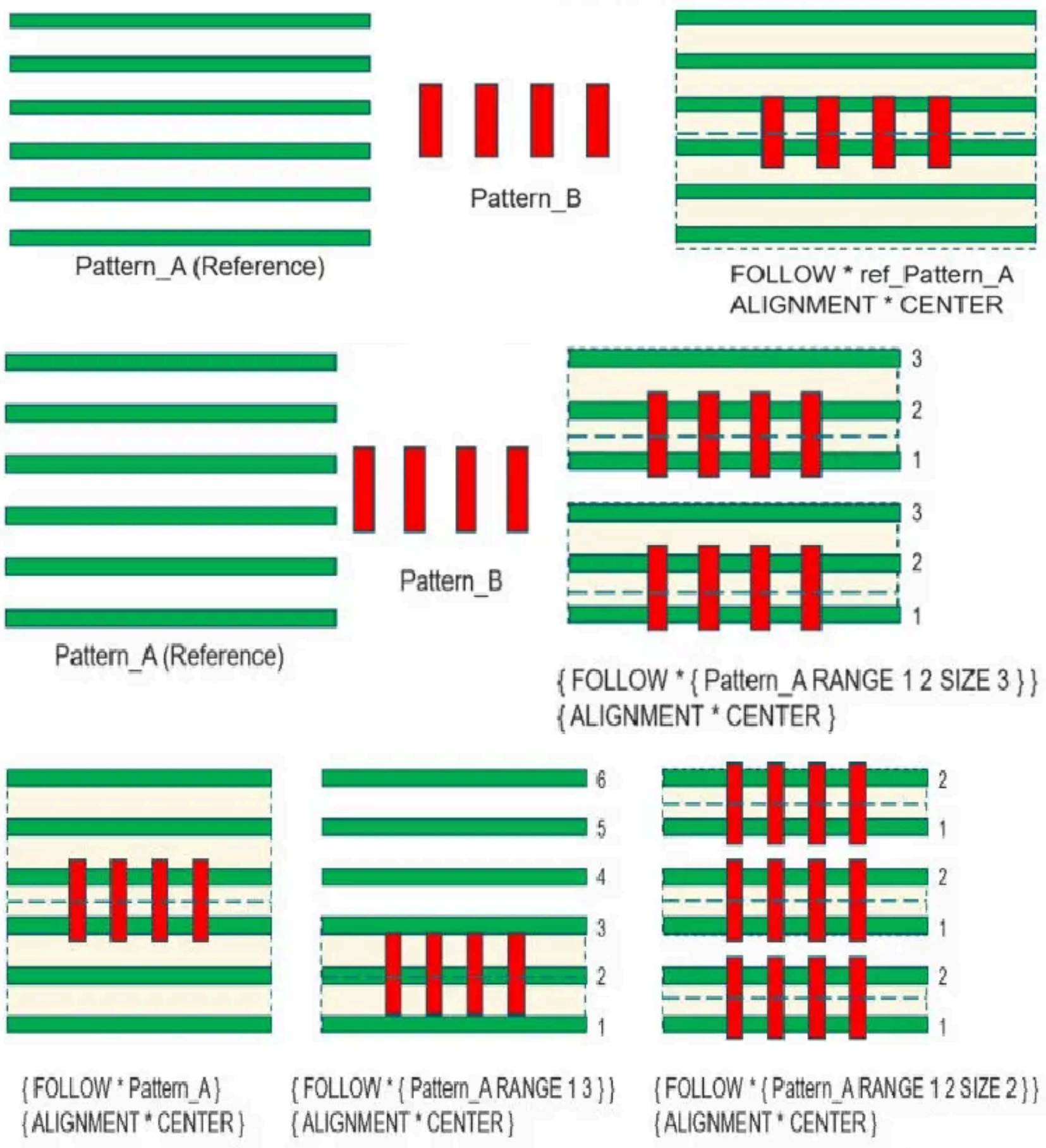
```
  { METAL m3_vdd m3_staple { NET VDD } { FOLLOW USERGRID grid1 m1_fpin_vdd} {
```

```
ALIGNMENT CENTER CENTER } }
```

```
}
```


}

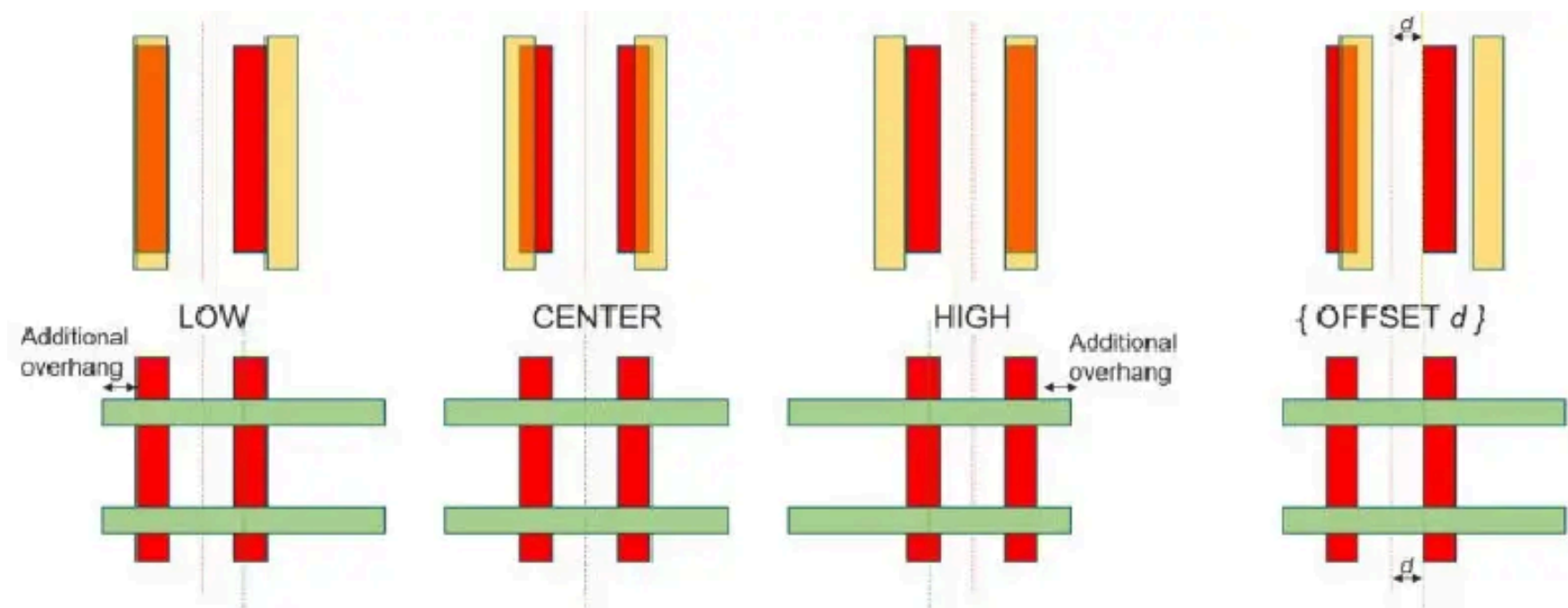
In the above example, with specifying the FOLLOW USERGRID, the *center* of the lowest metal will be placed on the left/bottom grid, when ALIGNMENT is set to LOW.



Following is the syntax of the ALIGNMENT statement:

```
{ ALIGNMENT ( * | LOW | CENTER | HIGH | { OFFSET      xAlignment  } )
              ( * | LOW | CENTER | HIGH | { OFFSET      yAlignment  } ) }
```

Following are the outputs of various ALIGNMENT values:

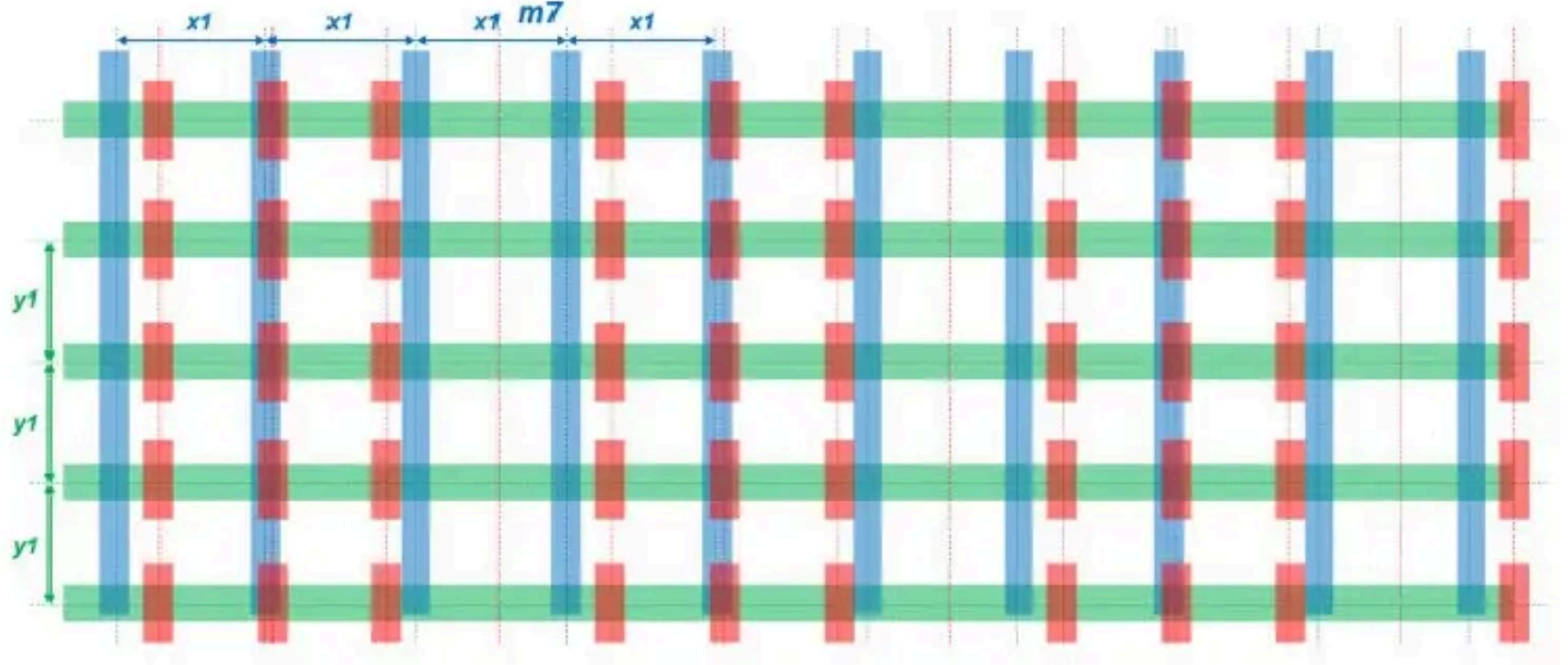


See the examples below:

- Following is an example of using the FOLLOW references for both X and Y axis for the STAPLE statement: { LAYER M7 { ONGRIDONLY }

```
{ METAL   m7  pM7 { NET VDD }
  { OFFSET  x0  * }
  { STEPDISTANCE  x1  * } } }
{ LAYER M6 { ONGRIDONLY }
  { METAL   m6  pM6 { NET VDD }
    { OFFSET *    y0 }
    { STEPDISTANCE *    y1 } } }
{ LAYER M5 { ONGRIDONLY }
  { METAL   m5  pM5 { NET VDD }
    { FOLLOW  m7  m6 }
    { ALIGNMENT CENTER CENTER } } }
```

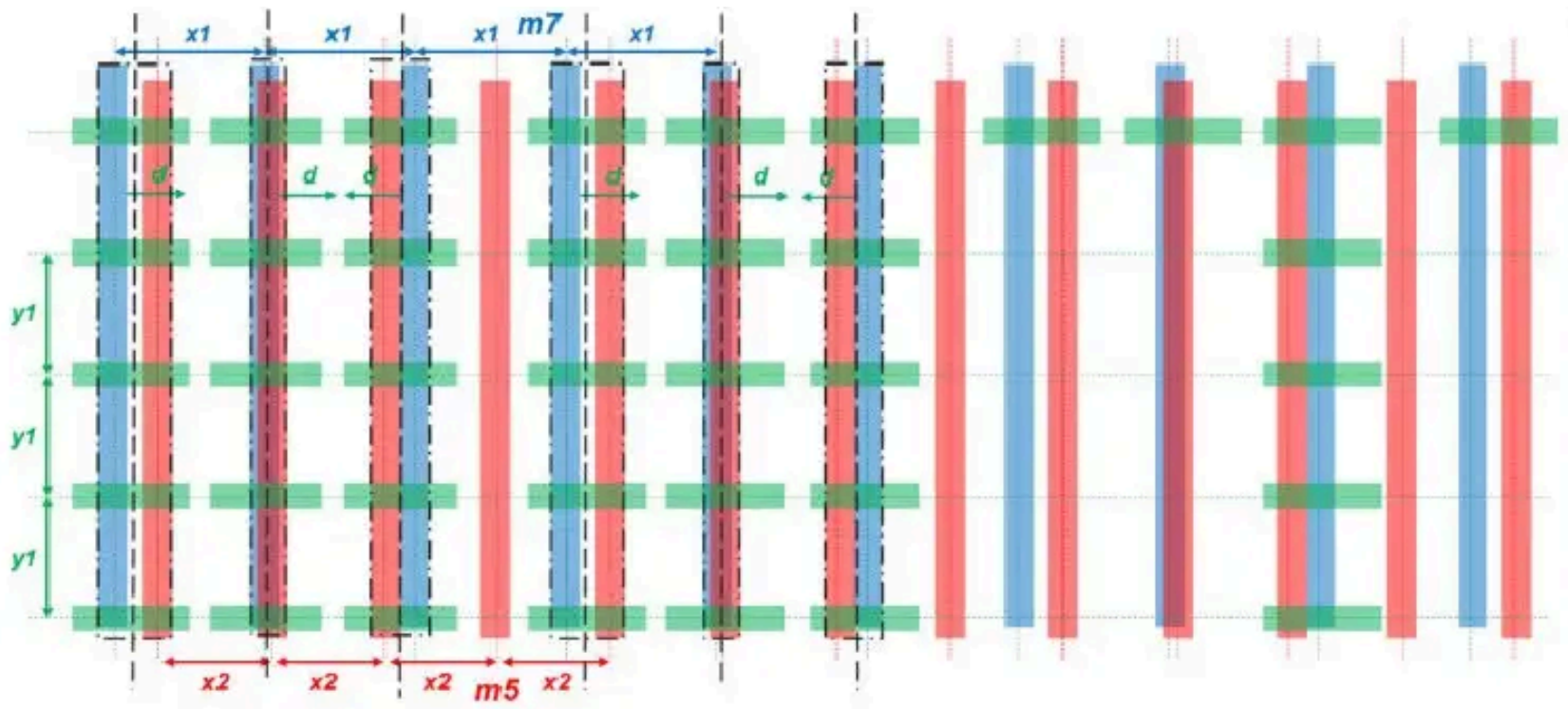
Following is the output of the above example:



- Following is an example of using the FOLLOW statement with two METAL references:

```
{ LAYER M7 { ONGRIDONLY }
  { METAL m7 pM7 { NET VDD }
    { OFFSET x0 * }
    { STEPDISTANCE x1 * } } }
{ LAYER M5 { ONGRIDONLY }
  { METAL m5 pM5 { NET VDD }
    { OFFSET xa * }
    { STEPDISTANCE x2 * } } }
{ LAYER M6 { ONGRIDONLY }
  { METAL m6 pM6 { NET VDD }
    { FOLLOW { m7 m5 d } * }
    { ALIGNMENT CENTER * }
    { STEPDISTANCE * y1 } } } # d denotes maximum distance measured
from first and second metal references
```

Following is the output of the above example:



VIA Statement

A VIA statement defines the preference order of the via(s) to be used for the connection between two METAL regions on the adjacent layers. The via type can be specified as VIARULE, VIACELL, or CUTCLASS, where VIARULE has the highest precedence, and CUTCLASS has the least.

The software chooses the first DRC clean option from the predefined rule via, predefined via cell, and generated via, first based on the specified via cut-class. If none of the vias are specified in the list and a DRC clean solution is provided, then a regular via is generated based on the surrounding metal that can bring maximum cut area.

Following is the use model of the VIA statement:

```
{ VIA    ( { { ( VIARULE | VIACELL [IGNOREDRC] | CUTCLASS )           x1 x2 ... xn } ... }
          [ OFFSET value STEPDISTANCE center_distance [ LENGTH length ] ]
  TO    ( ALL | ABOVE | BELOW | { metalName1 metalName2 ... metalNamen } ) ) ...
}
```

Here:

- IGNOREDRC allows you to add your preferred vias without running a DRC check. You can run a DRC check later to ensure a DRC clean design.
- The OFFSET, STEPDISTANCE, and LENGTH options are used only for two adjacent parallel wires.
- OFFSET is measured from the lowest-left edge of the region to the center of the pattern

bounding box.

- LENGTH is via metal enclosure.

See the examples below:

- Following is an example of using the VIA statement with adding a via preference without checking for DRCs:

```
{ REGION { AREA designbox } { EXCEPTAREA { BLOCK * } }
  { LAYER M1 { SNAPT0 GRID }
    { METAL m1_1 m1_staple1 { NET VDD} { OFFSET x1 y1} { STEPDISTANCE xs1 xy1}
  }
    { METAL m1_2 m1_staple2 { NET VDD} { OFFSET x2 y2} { STEPDISTANCE xs2 xy2}
  }
    { METAL m1_3 m1_staple3 { NET VSS } { OFFSET x3 y3} { STEPDISTANCE xs3
xy3} }
  }
  { LAYER M2 { SNAPT0 GRID } { VIA { VIACELL IGNOREDRC VIA12_HV_2 VIA12_HV_1 }
TO { m1_1 m1_2 } }
    { VIACELL VIA12_HV_3 } TO { m1_1 m1_2 } }
    { VIARULE VIA12_RULE_big } TO { m1_3 } }
    { METAL m2_vdd m2_staple { NET VDD } { FOLLOW m1_1 m0_vdd} { ALIGNMENT
CENTER CENTER } }
    { METAL m2_vdd2 m2_staple { NET VDD } { FOLLOW m1_2 m0_vdd} { ALIGNMENT
CENTER CENTER } }
    { METAL m2_vss m2_staple { NET VSS } { FOLLOW m1_3 m0_vss} { ALIGNMENT
CENTER CENTER } }
  }
}
```

In this example, the Flash PG engine will first attempt to use VIA12_HV_2 for connecting m1_1/m1_2 with the M2 layer. If this VIACELL will not work for the intersection area, the second attempt will be to use VIA12_HV_1 for connecting m1_1/m1_2 with the M2 layer. In case these VIACELLS will not work for the intersection area, a third attempt will be to use VIA12_HV_3 for connecting m1_1/m1_2 with the M2 layer.

- Following is an example of using the VIA statement within multiple the LAYER statements:

```
{ REGION { AREA designbox }      { EXCEPTAREA { BLOCK * } }
```



```

{ LAYER M1 { SNAPT0 GRID }
    { METAL m1_1 m1_staple1 { NET VDD} { OFFSET x1 y1} { STEPDISTANCE xs1 xy1}
}

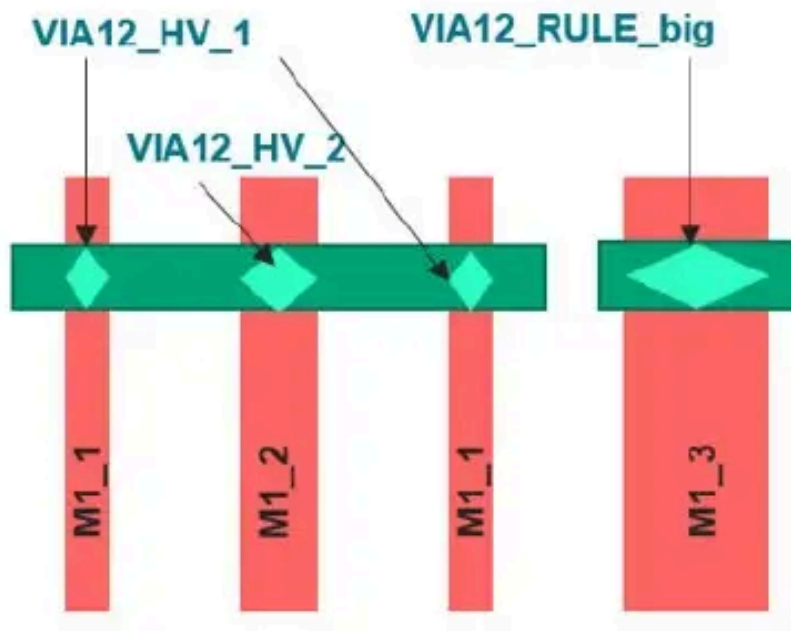
    { METAL m1_2 m1_staple2 { NET VDD} { OFFSET x2 y2} { STEPDISTANCE xs2 xy2}
}

    { METAL m1_3 m1_staple3 { NET VSS } { OFFSET x3 y3} { STEPDISTANCE xs3
xy3} }
}

{ LAYER M2 { SNAPT0 GRID }
    { VIA { VIACELL VIA12_HV_2 VIA12_HV_1 } TO { m1_1 m1_2 } } {
VIARULE VIA12_RULE_big } TO { m1_3 } }
    { METAL m2_vdd m2_staple { NET VDD } { FOLLOW m1_1 m0_vdd} { ALIGNMENT CENTER
CENTER } }
    { METAL m2_vdd2 m2_staple { NET VDD } { FOLLOW m1_2 m0_vdd} { ALIGNMENT
CENTER CENTER } }
    { METAL m2_vss m2_staple { NET VSS } { FOLLOW m1_3 m0_vss} { ALIGNMENT
CENTER CENTER } }
}

```

Following is the output of the above example:



- Following is an example of using the VIA statement within a single LAYER statement:

```

{ REGION { CORRECT }
    { LAYER METAL1 ...}
    { LAYER METAL2 ... }
}

```

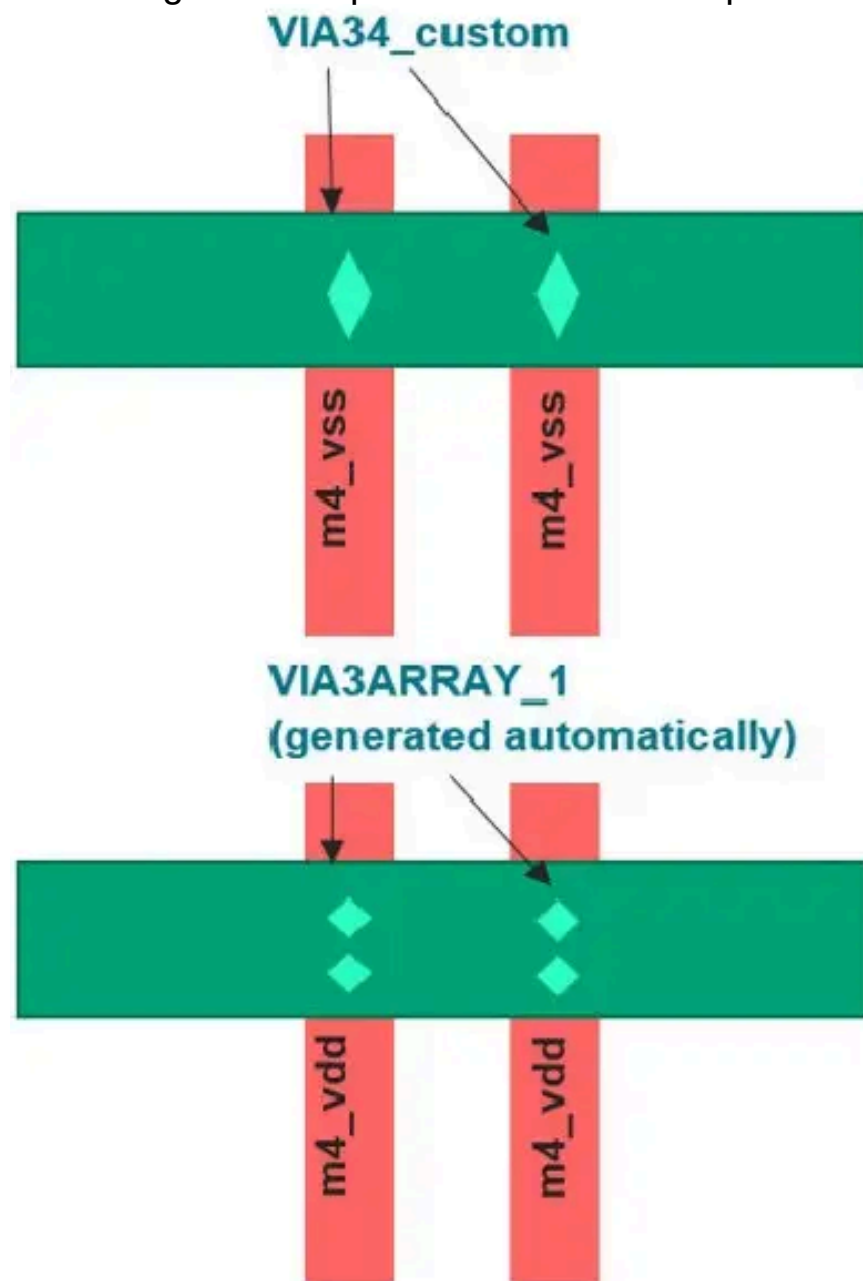


```

{ LAYER METAL3 ... }
{ LAYER METAL4 { SNAPT0 GRID }
  { VIA { VIACELL VIA34_custom } TO { m3_vss } }
  { METAL m4_vdd m4_staple { NET VDD } { FOLLOW m2_vdd m3_vdd} { ALIGNMENT
CENTER CENTER } }
  { METAL m4_vss m4_staple { NET VSS } { FOLLOW m2_vss m3_vss} { ALIGNMENT
CENTER CENTER } }
}
}

```

Following is the output of the above example:



UDA Statement

Use the UDA (User Defined Attribute) statement to attach a string identifier/name for the PG wires and vias. This attribute is useful for performing group query at Tcl level, once the complete PG mesh is generated. Following is the definition of UDA statement:

```
{ UDA  udaName }
```

You can add one UDA statement to either REGION, RING, LAYER, or METAL statements. The scope of this attribute assignment depends on the statement, where the UDA statement is added. PG vias always take the UDA value of the bottom PG wires, where they are connected to. `udaName` must follow the C-style variable naming rule (it means, it cannot start with a number).

You can use the UDA statement in a PSDL file in the following ways:

```
{ REGION { UDA  uda1 }
  { LAYER  M1 { UDA  uda2 }
    { METAL  met1 pat ... { UDA  uda3 } }
    { METAL  met2 pat ... } }
  { LAYER  M2 { METAL  met3 pat ... } } }
```

Here:

- All wires generated for `met1` will have `uda3` .
- All wires generated for `met2` will have `uda2` (based on the LAYER's UDA).
- All wires generated for `met3` will have `uda1` (based on the REGION's UDA).

Related Topics

- [route_pg](#)
- [Components of a PSDL File](#)
- [PG Structure Description Language](#)
- [PSDL Examples](#)

PSDL Examples

- [Processing of Design Flow Data into a PSDL File](#)
- [Resulting PG Mesh using PSDL](#)
- [Single Variable Change for PG Structure Exploration](#)

- [Related Topics](#)

Processing of Design Flow Data into a PSDL File

See the example below, where the following database design data is provided to the Flash PG flow to process a PSDL file:

Layer	Count	Width	Space	Length	Type
M0 (H)	-	-	-	-	followpin
M1 (V)	2	0.032	0.420	0.580	staple
M2 (H)	2	0.022	0.482	0.600	staple
M3 (V)	2	0.022	0.400	0.620	staple
M4 (H)	1	0.032	-	0.600	staple
M5 (V)	2	0.032	0.320	-	stripe
M6 (H)	1	0.040	-	-	stripe

Following is the pattern statement definition from the available values:

```
{ PATTERN pat0 { TYPE FOLLOWPIN } }
{ PATTERN pat1 { TYPE STAPLE } { DIRECTION VERTICAL }
  { WIDTH 0.032 0.032 } { SPACING 0.420 } { LENGTH 0.580 0.580 } }
{ PATTERN pat2 { TYPE STAPLE } { DIRECTION HORIZONTAL }
  { WIDTH 0.022 0.022 } { SPACING 0.482 } { LENGTH 0.600 0.600 } }
{ PATTERN pat3 { TYPE STAPLE } { DIRECTION VERTICAL }
  { WIDTH 0.022 0.022 } { SPACING 0.400 } { LENGTH 0.620 0.620 } }
{ PATTERN pat4 { TYPE STAPLE } { DIRECTION HORIZONTAL }
  { WIDTH 0.032 } { LENGTH 0.600 0.600 } }
{ PATTERN pat5 { TYPE STRIPE } { DIRECTION VERTICAL }
  { WIDTH 0.032 0.032 } { SPACING 0.320 } }
{ PATTERN pat6 { TYPE STRIPE } { DIRECTION HORIZONTAL } { WIDTH 0.040 } }
```

The following snippet presents instantiation of pattern template in the PG routing flow design:

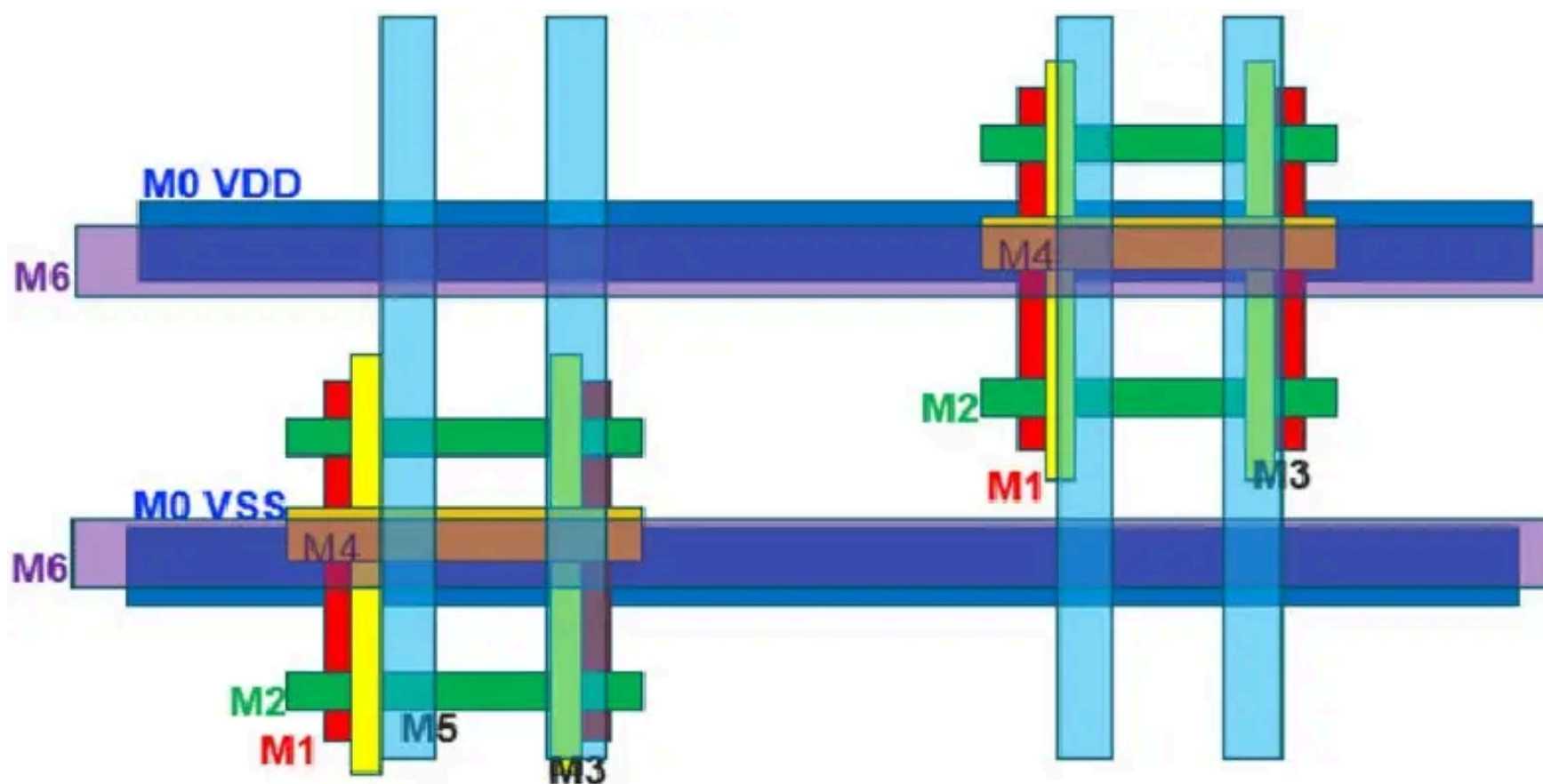
```
{ REGION { COREAREA }
  { LAYER M0      { METAL m0_vss pat0 { NET vss } }
    { METAL m0_vdd pat0 { NET vdd } } }
  { LAYER M1      { METAL m1_vss pat1 { NET vss } ... }
    { METAL m1_vdd pat1 { NET vdd } ... } }
```

```

{ LAYER M2      { METAL m2_vss pat2 { NET vss } ... }
                  { METAL m2_vdd pat2 { NET vdd } ... } }
{ LAYER M3      { METAL m3_vss pat3 { NET vss } ... }
                  { METAL m3_vdd pat3 { NET vdd } ... } }
{ LAYER M4      { METAL m4_vss pat4 { NET vss } ... }
                  { METAL m4_vdd pat4 { NET vdd } ... } }
{ LAYER M5      { METAL m5_vss pat5 { NET vss } ... }
                  { METAL m5_vdd pat5 { NET vdd } ... } }
{ LAYER M6      { METAL m6_vss pat6 { NET vss } ... }
                  { METAL m6_vdd pat6 { NET vdd } ... } } }

```

Following is the PG routing flow developed by Flash PG:



Completed PSDL Region Syntax

Following is the completed PSDL region syntax in this example:

```

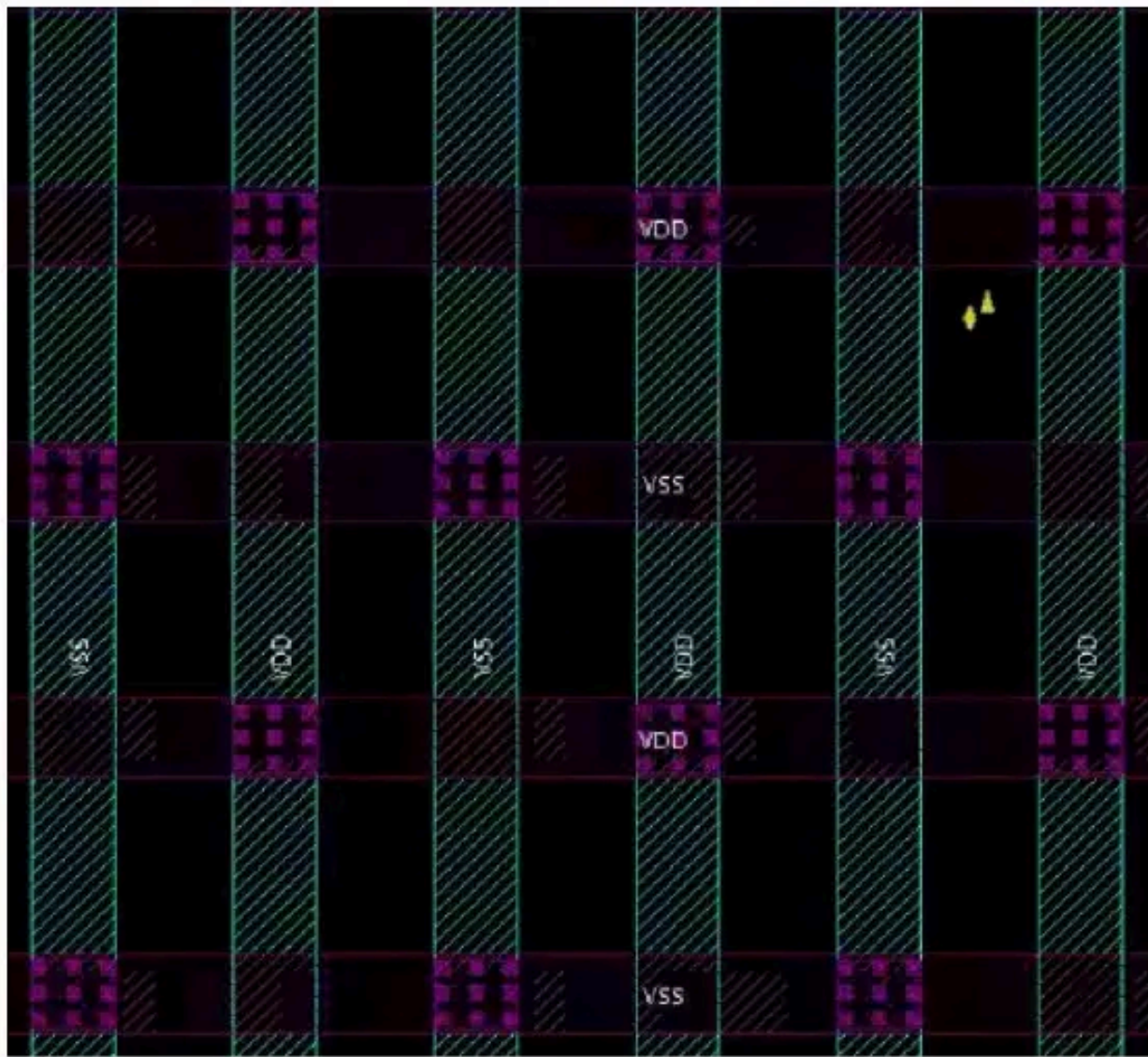
REGION
{ LAYER M14 { ONGRIDONLY } { METAL m14 vss m14 H { NET VSS } { OFFSET * 2.590 } { STEPDISTANCE * 13.312 } }
{ METAL m14 vdd m14 H { NET VDD } { OFFSET * 9.246 } { STEPDISTANCE * 13.312 } } }
{ LAYER M13 { METAL m13 vss m13 V { NET VSS } { OFFSET 1.386 * } { STEPDISTANCE 9.792 * } }
{ METAL m13 vdd m13 V { NET VDD } { OFFSET 6.282 * } { STEPDISTANCE 9.792 * } } }
{ LAYER M12 { METAL m12 vss m12 H { NET VSS } { OFFSET * 0.670 } { STEPDISTANCE * 3.328 } }
{ METAL m12 vdd m12 H { NET VDD } { OFFSET * 2.308 } { STEPDISTANCE * 3.328 } } }
{ LAYER M11 { ONGRIDONLY } { METAL m11 vss m11 V { NET VSS } { OFFSET 0.258 * } { STEPDISTANCE 1.224 * } }
{ METAL m11 vdd m11 V { NET VDD } { OFFSET 0.874 * } { STEPDISTANCE 1.224 * } } }
{ LAYER M10 { ONGRIDONLY } { METAL m10 vss m10 H { NET VSS } { OFFSET 0.181 0.532 } { STEPDISTANCE 1.224 0.840 } }
{ METAL m10 vdd m10 H { NET VDD } { OFFSET 0.797 0.797 } { STEPDISTANCE 1.224 0.840 } } }
{ LAYER M9 { ONGRIDONLY } { METAL m9 vss m9 V { NET VSS } { OFFSET 0.258 0.3345 } { STEPDISTANCE 1.224 0.840 } }
{ METAL m9 vdd m9 V { NET VDD } { OFFSET 0.874 0.6385 } { STEPDISTANCE 1.224 0.840 } } }
{ LAYER M8 { ONGRIDONLY } { METAL m8 vss m8 H { NET VSS } { OFFSET 0.181 0.532 } { STEPDISTANCE 1.224 0.840 } }
{ METAL m8 vdd m8 H { NET VDD } { OFFSET 0.723 0.797 } { STEPDISTANCE 1.224 0.840 } } }
{ LAYER M7 { ONGRIDONLY } { METAL m7 vss m7 V { NET VSS } { OFFSET 0.258 0.3345 } { STEPDISTANCE 1.224 0.840 } }
{ METAL m7 vdd m7 V { NET VDD } { OFFSET 0.874 0.6385 } { STEPDISTANCE 1.224 0.840 } } }
{ LAYER M6 { ONGRIDONLY } { METAL m6 vss m6 H { NET VSS } { OFFSET 0.181 0.532 } { STEPDISTANCE 1.224 0.840 } }
{ METAL m6 vdd m6 H { NET VDD } { OFFSET 0.723 0.797 } { STEPDISTANCE 1.224 0.840 } } }
{ LAYER M5 { ONGRIDONLY } { METAL m5 vss m5 V { NET VSS } { OFFSET 0.2095 0.265 } { STEPDISTANCE 1.224 3.360 } }
{ METAL m5 vdd m5 V { NET VDD } { OFFSET 0.9015 0.475 } { STEPDISTANCE 1.224 3.360 } } }
{ LAYER M4 { ONGRIDONLY } { METAL m4 vss m4 H { NET VSS } { OFFSET 0.159 0.410 } { STEPDISTANCE 1.224 0.840 } }
{ METAL m4 vdd m4 H { NET VDD } { OFFSET 0.771 0.620 } { STEPDISTANCE 1.224 0.840 } } }
{ LAYER M3 { ONGRIDONLY } { METAL m3 vss m3 V { NET VSS } { OFFSET 0.2025 0.160 } { STEPDISTANCE 1.224 2.520 } }
{ METAL m3 vdd m3 V { NET VDD } { OFFSET 0.8385 0.370 } { STEPDISTANCE 1.224 2.520 } } }
{ LAYER M2 { ONGRIDONLY } { METAL m2 vss m2 H { NET VSS } { OFFSET 0.1555 0.200 } { STEPDISTANCE 1.224 0.420 } }
{ METAL m2 vdd m2 H { NET VDD } { OFFSET 0.7675 0.410 } { STEPDISTANCE 1.224 0.420 } } }
{ LAYER M1 { ONGRIDONLY } { METAL m1 vss m1 V { NET VSS } { OFFSET 0.2705 * } { STEPDISTANCE 1.224 * } }
{ METAL m1 vdd m1 V { NET VDD } { OFFSET 0.8825 * } { STEPDISTANCE 1.224 * } } }
{ LAYER M0 { METAL m0 vss m0 H { NET VSS } }
{ METAL m0 vdd m0 H { NET VDD } } }

```

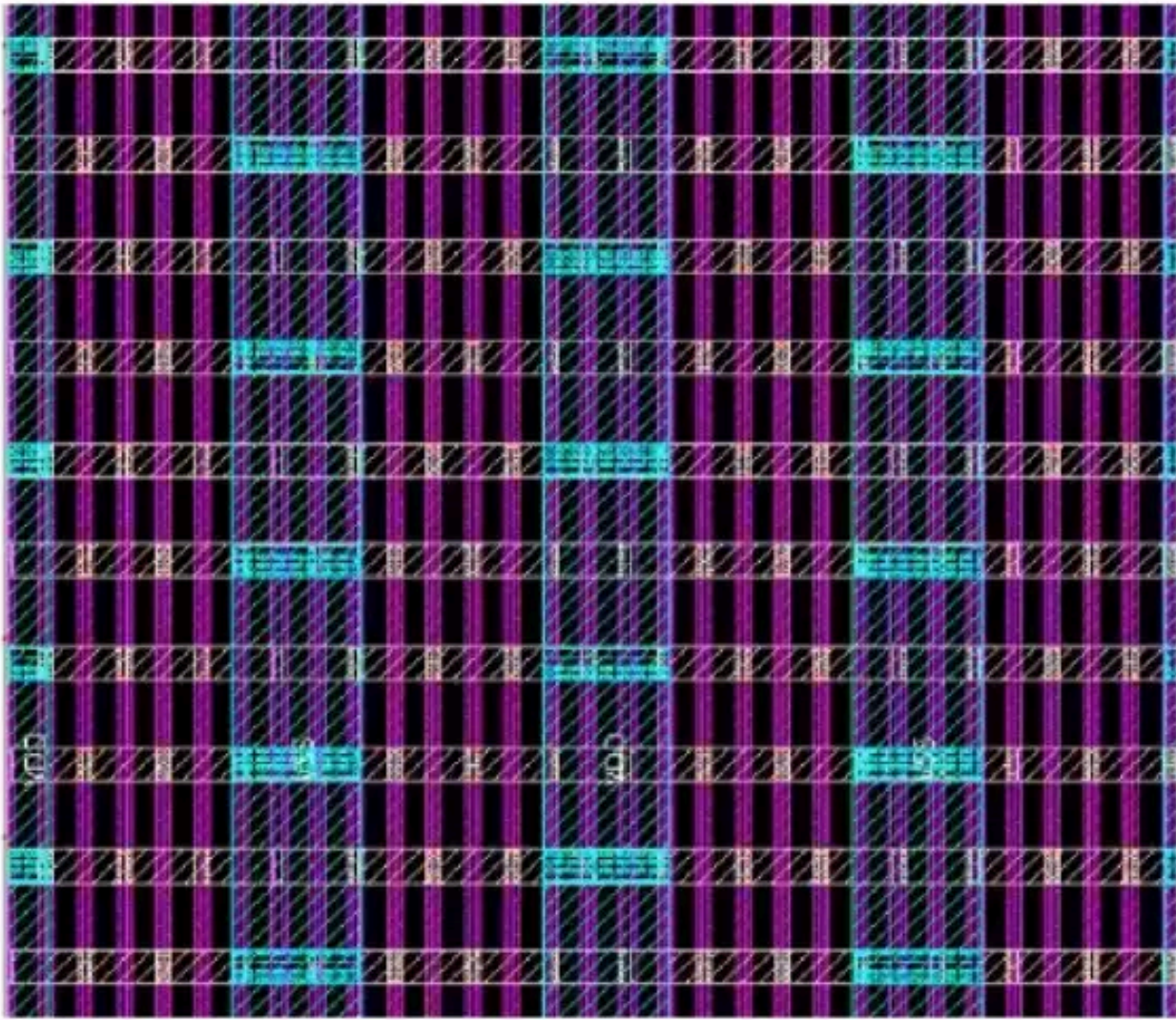
Resulting PG Mesh using PSDL

Following is the resulting PG mesh of different metal layer combination sequences using PSDL:

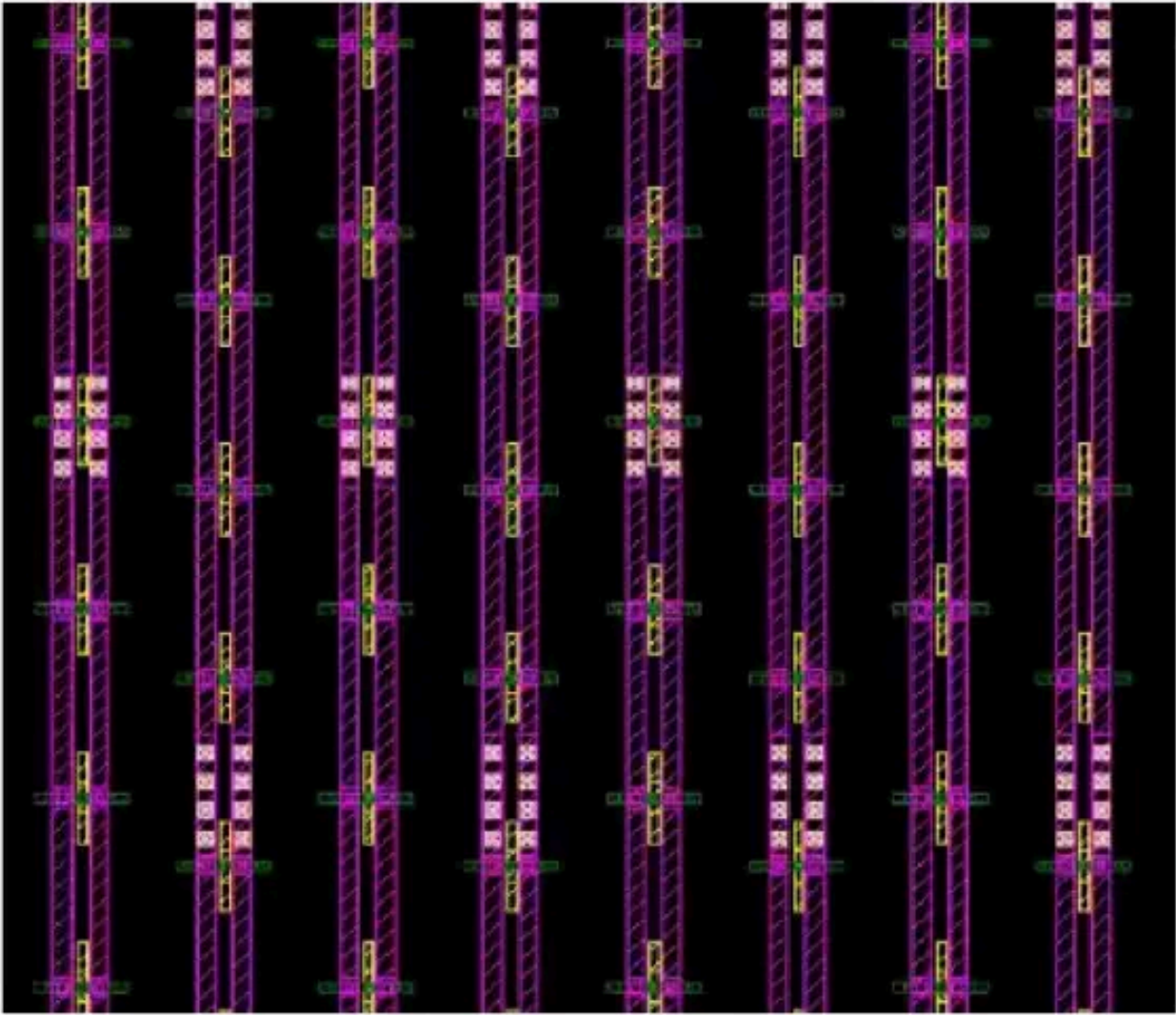
M13-VIA13-M14



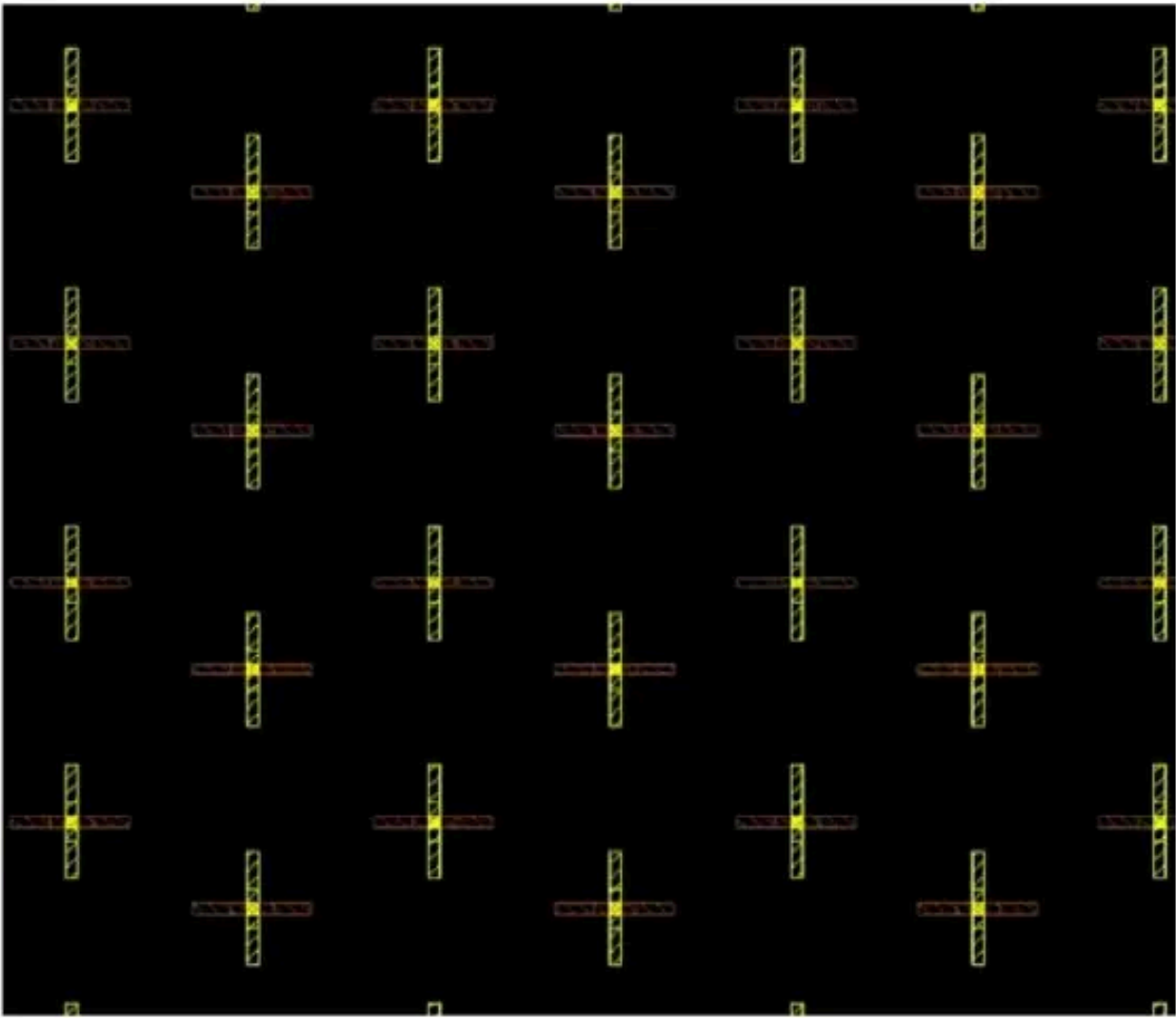
M11-VIA11-M12-VIA12-M13



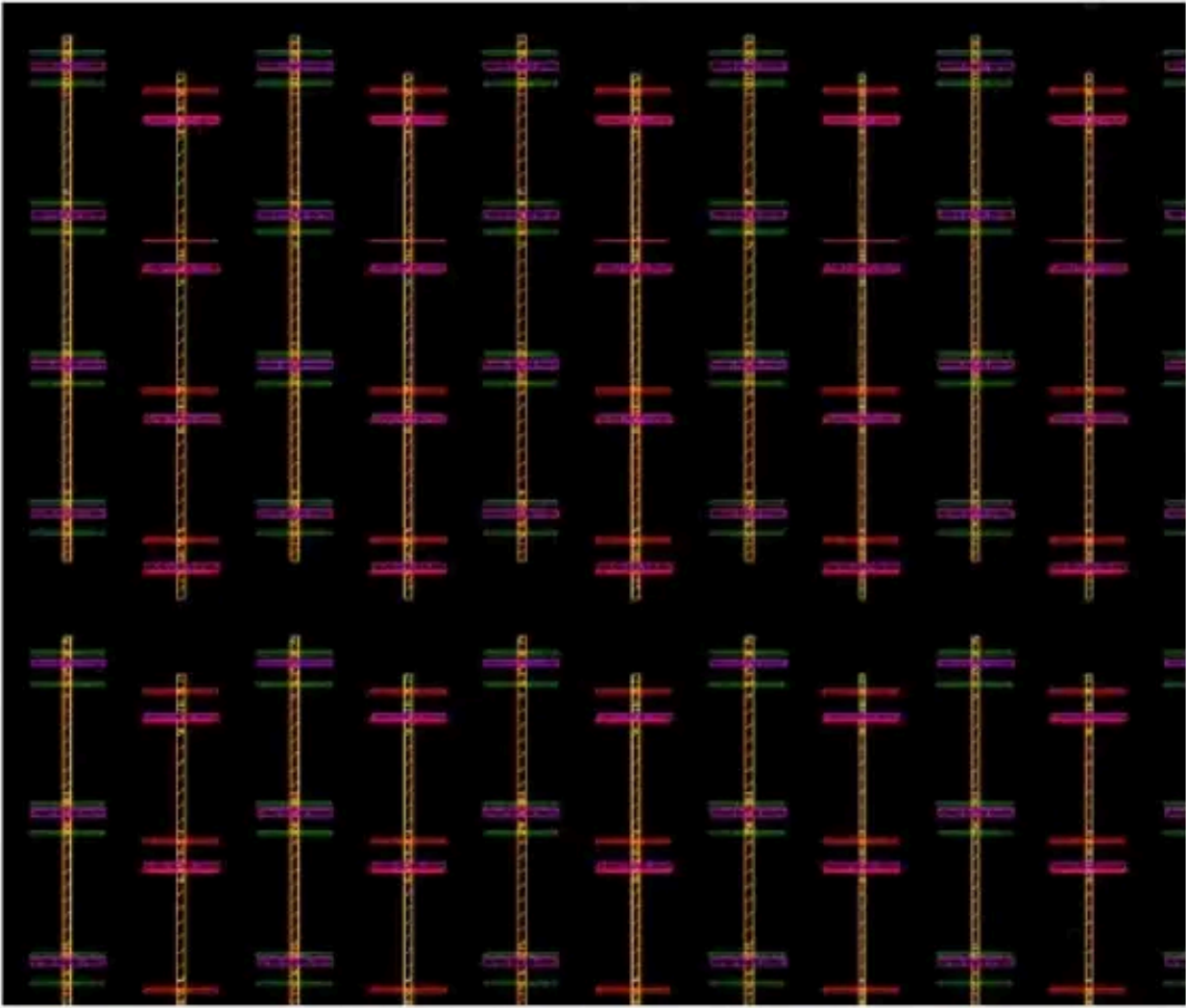
M9-VIA9-M10-VIA10-M11-VIA11



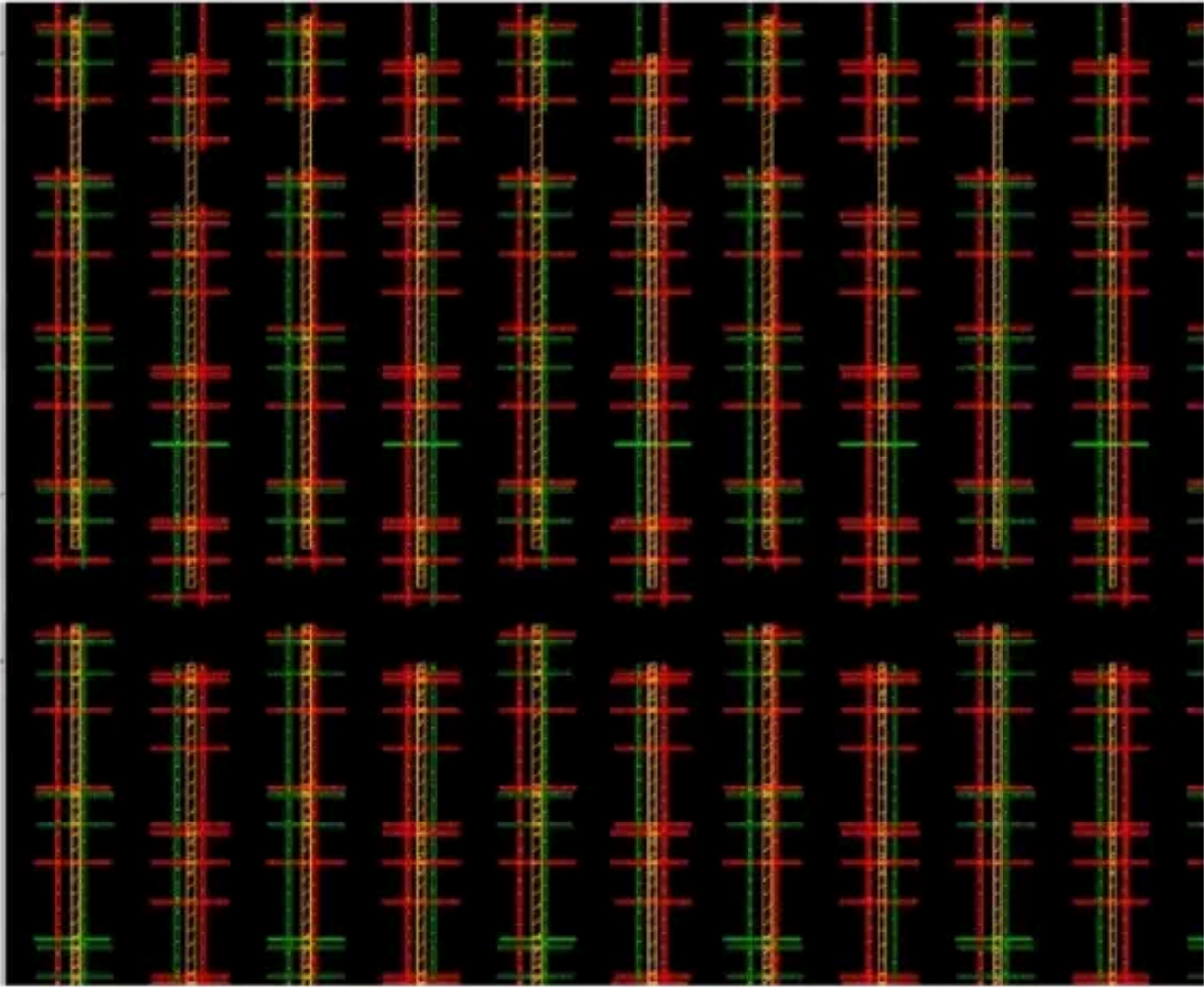
M6-VIA6-M7-VIA7-M8-VIA8-M9



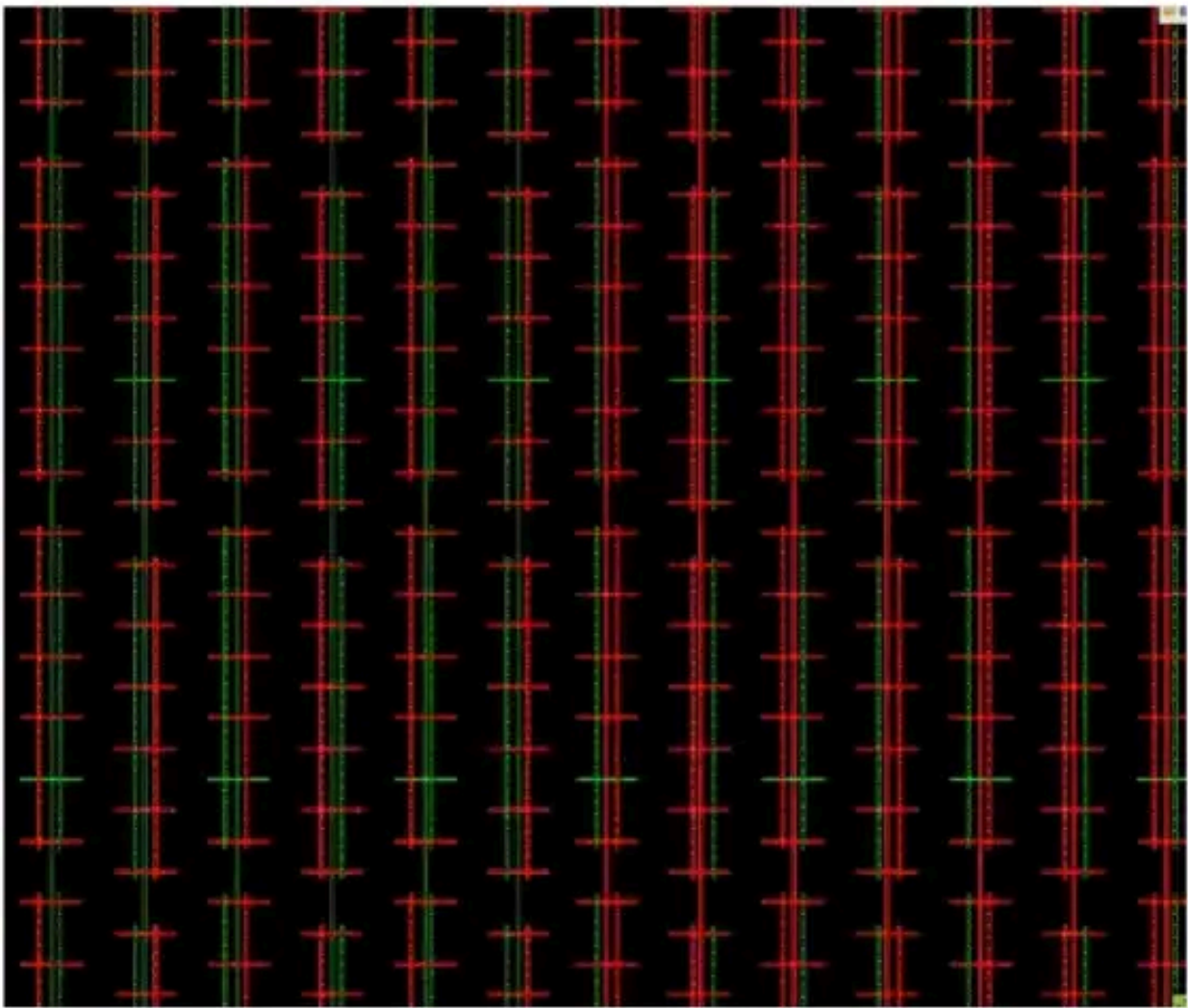
M4-VIA4-M5-VIA5-M6



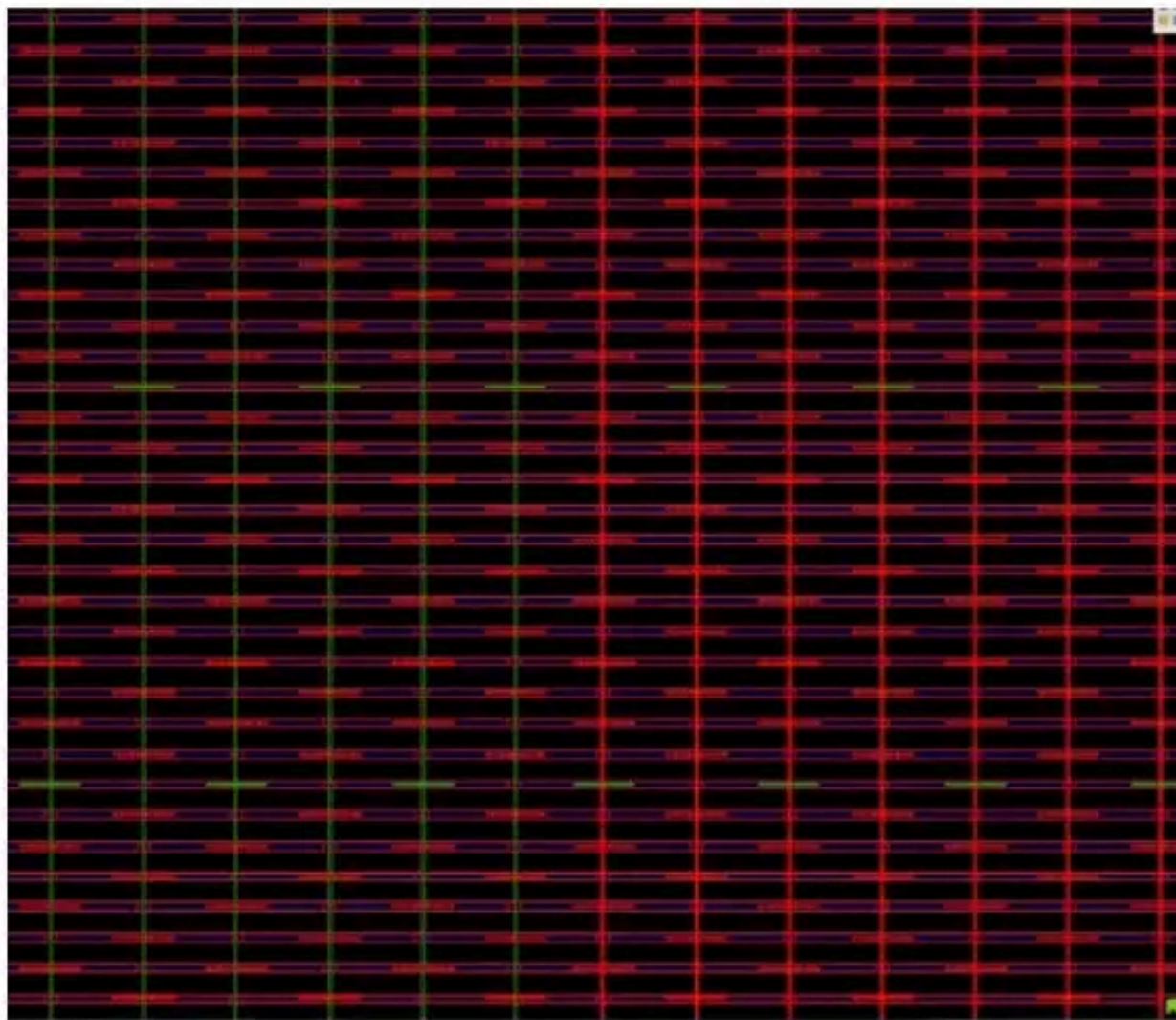
M2-VIA2-M3-VIA3-M4-VIA4-M5



M1-VIA1-M2-VIA2-M3



M0-VIA0-M1-VIA1-M2

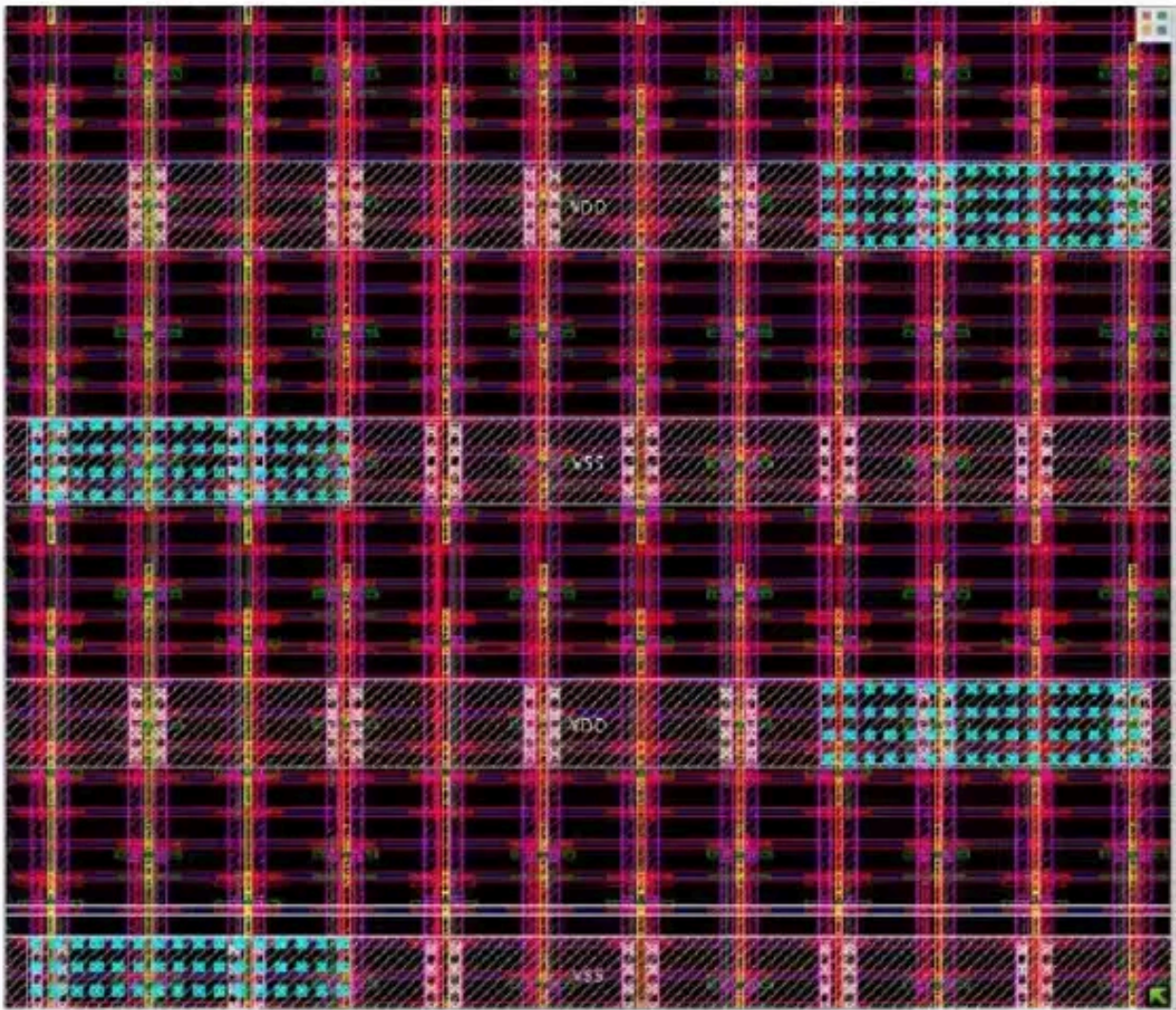


Single Variable Change for PG Structure Exploration

The following example presents the impact on the PG structure on modifying the metal pitch value from 1.224 to 1.836:

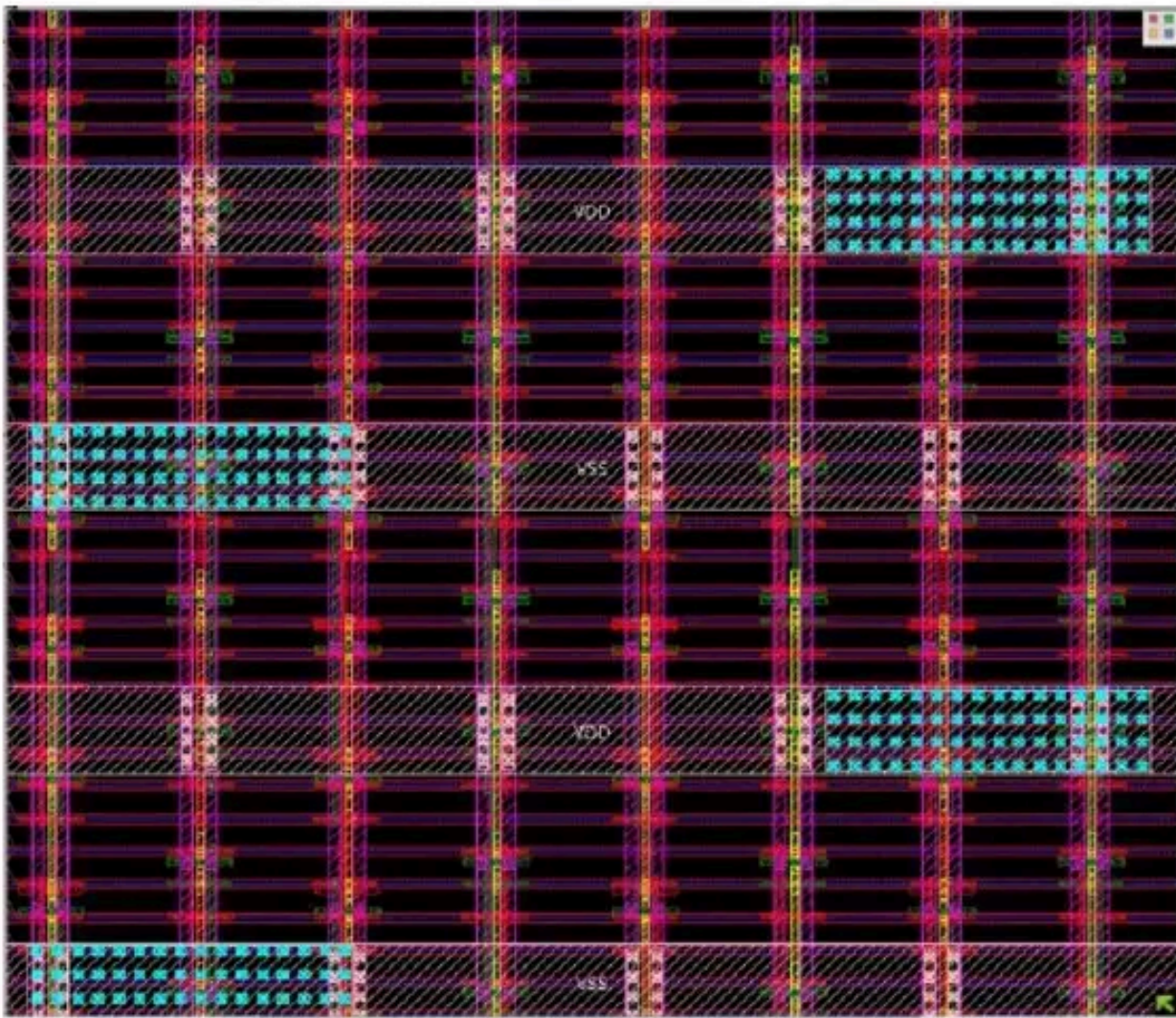
PG structure before modifying the metal pitch value:

```
source floorplan.enc  
  
set m11_pitch 1.224  
  
route_pg -fast -psdl_file N5_var.psdl
```



PG structure after modifying the metal pitch value:

```
source floorplan.enc  
  
set m11_pitch 1.836  
  
route_pg -fast -psdl_file N5_var.psdl
```

Related Topics

- [route_pg](#)
- [Components of a PSDL File](#)
- [PG Structure Description Language](#)